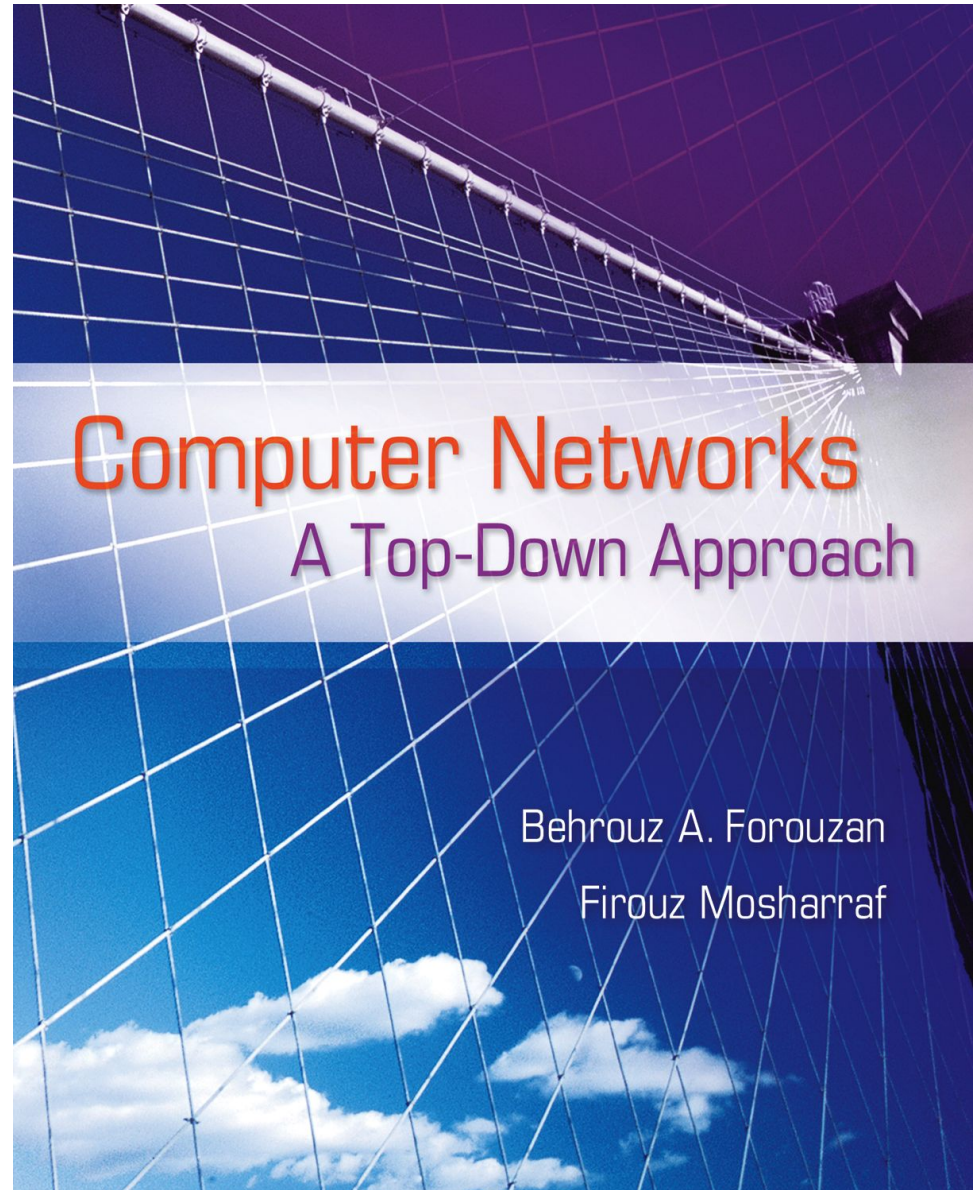
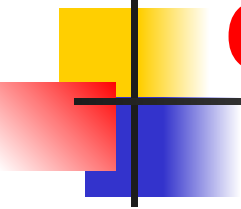


Chapter 4

Network Layer





Chapter 4: Outline

4.1 *INTRODUCTION*

4.2 *NETWORK-LAYER PROTOCOLS*

4.3 *UNICAST ROUTING*

4.4 *MULTICAST ROUTING*

4.5 *NEXT GENERATION IP*



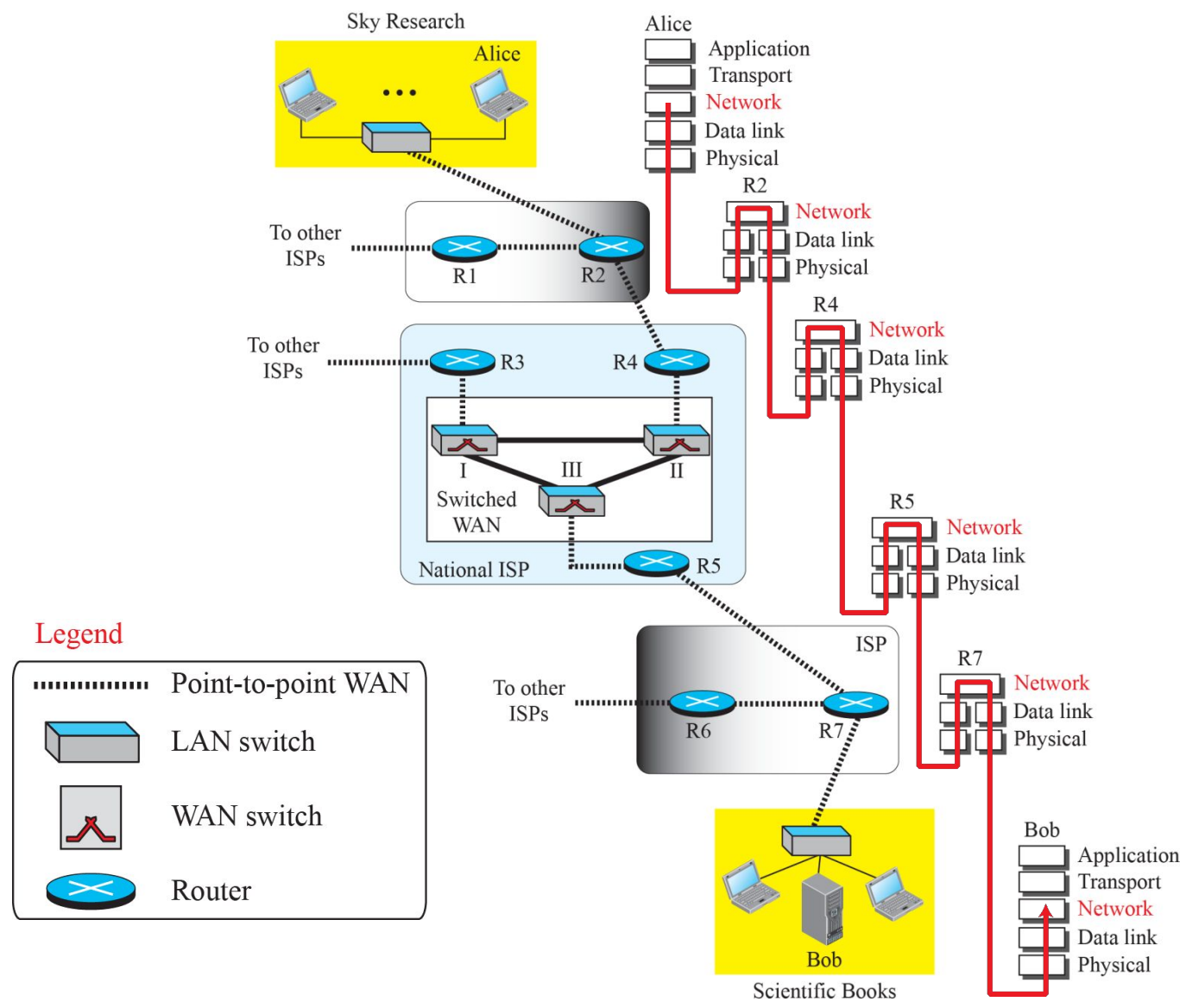
Chapter 4: Objective

- ☐ *We first discuss services that can be provided at the network layer: packetizing, routing, and forwarding.*
- ☐ *We then discuss the network layer at the TCP/IP suite: IPv4 and ICMPv4. We also discuss IPv4 addressing and related issues.*
- ☐ *We then concentrate on the unicast routing and unicast routing protocols.*
- ☐ *We then move to multicasting and multicast routing protocols protocol.*
- ☐ *We finally discuss the new generation of network-layer protocols, IPv6 and ICMPv6.*

4-1 INTRODUCTION

Figure 4.1 shows the communication between Alice and Bob at the network layer. This is the same scenario we used in Chapters 2 and 3 to show the communication at the application and the transport layers, respectively.

Figure 4.1: *Communication at the network layer*





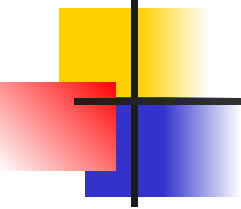
4.1.1 Network-Layer Services

Before discussing the network layer in the Internet today, let's briefly discuss the network-layer services that, in general, are expected from a network-layer protocol.

☐ *Packetizing*

☐ *Routing*

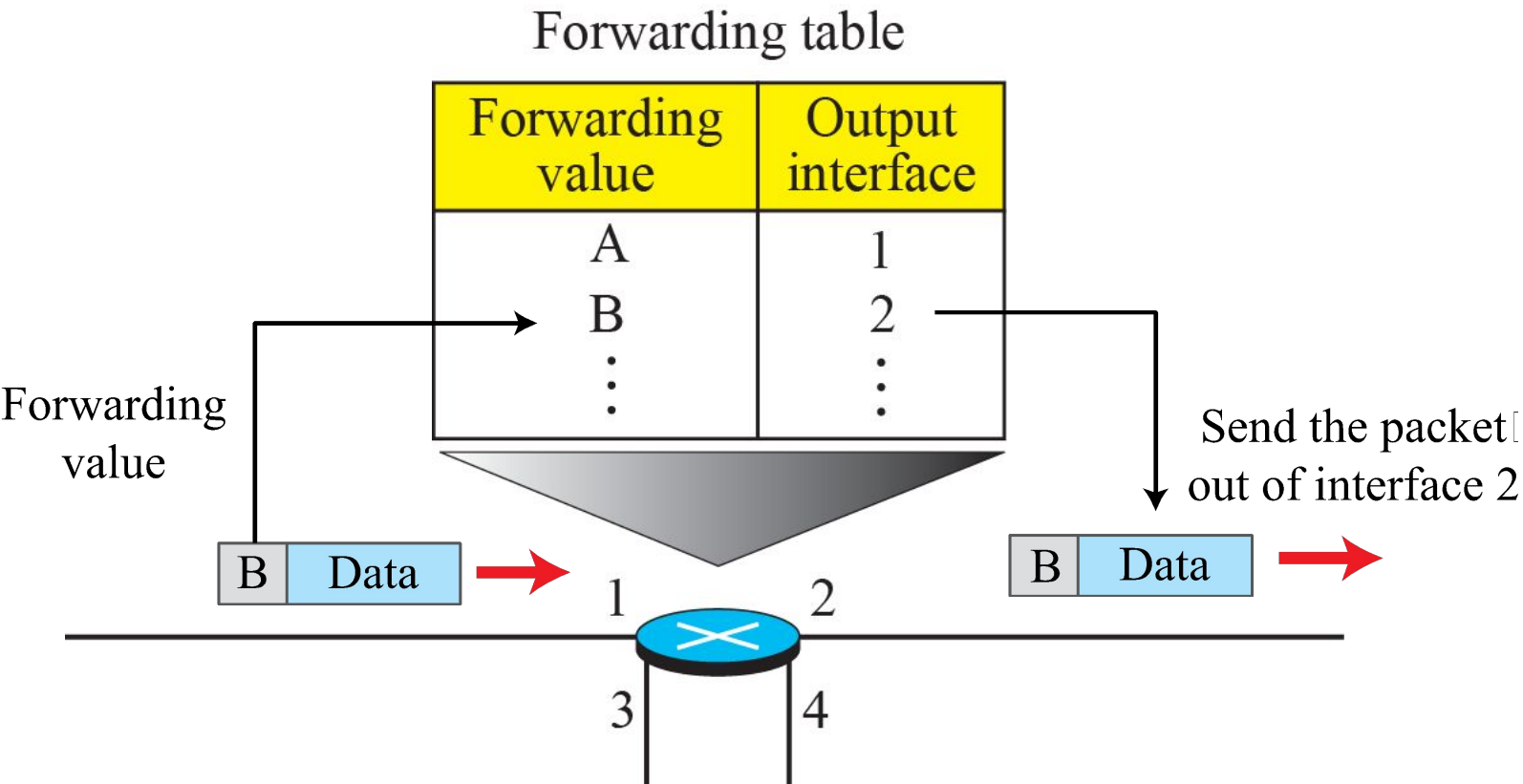
☐ *Forwarding*



4.1.1 (continued)

- ☐ *Error Control*
- ☐ *Flow Control*
- ☐ *Congestion Control*
- ☐ *Quality of Service*
- ☐ *Security*

Figure 4.2: Forwarding process





4.1.2 Packet Switching

From the discussion of routing and forwarding in the previous section, we infer that a kind of switching occurs at the network layer. A router, in fact, is a switch that creates a connection between an input port and an output port (or a set of output ports), just as an electrical switch connects the input to the output to let electricity flow.

□ *Datagram Approach*

□ *Virtual-Circuit Approach*

◆ *Setup Phase*

◆ *Data-Transfer Phase*

◆ *Teardown Phase*

Figure 4.3: *A connectionless packet-switched network*

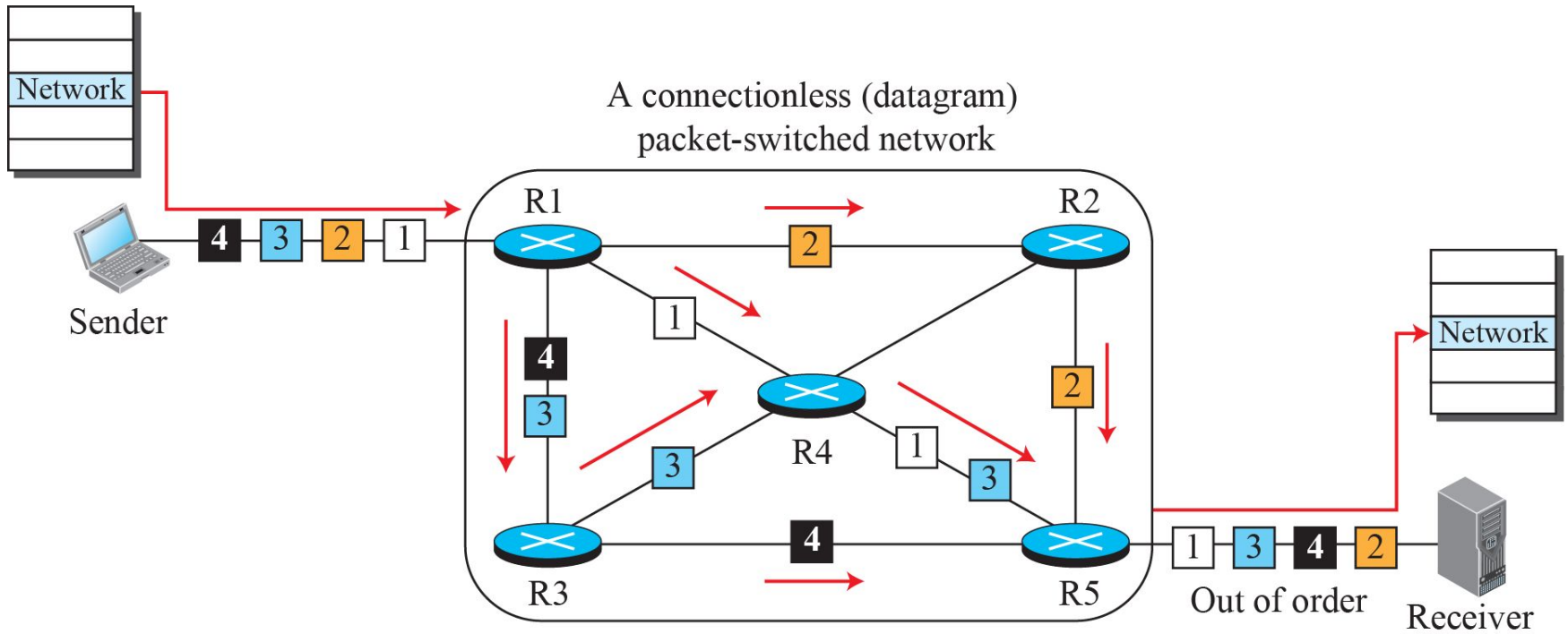


Figure 4.4: Forwarding process in a router when used in a connectionless network

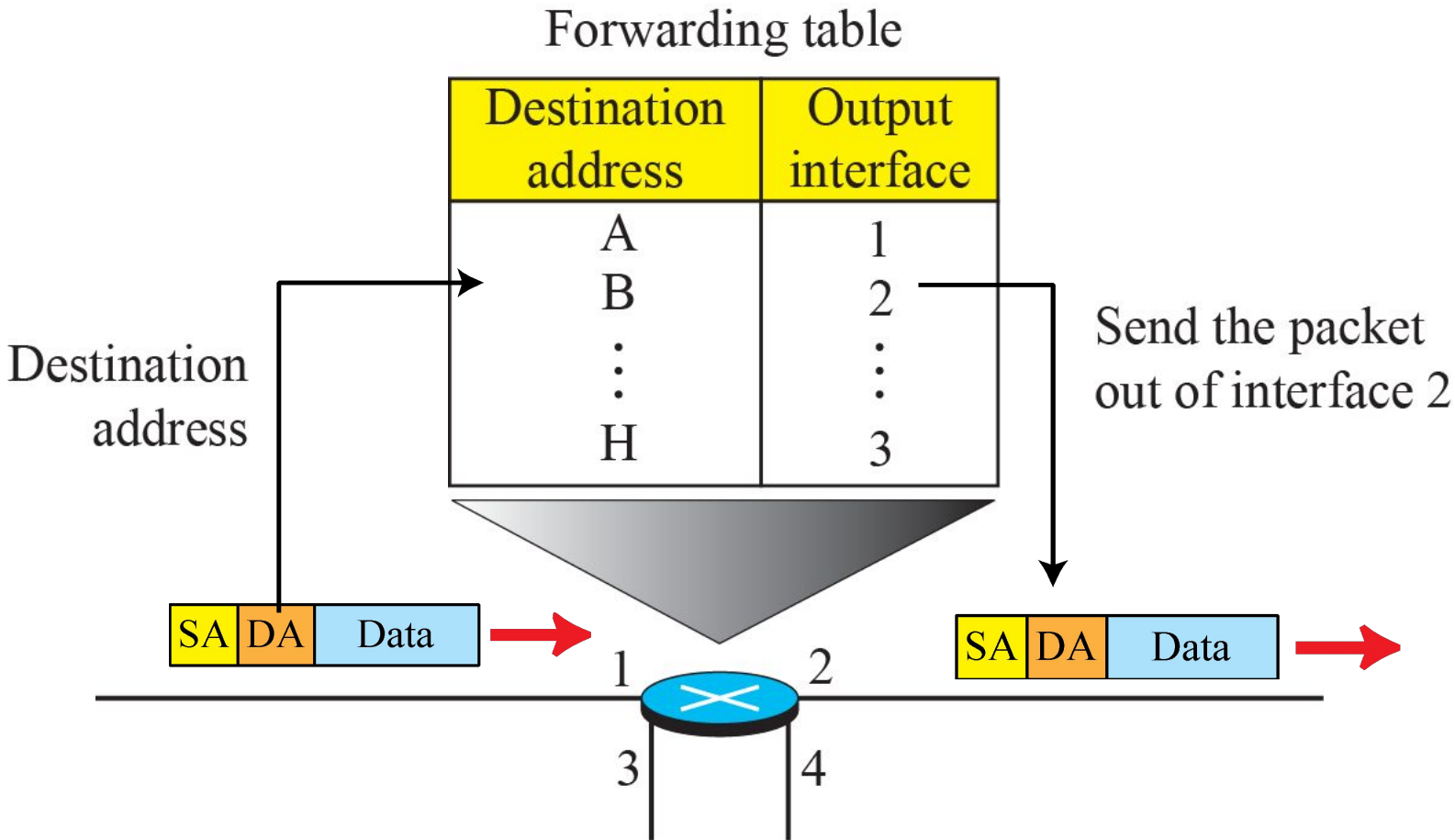


Figure 4.5: *A virtual-circuit packet-switched network*

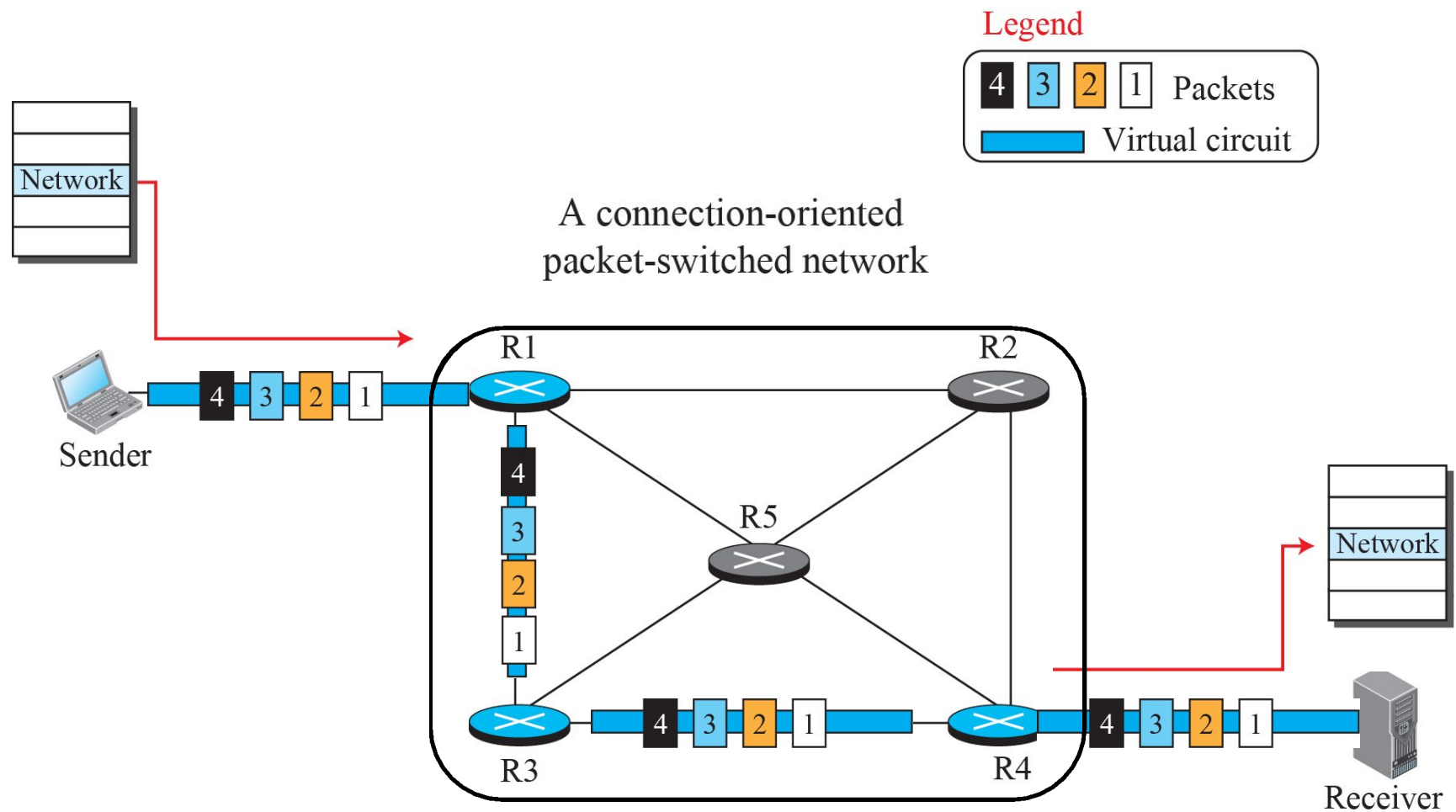


Figure 4.6: Forwarding process in a router when used in a virtual circuit network

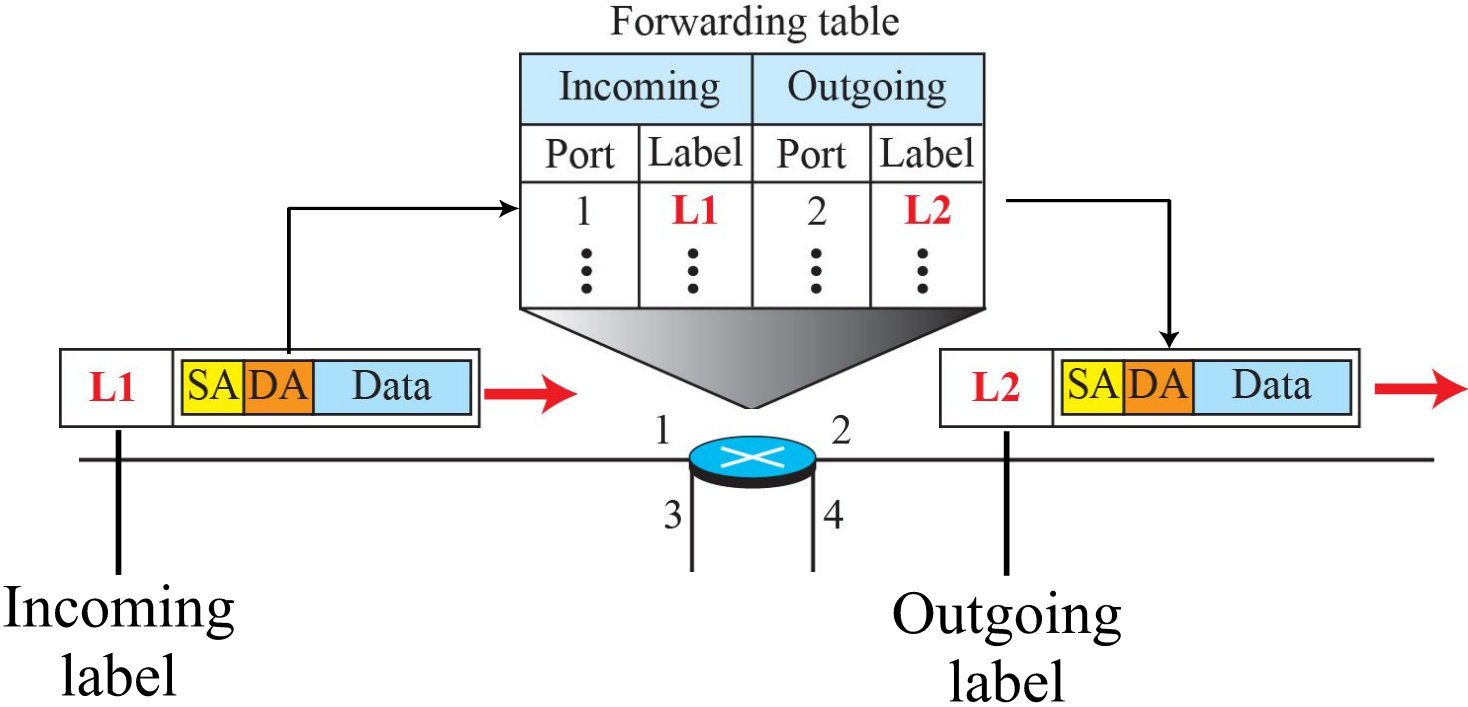


Figure 4.7: *Sending request packet in a virtual-circuit network*

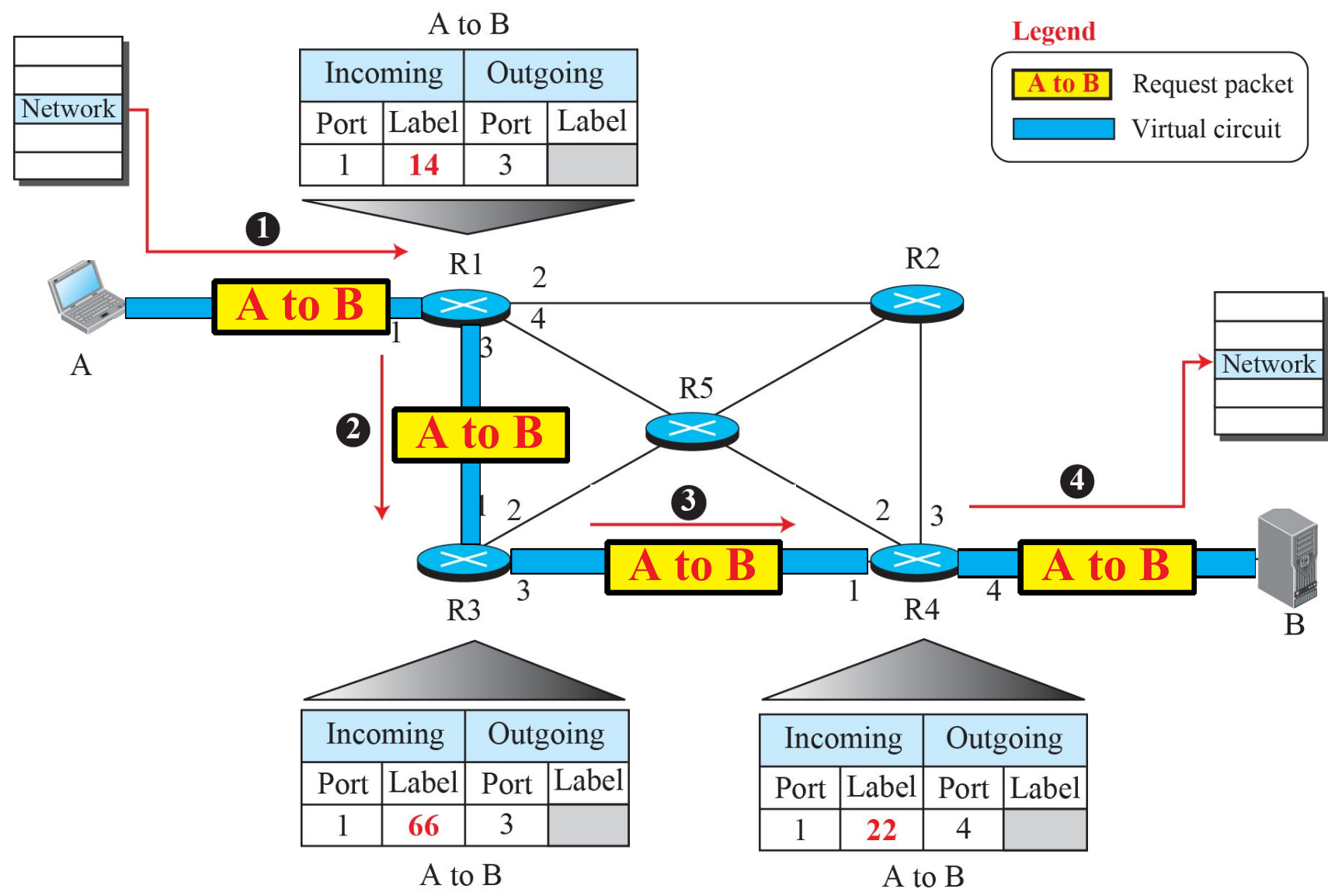


Figure 4.8: *Sending acknowledgments in a virtual-circuit network*

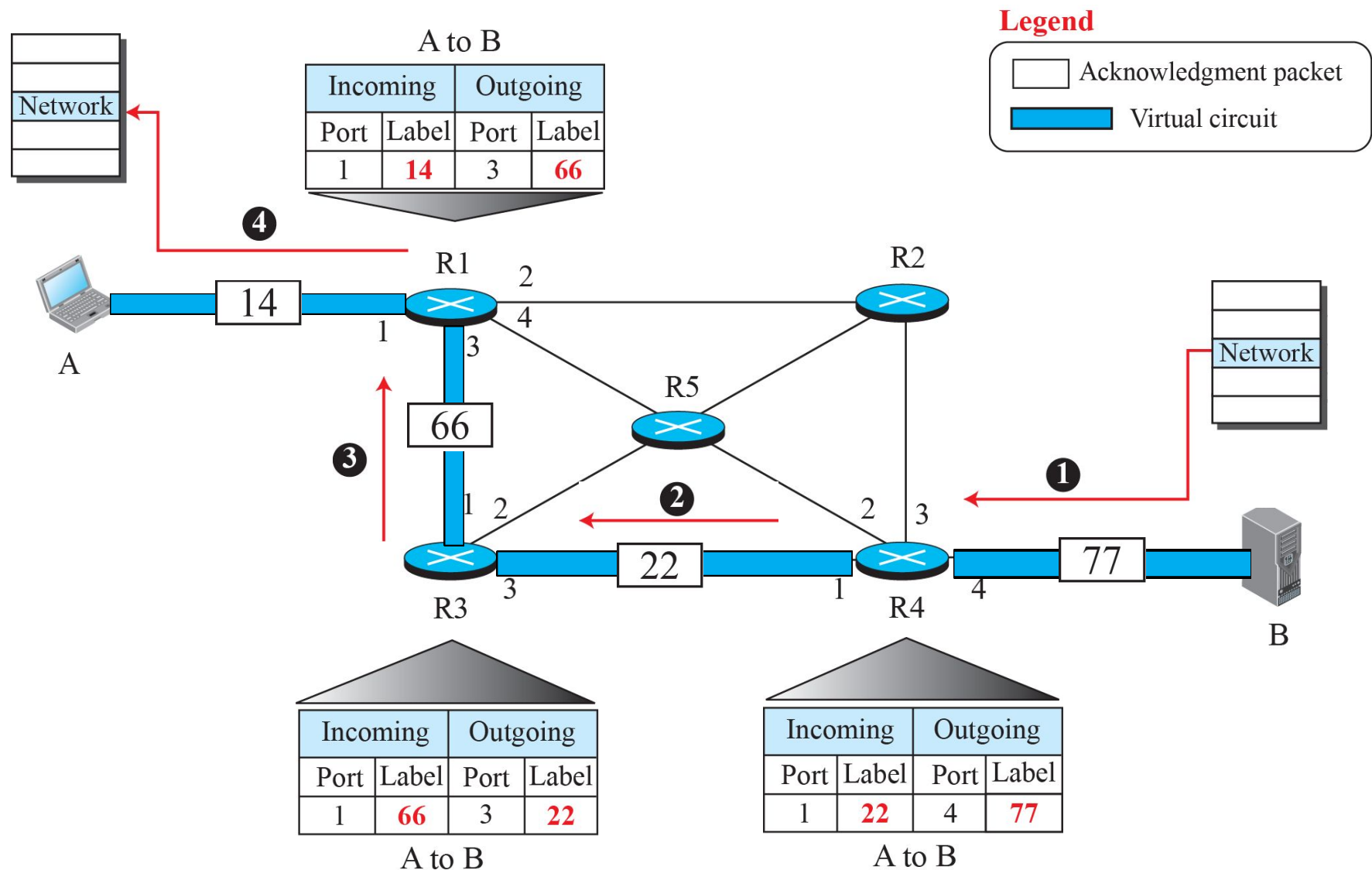
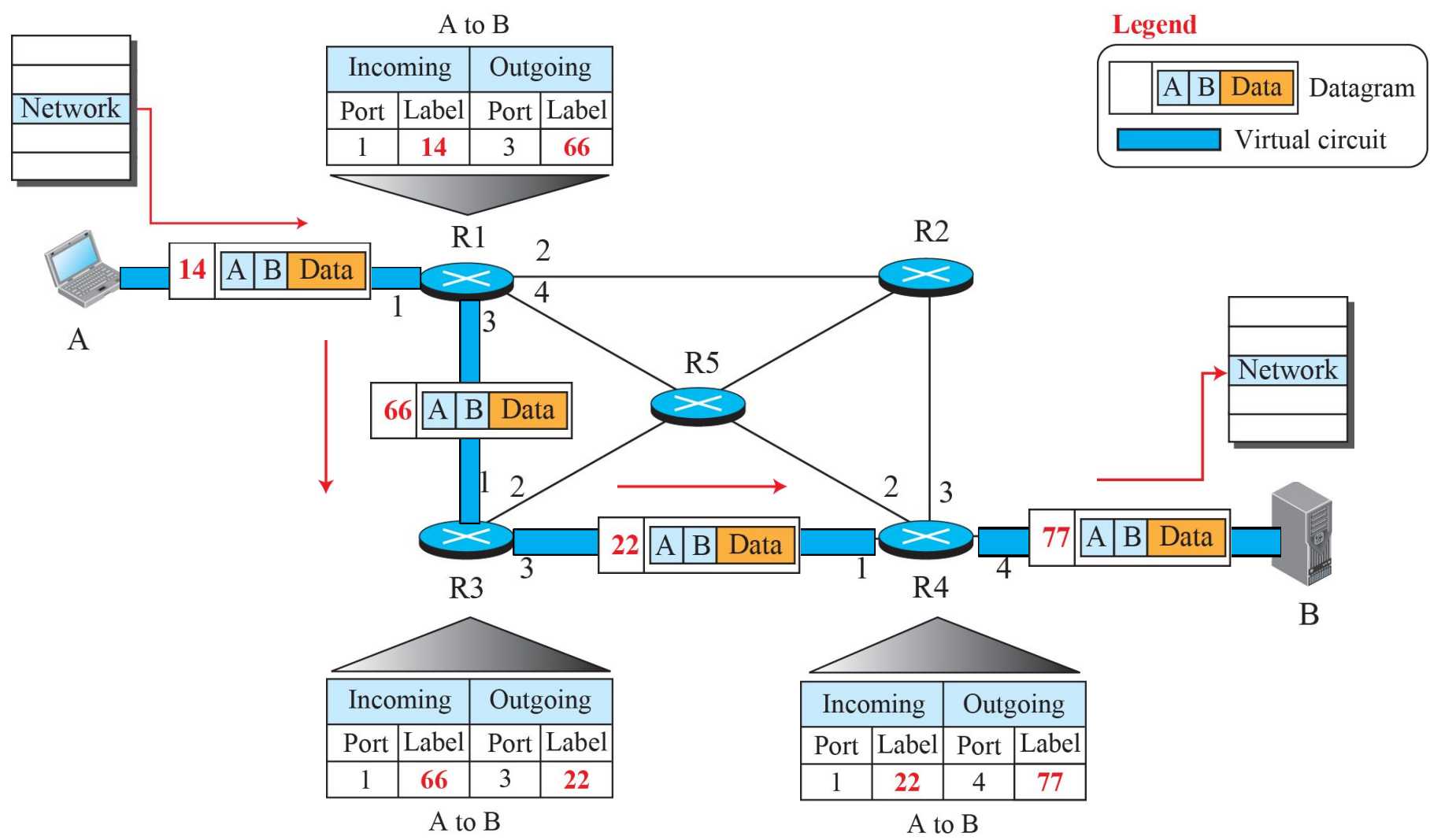


Figure 4.8: *Sending acknowledgments in a virtual-circuit network*





4.1.3 Network-Layer Performance

The upper-layer protocols that use the service of the network layer expect to receive an ideal service, but the network layer is not perfect. The performance of a network can be measured in terms of delay, throughput, and packet loss. We first define these three terms in a packet-switched network before we discuss their effects on performance.



4.1.3 (continued)

☐ *Delay*

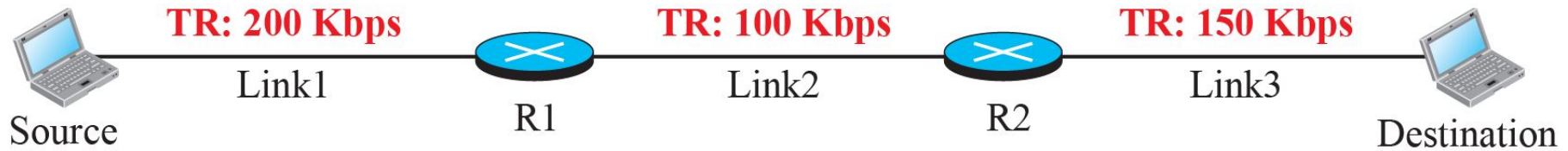
- ❖ *Transmission Delay*
- ❖ *Propagation Delay*
- ❖ *Processing Delay*
- ❖ *Queuing Delay*
- ❖ *Total Delay*

☐ *Throughput*

☐ *Packet Loss*

Figure 4.10: *Throughput in a path with three links in a series*

TR: Transmission rate



a. A path through three links



b. Simulation using pipes

Figure 4.11: *A path through the Internet backbone*

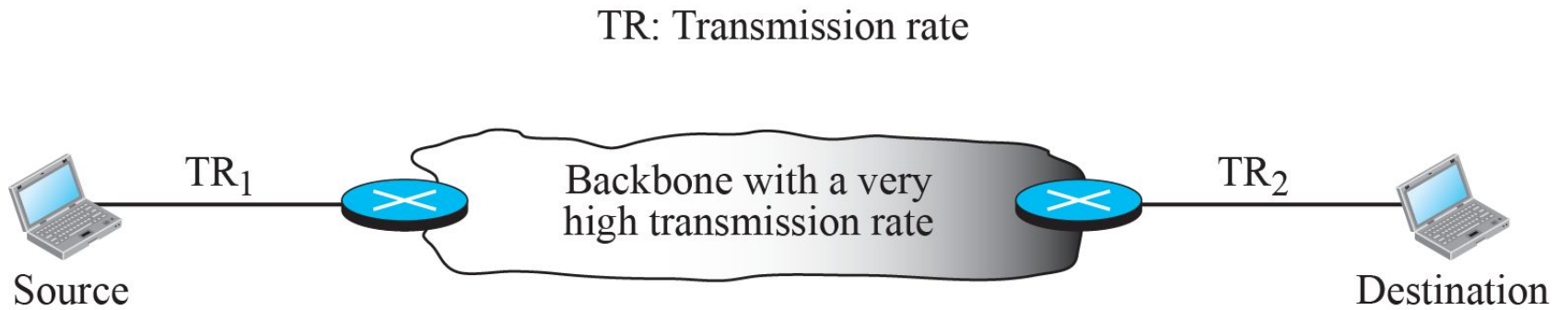
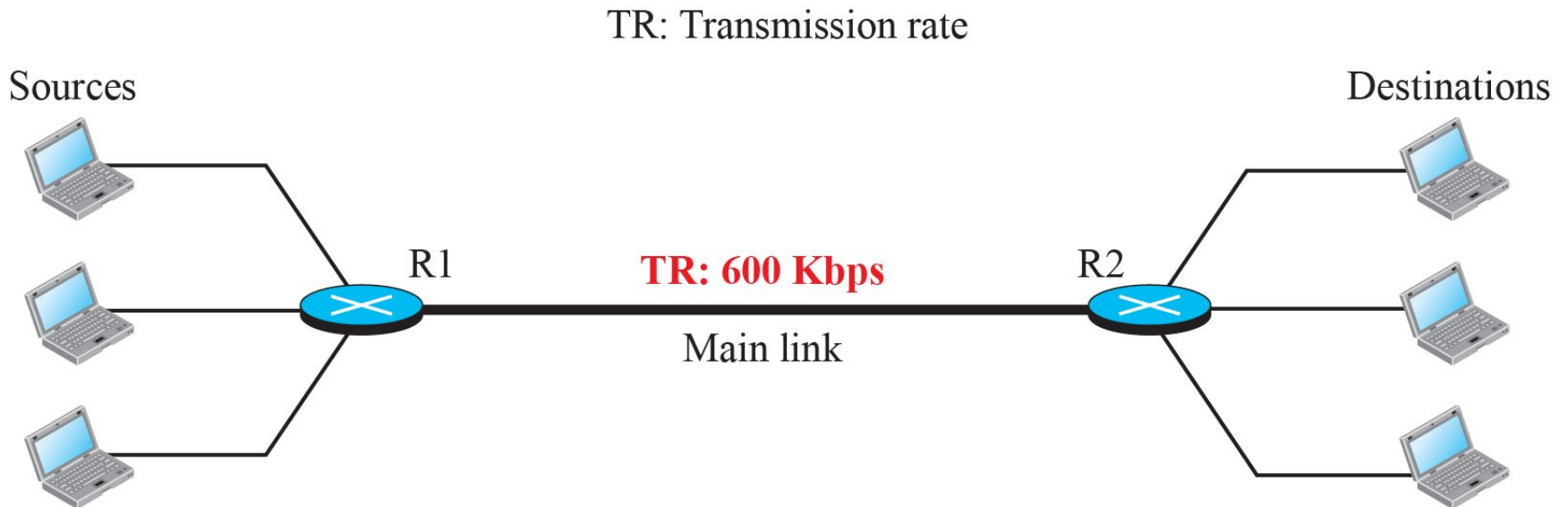


Figure 4.12: *Effect of throughput in shared links*





4.1.4 Network-Layer Congestions

In Chapter 3, we discussed congestion at the transport layer. Although congestion at the network layer is not explicitly addressed in the Internet model, the study of congestion at this layer may help us to better understand the cause of congestion at the transport layer and find possible remedies to be used at the network layer. Congestion at the network layer is related to two issues, throughput and delay, which we discussed in the previous section.

□ Congestion Control

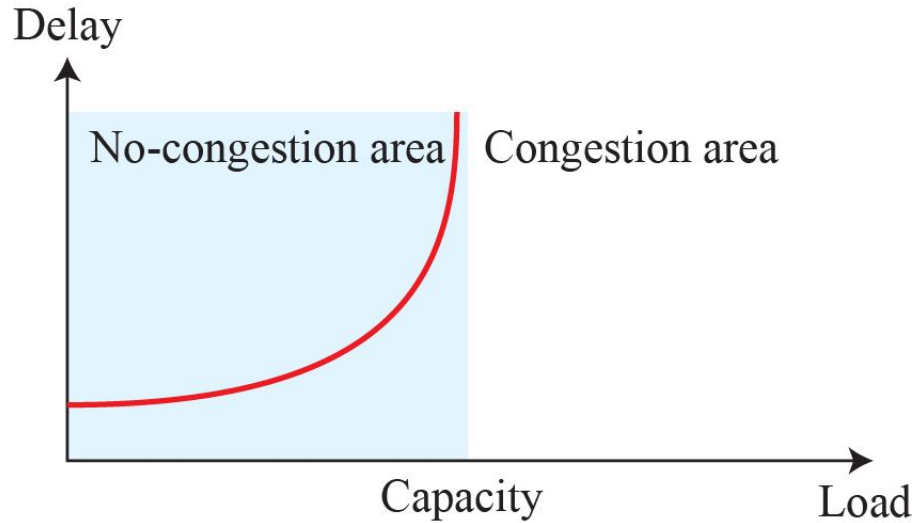
◆ Open-Loop Congestion Control

- *Retransmission Policy*
- *Window Policy*
- *Acknowledgment Policy*
- *Discarding Policy*
- *Admission Policy*

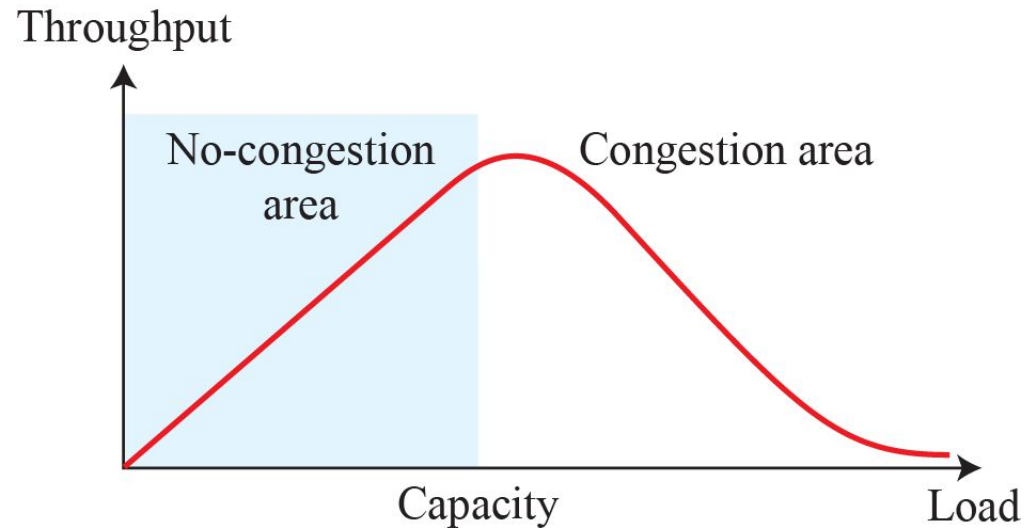
◆ Closed-Loop Congestion Control

- *Backpressure*
- *Choke Packet*
- *Implicit Signaling*
- *Explicit Signaling*

Figure 4.13: *Packet delay and throughput as functions of load*



a. Delay as a function of load



b. Throughput as a function of load

Figure 4.14: *Backpressure method for alleviating congestion*

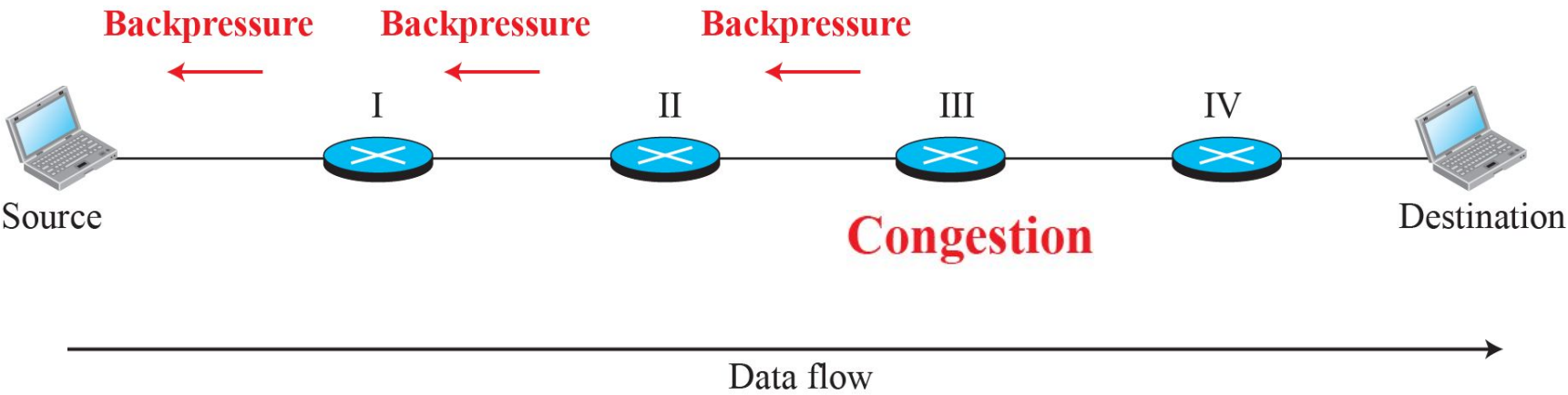
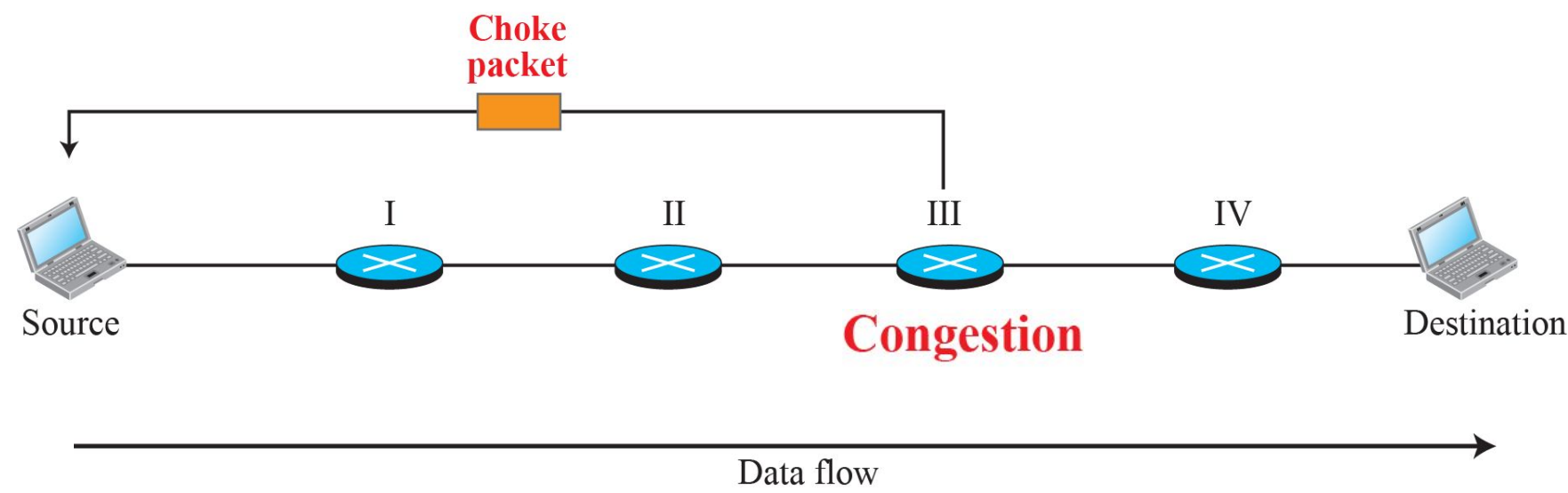


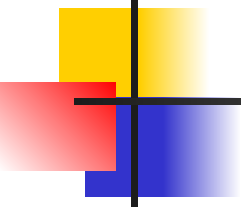
Figure 4.15: Choke packet





4.1.5 *Structure of A Router*

In our discussion of forwarding and routing, we represented a router as a black box that accepts incoming packets from one of the input ports (interfaces), uses a forwarding table to find the output port from which the packet departs, and sends the packet from this output port. In this section we open the black box and look inside. However, our discussion won't be very detailed; entire books have been written about routers. We just give an overview to the reader.



4.1.5 (continued)

Components

- ◆ *Input Ports*
- ◆ *Output Ports*
- ◆ *Routing Processor*
- ◆ *Switching Fabrics*
 - *Crossbar Switch*
 - *Banyan Switch*
 - *Batcher-Banyan Switch*

Figure 4.16: Router components

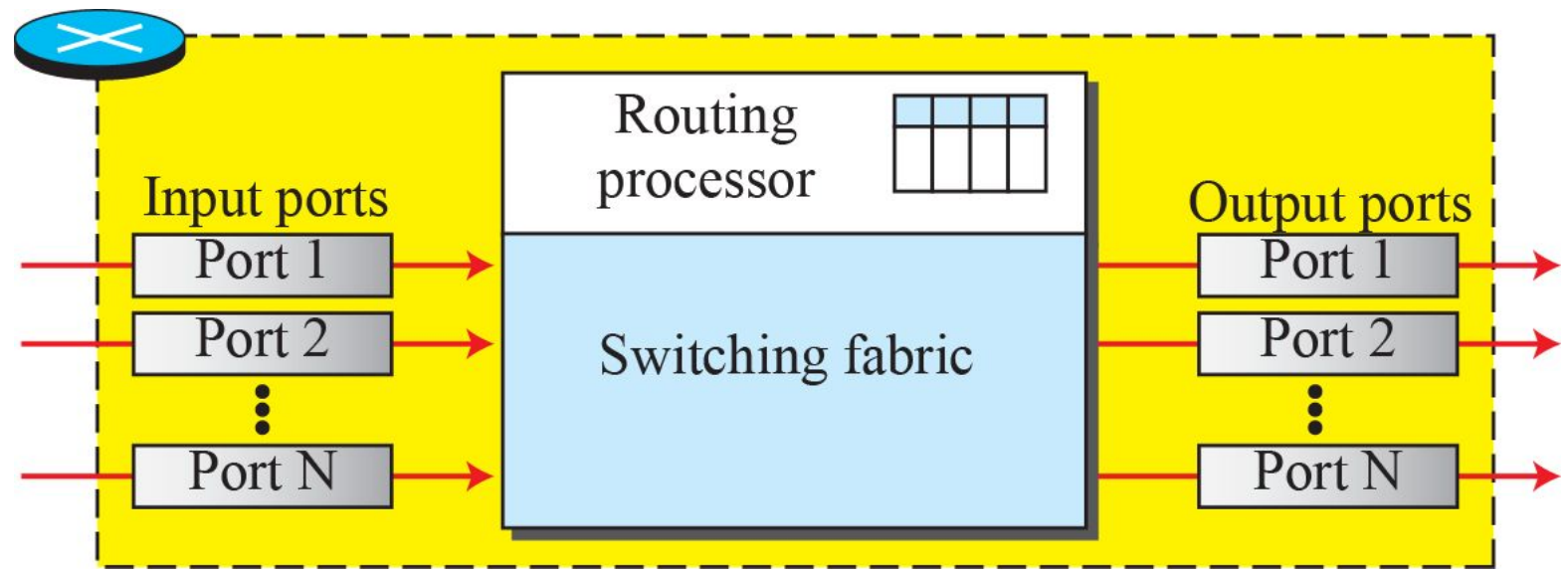


Figure 4.17: Input port

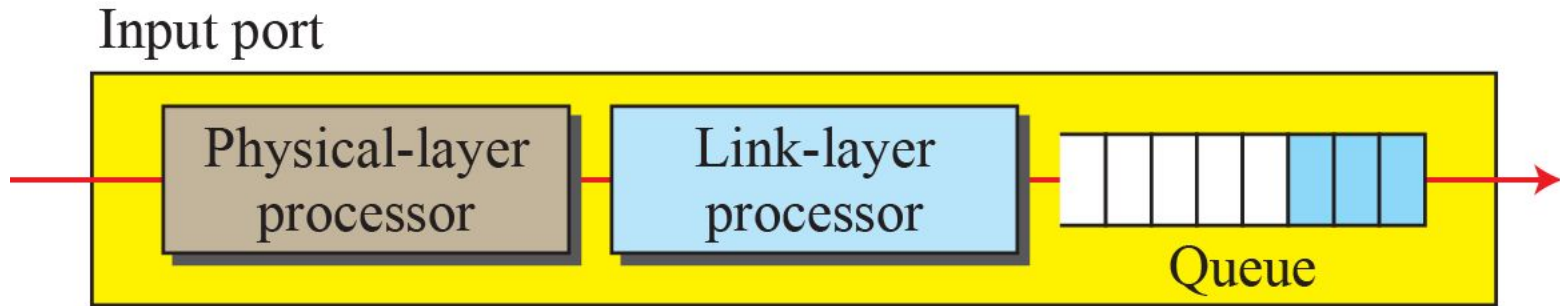


Figure 4.18: *Output port*

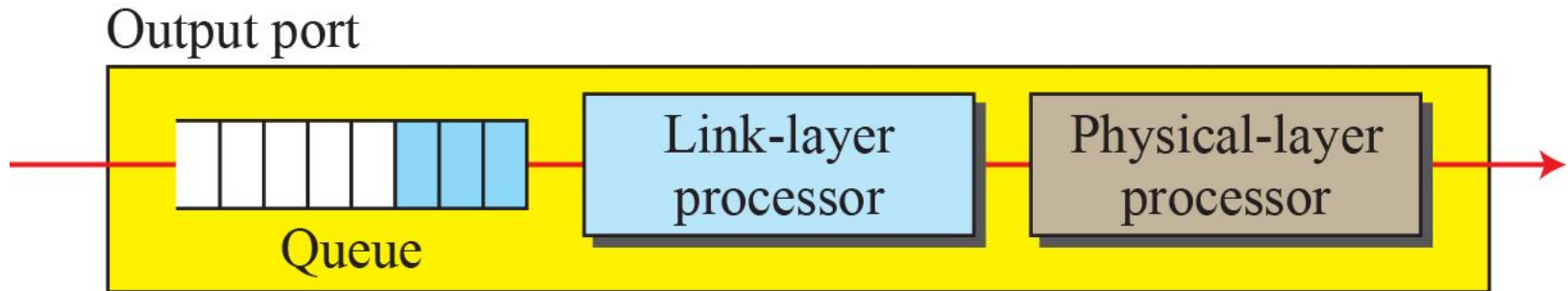


Figure 4.19: *Crossbar switch*

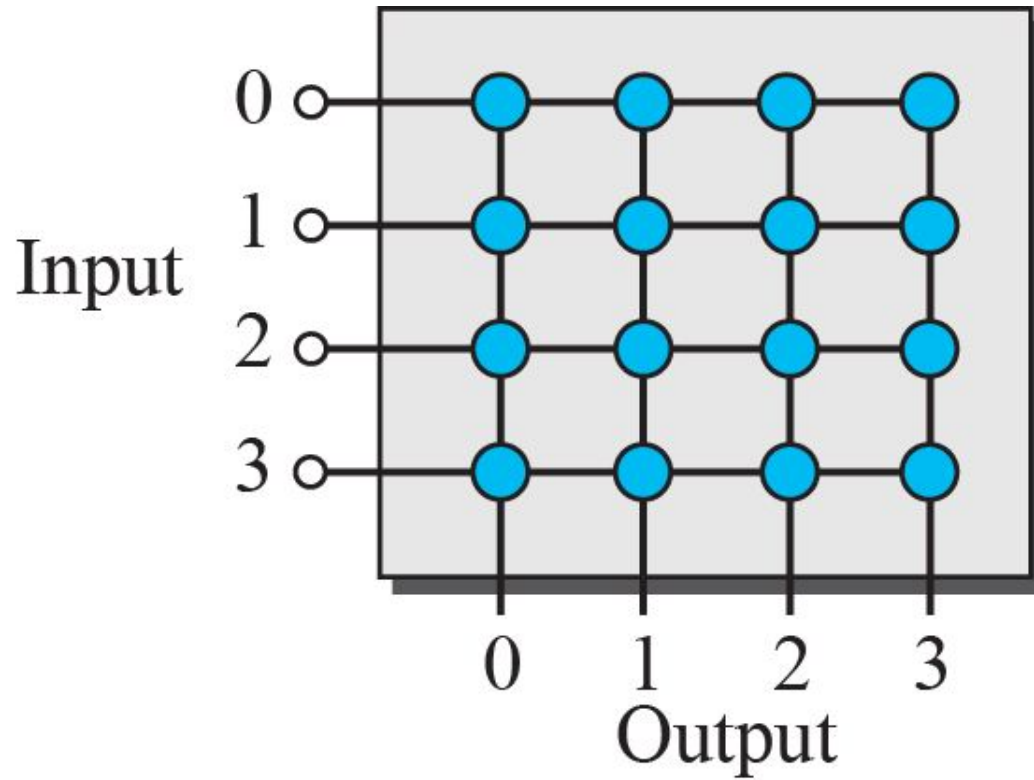


Figure 4.20: *Banyan switch*

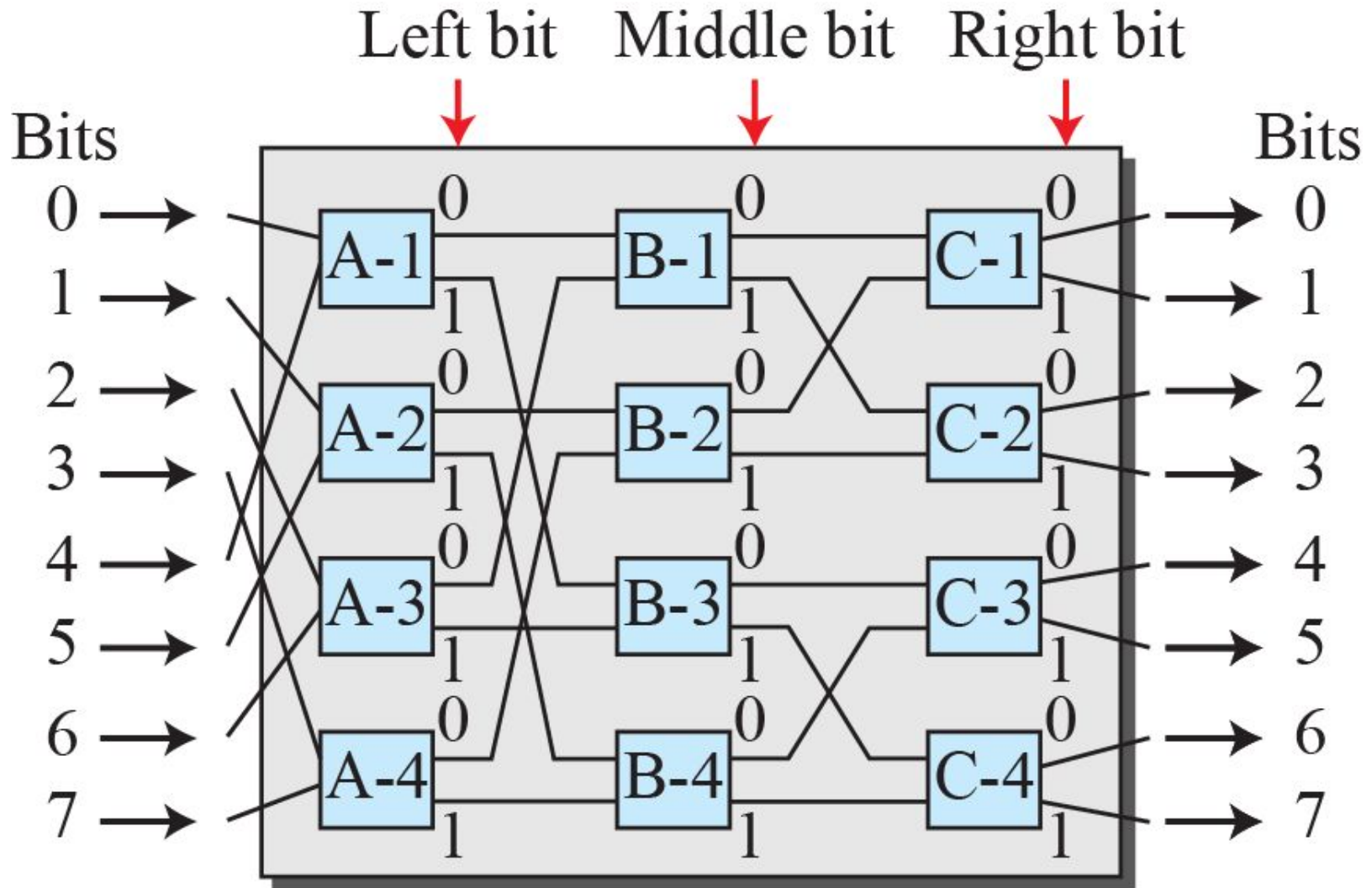
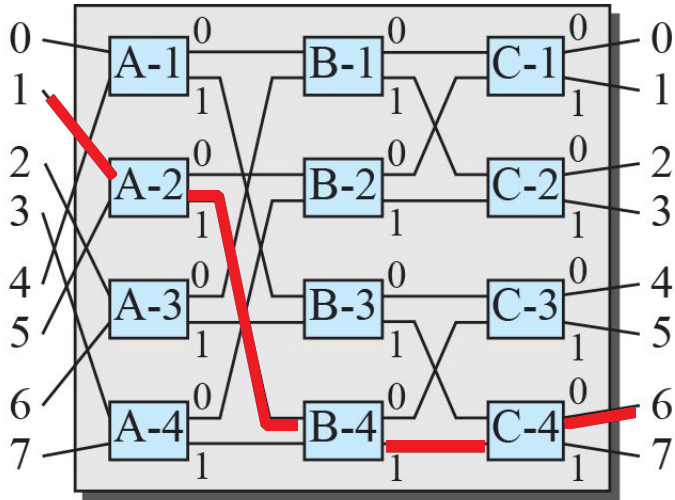
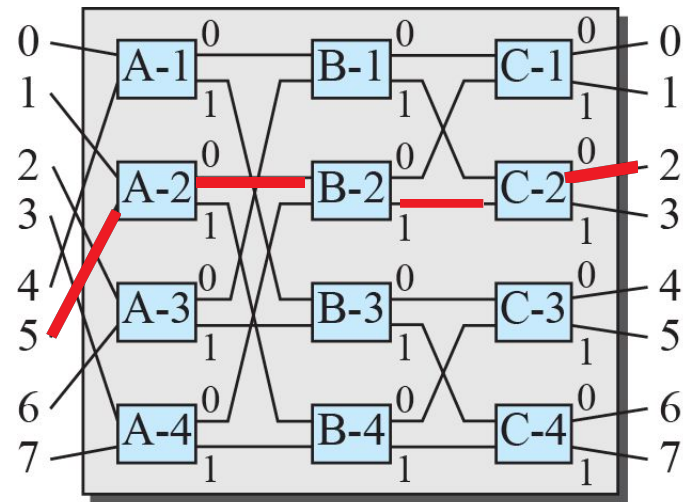


Figure 4.21: *Examples of routing in a banyan switch*

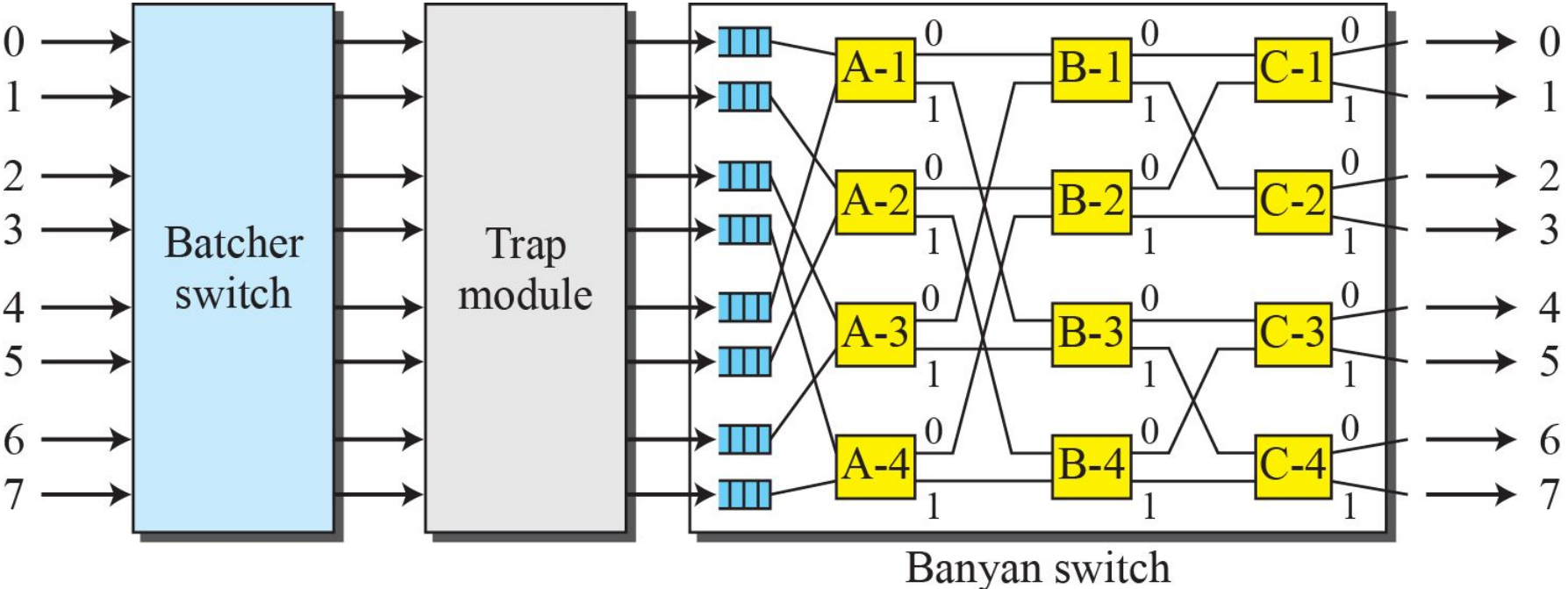


a. Input 1 sending to output 6 (110)



b. Input 5 sending to output 2 (010)

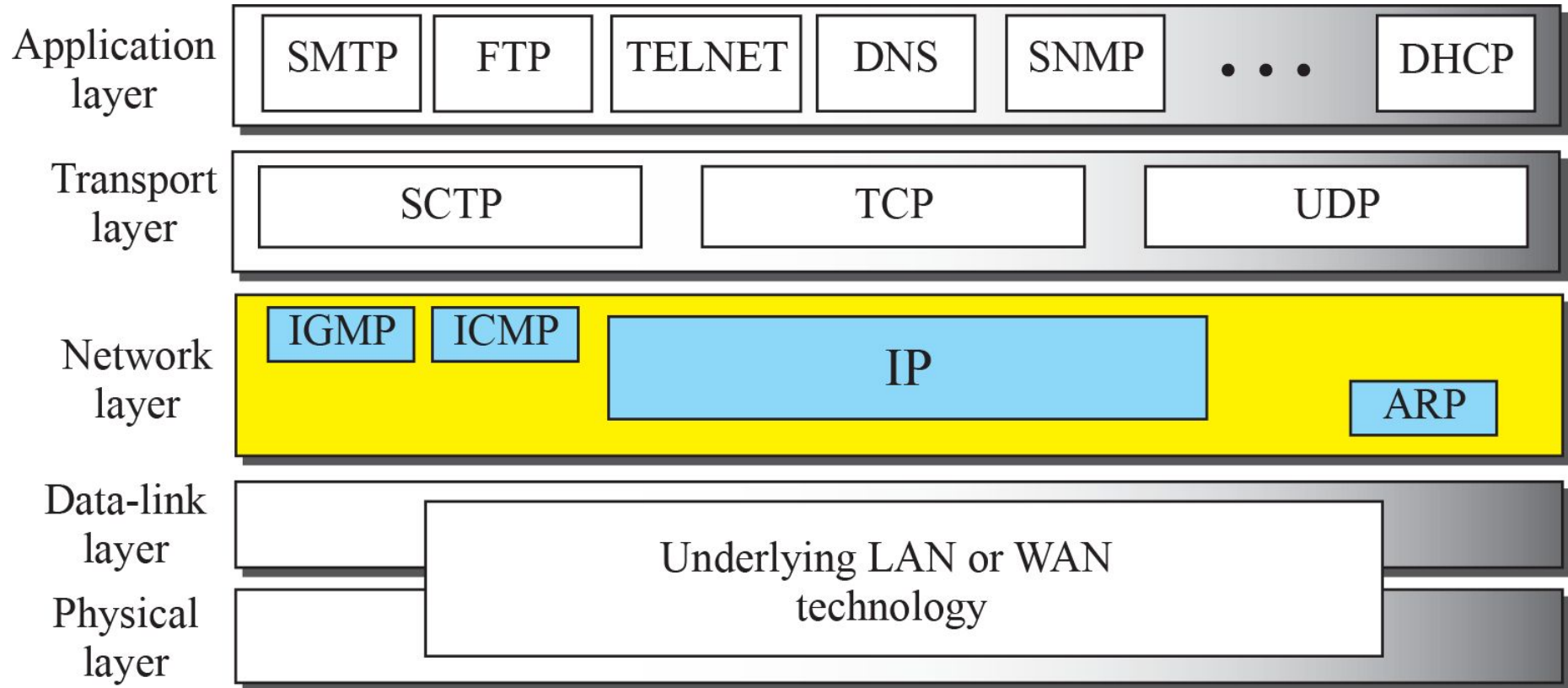
Figure 4.22: *Batcher-banyan switch*

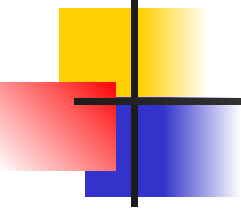


4-2 NETWORK-LAYER PROTOCOLS

In this section, we show how the network layer is implemented in the TCP/IP protocol suite. The protocols in the network layer have gone through several versions; in this section, we concentrate on the current version (4), in the last section of this chapter, we briefly discuss version 6, which is on the horizon.

Figure 4.23: *Position of IP and other network-layer protocols in TCP/IP protocol suite*





4.2.1 IPv4 Datagram Format

Packets used by the IP are called datagrams. Figure 4.24 shows the IPv4 datagram format. A datagram is a variable-length packet consisting of two parts: header and payload (data). The header is 20 to 60 bytes in length and contains information essential to routing and delivery. It is customary in TCP/IP to show the header in 4-byte sections.



4.2.1 (continued)

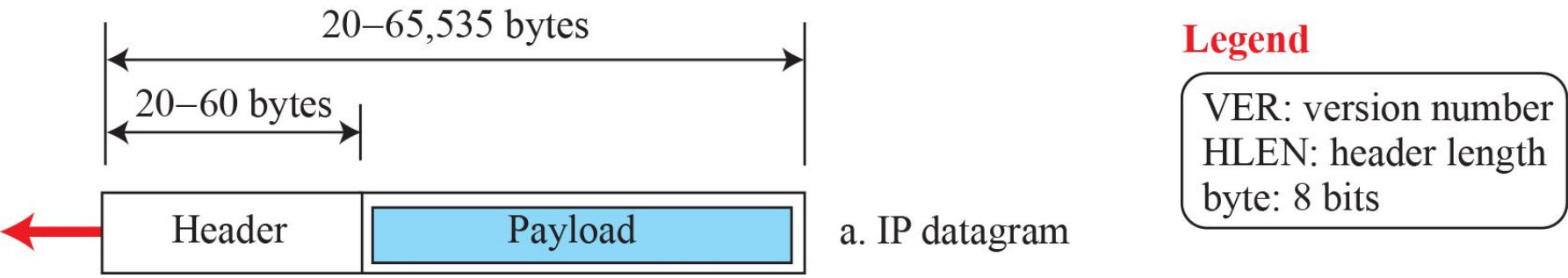
□ *Fragmentation*

- ◆ *Maximum Transfer Unit (MTU)*
- ◆ *Fields Related to Fragmentation*

□ *Security of IPv4 Datagrams*

- ◆ *Packet Sniffing*
- ◆ *Packet Modification*
- ◆ *IP Spoofing*
- ◆ *IPSec*

Figure 4.24: IP datagram



0

VER 4 bits	HLEN 4 bits	Service type 8 bits	Total length 16 bits	
Identification 16 bits			Flags 3 bits	Fragmentation offset 13 bits
Time-to-live 8 bits		Protocol 8 bits	Header checksum 16 bits	
Source IP address (32 bits)				
Destination IP address (32 bits)				
Options + padding (0 to 40 bytes)				

1

b. Header format

Figure 4.25: *Multiplexing and demultiplexing using the value of the protocol field*

ICMP: 01	UDP: 17
IGMP: 02	OSPF: 89
TCP: 06	

Some protocol values

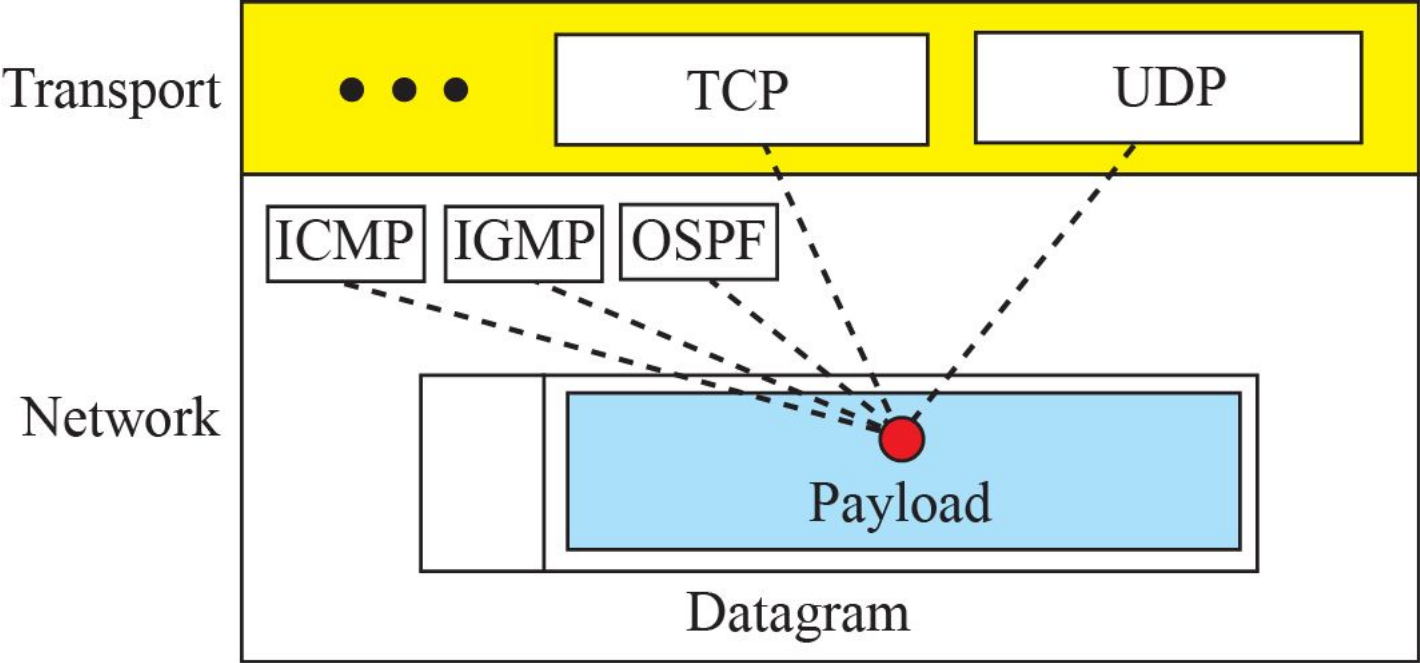


Figure 4.26: *Maximum transfer unit (MTU)*

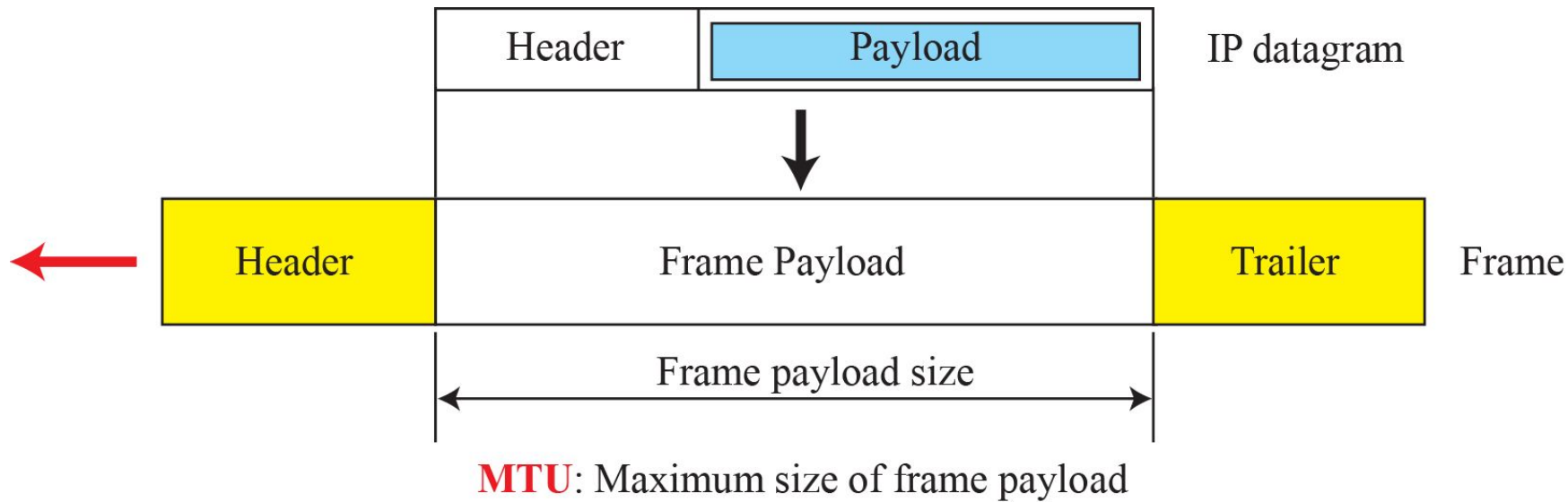


Figure 4.27: *Fragmentation example*

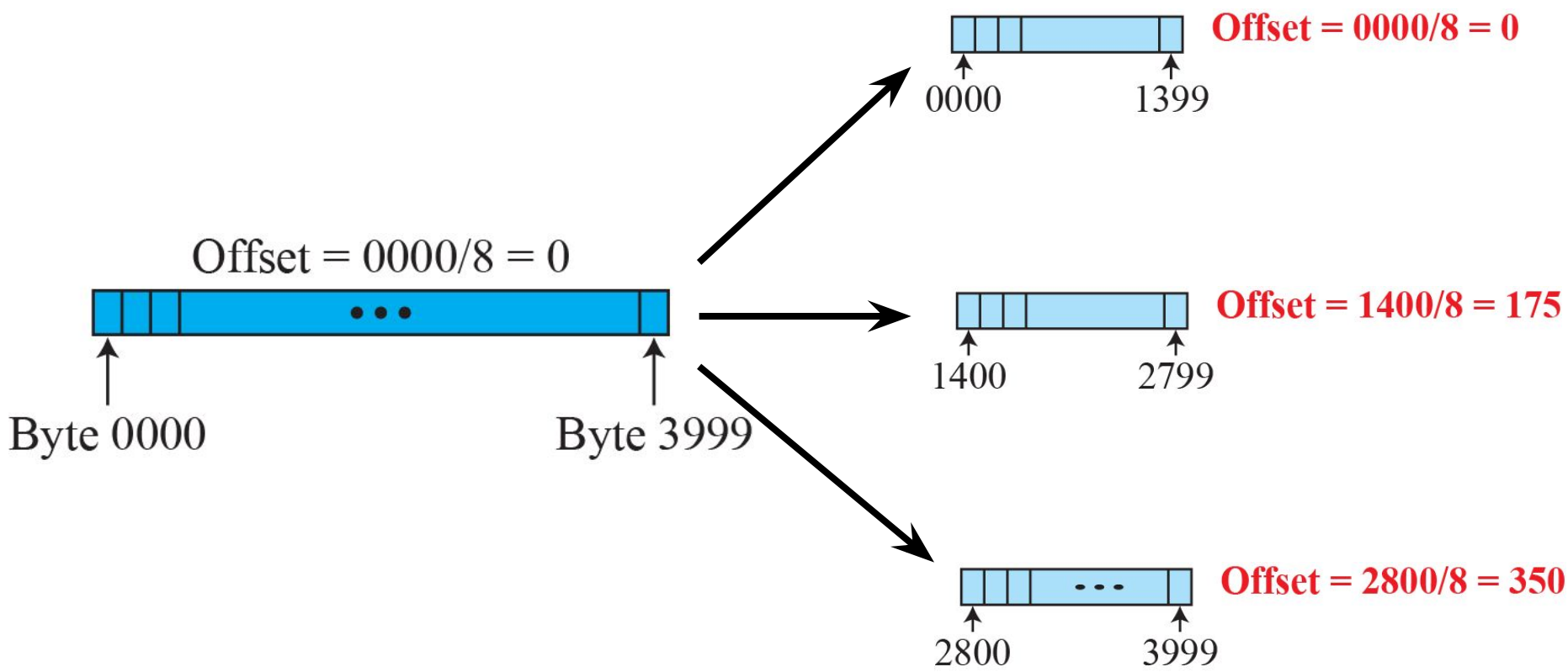
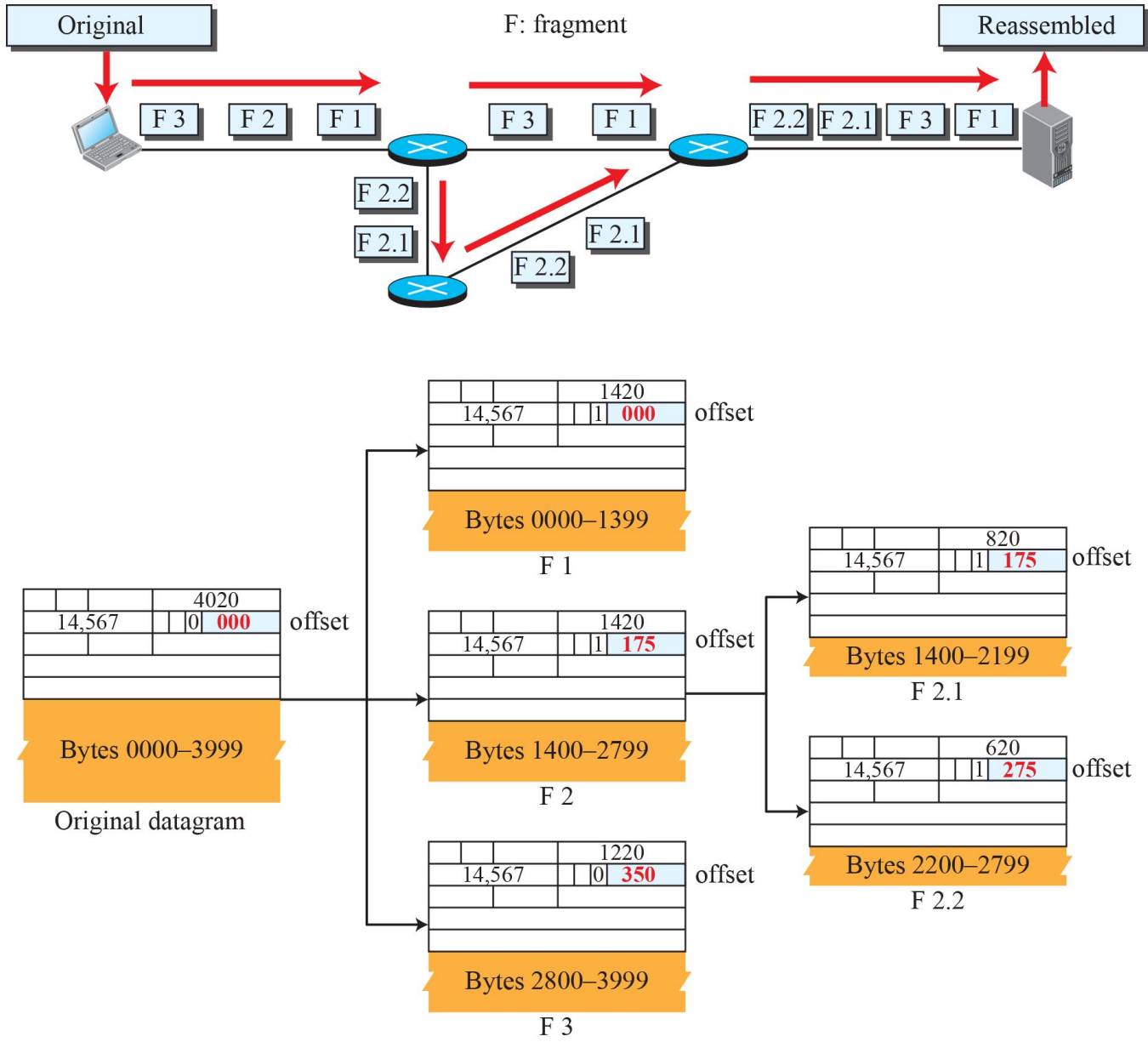
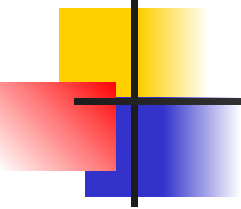


Figure 4.28: Detailed fragmentation example





4.2.2 IPv4 Addresses

The identifier used in the IP layer of the TCP/IP protocol suite to identify the connection of each device to the Internet is called the Internet address or IP address. An IPv4 address is a 32-bit address that uniquely and universally defines the connection of a host or a router to the Internet. The IP address is the address of the connection, not the host or the router, because if the device is moved to another network, the IP address may be changed.



4.2.2 (continued)

□ *Address Space*

□ *Notation*

□ *Hierarchy in Addressing*

□ *Classful Addressing*

◆ *Address Depletion*

◆ *Subnetting and Supernetting*

◆ *Advantage of Classful Addressing*

Classless Addressing

- ◆ *Prefix Length: Slash Notation*
- ◆ *Extracting information from an address*
- ◆ *Address Mask*
- ◆ *Network Address*
- ◆ *Block Allocation*
- ◆ *Subnetting*
- ◆ *Address Aggregation*
- ◆ *Special Addresses*



4.2.2 (continued)

Dynamic Host Configuration Protocol (DHCP)

- ◆ *DHCP Message Format*
- ◆ *DHCP Operation*
- ◆ *Two Well-Known Ports*
- ◆ *Using FTP*
- ◆ *Error Control*
- ◆ *Transition States*

NAT

- ◆ *Address Translation*
- ◆ *Translation Table*

Figure 4.29: *Three different notations in IPv4 addressing*

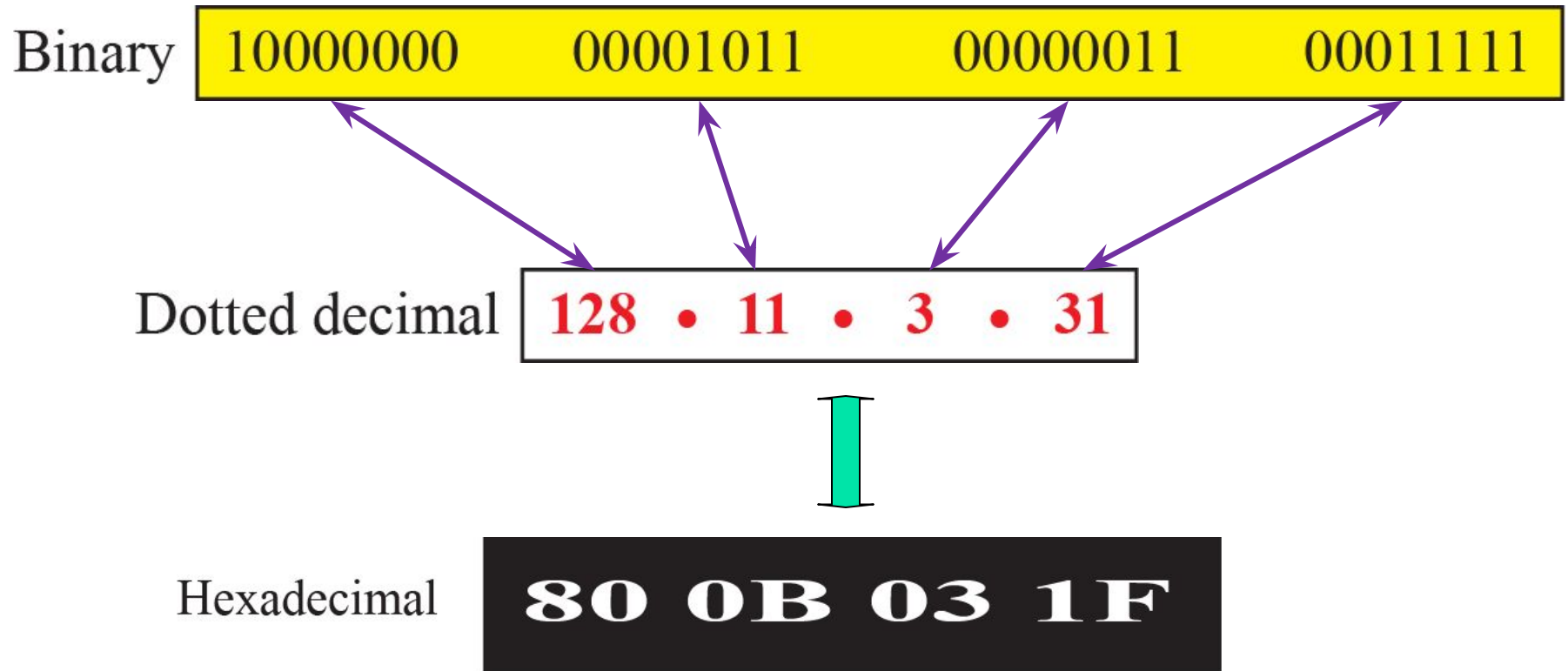


Figure 4.30: Hierarchy in addressing

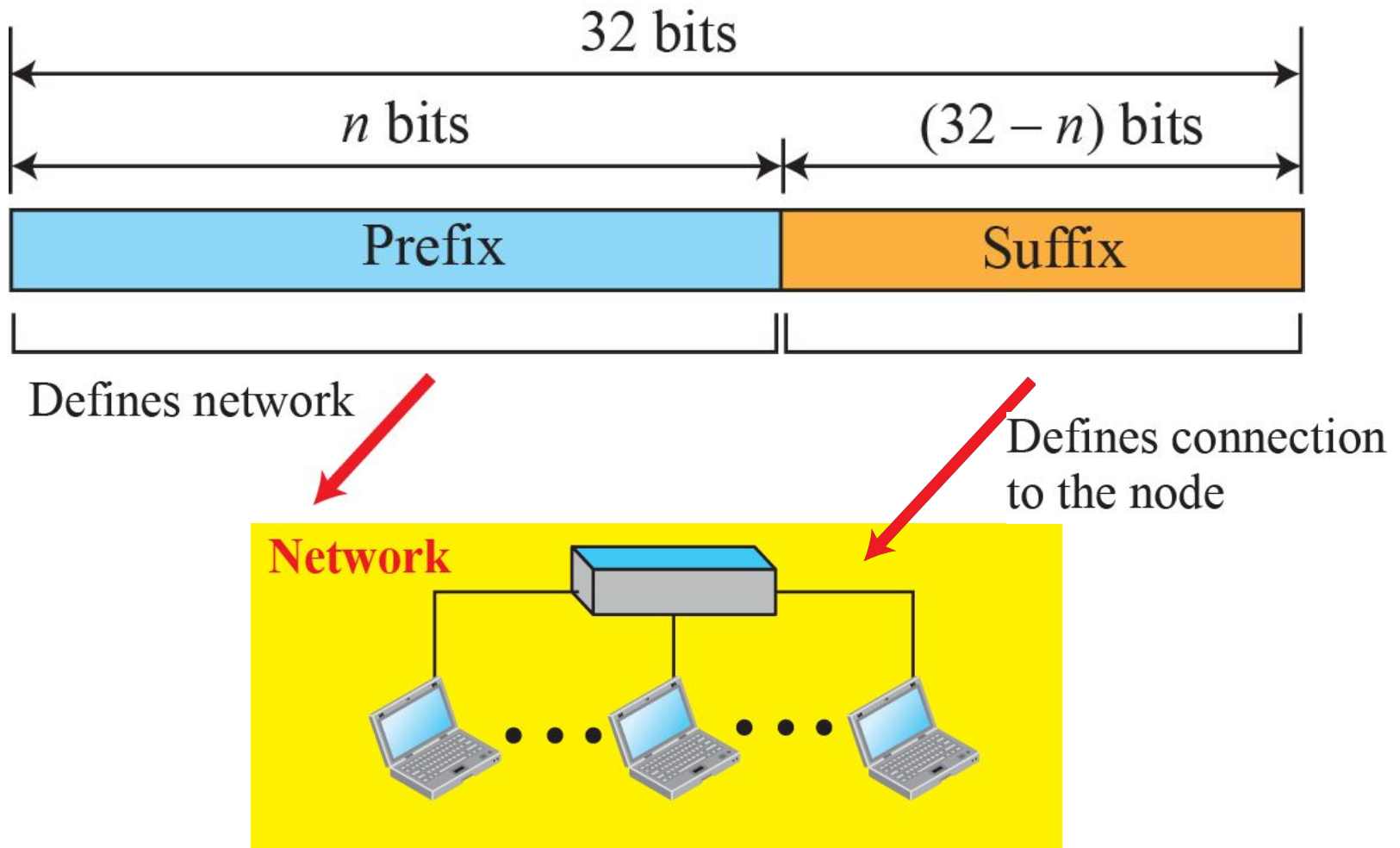
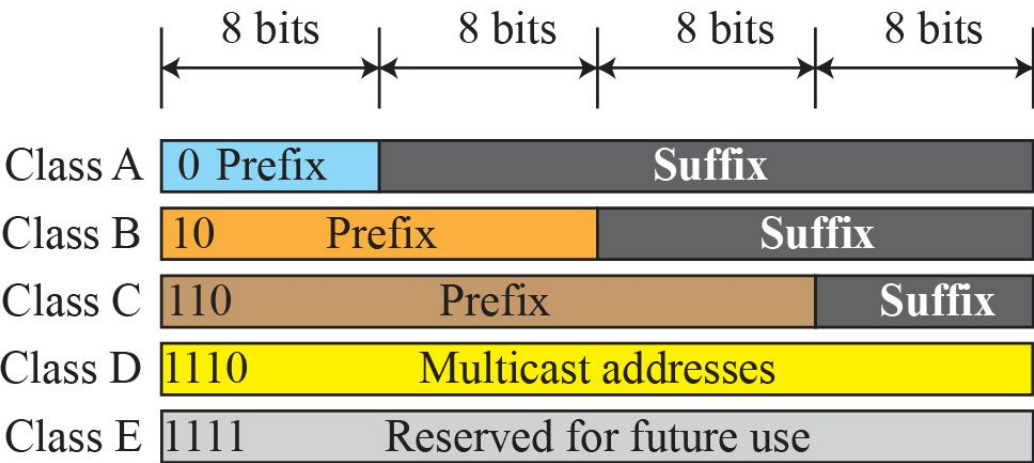
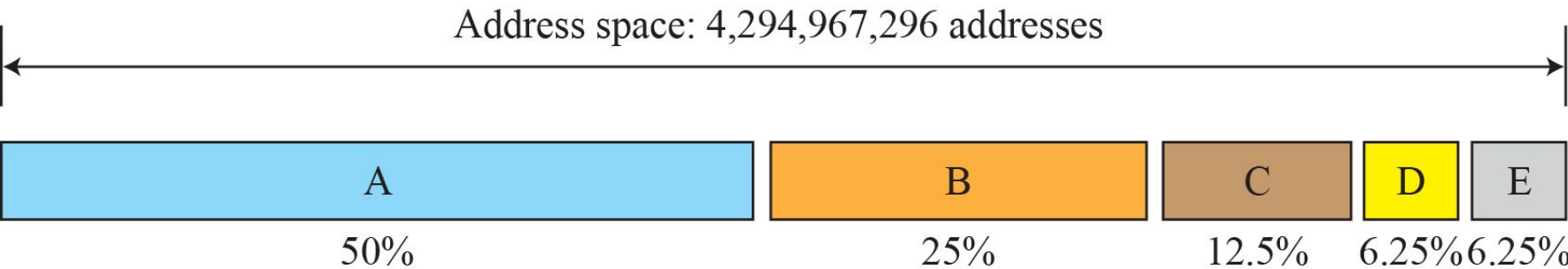


Figure 4.31: Occupation of the address space in classful addressing



Class	Prefixes	First byte
A	$n = 8$ bits	0 to 127
B	$n = 16$ bits	128 to 191
C	$n = 24$ bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

Figure 4.32: *Variable-length blocks in classless addressing*

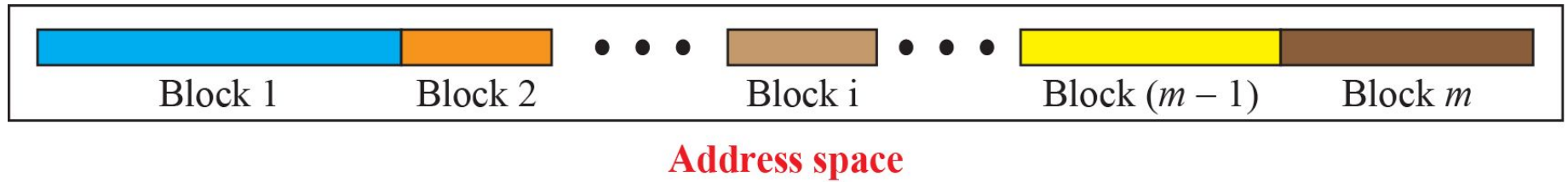
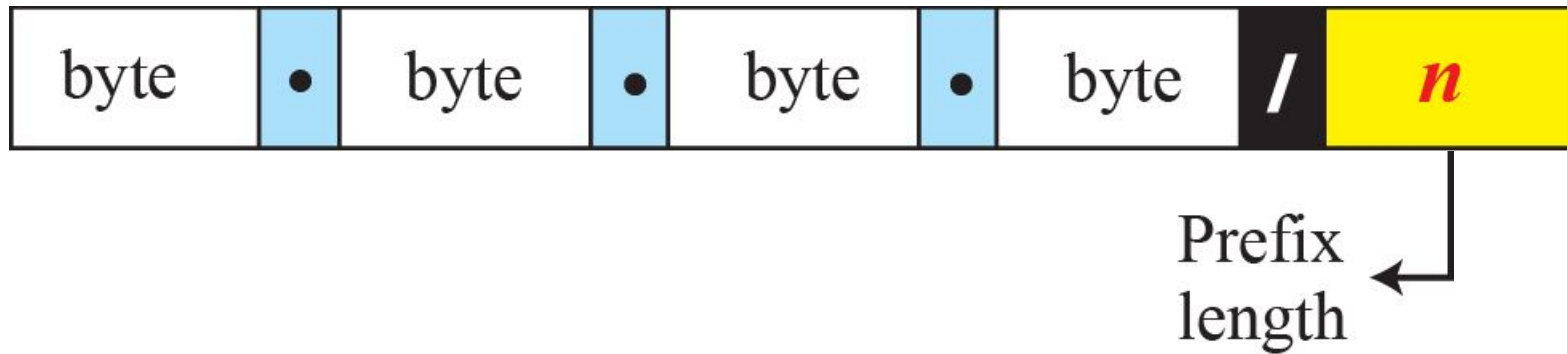


Figure 4.33: *Slash notation (CIDR)*



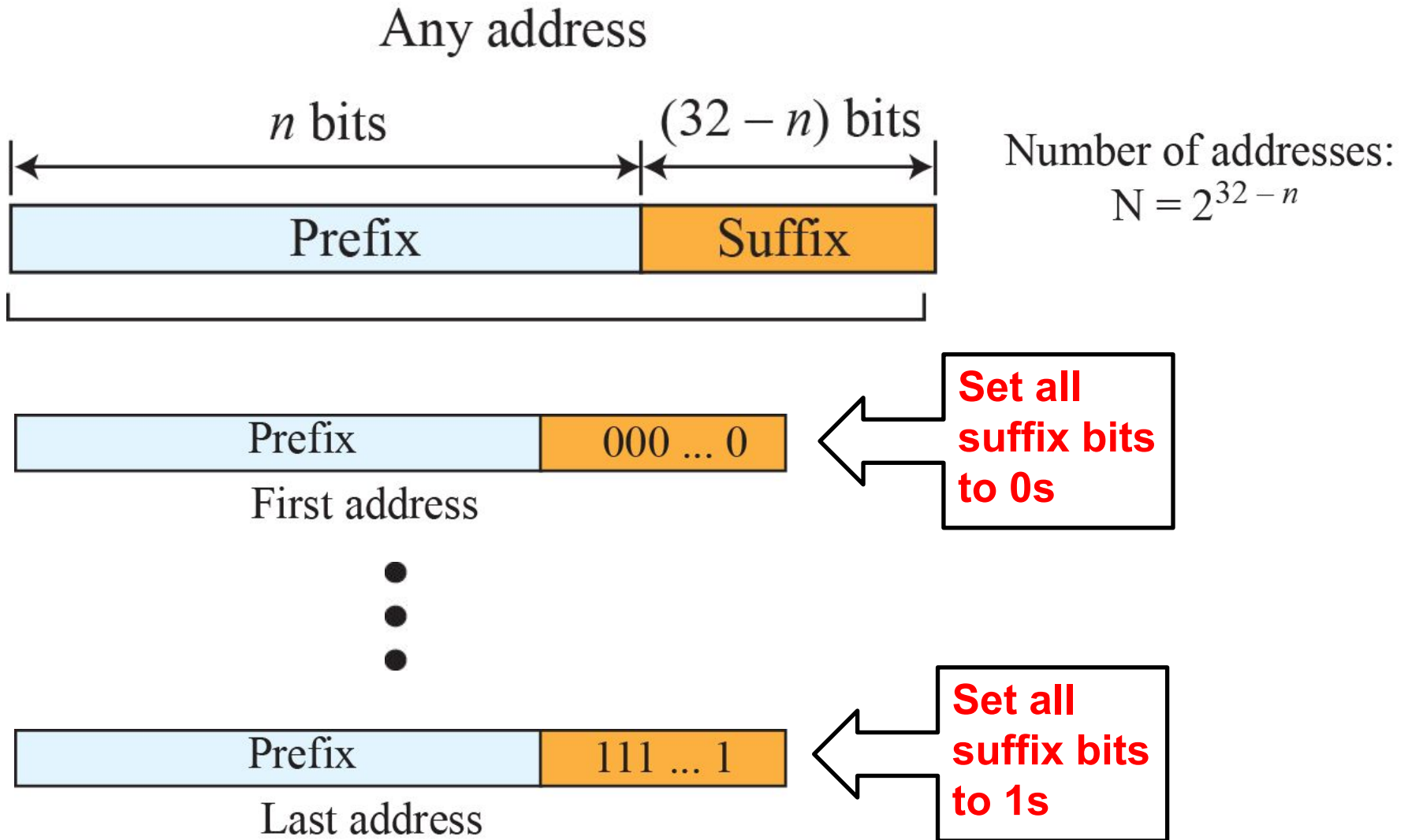
Examples:

12.24.76.8/**8**

23.14.67.92/**12**

220.8.24.255/**25**

Figure 4.34: *Information extraction in classless addressing*



Example 4.1

A classless address is given as 167.199.170.82/27. We can find the above three pieces of information as follows. The number of addresses in the network is $2^{32-n} = 2^5 = 32$ addresses. The first address can be found by keeping the first 27 bits and changing the rest of the bits to 0s.

Address: 167.199.170.82/27	10100111	11000111	10101010	01010010
First address: 167.199.170.64/27	10100111	11000111	10101010	01000000

The last address can be found by keeping the first 27 bits and changing the rest of the bits to 1s.

Address: 167.199.170.82/27	10100111	11000111	10101010	01011111
Last address: 167.199.170.95/27	10100111	11000111	10101010	01011111

Example 4.2

We repeat Example 4.1 using the mask. The mask in dotted-decimal notation is 256.256.256.224. The AND, OR, and NOT operations can be applied to individual bytes using calculators and applets at the book website.

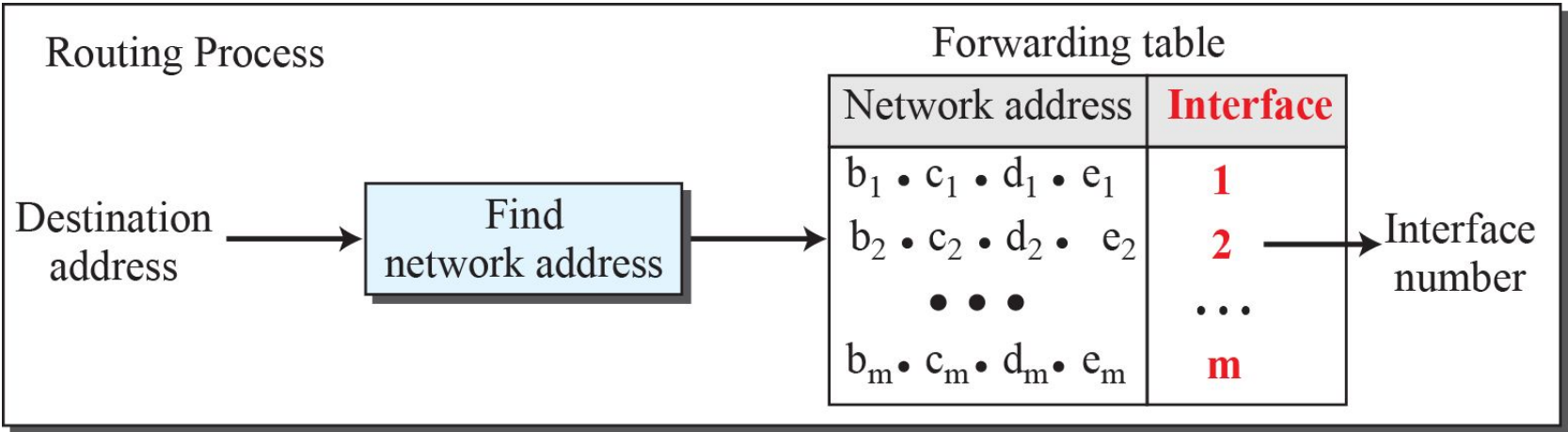
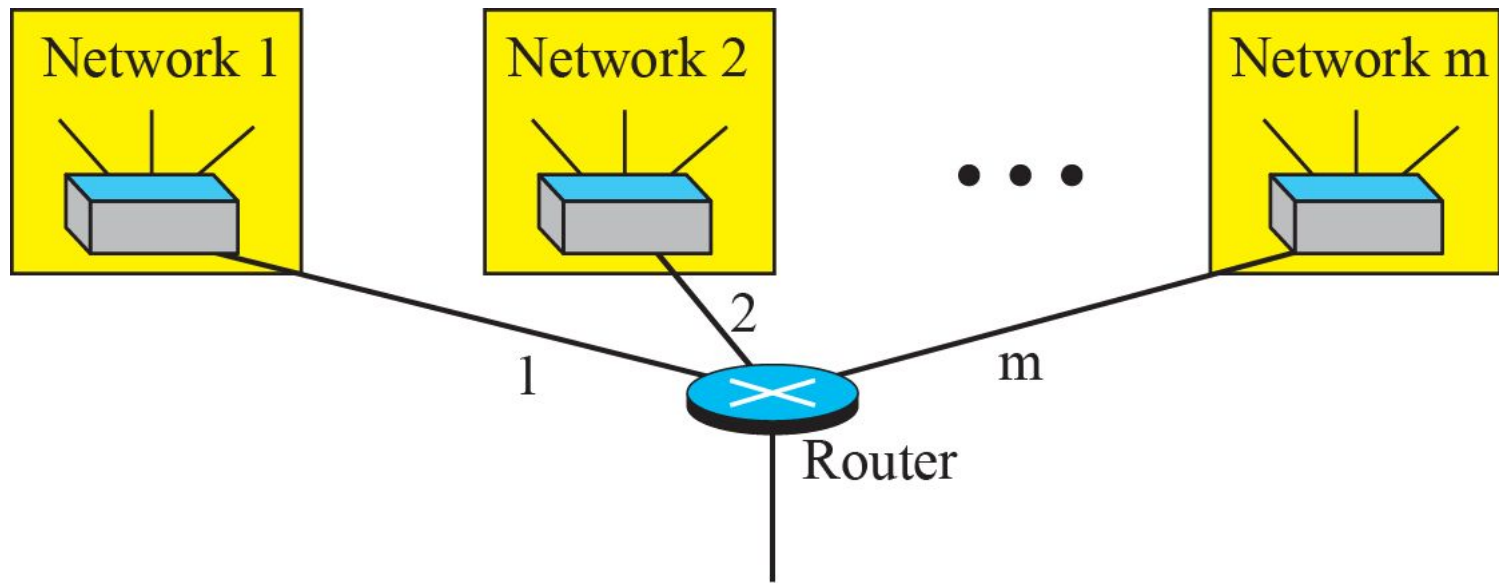
Number of addresses in the block:	$N = \text{NOT}(\text{mask}) + 1 = 0.0.0.31 + 1 = 32 \text{ addresses}$
First address:	$\text{First} = (\text{address}) \text{ AND } (\text{mask}) = 167.199.170.82$
Last address:	$\text{Last} = (\text{address}) \text{ OR } (\text{NOT mask}) = 167.199.170.255$

Example 4.3

In classless addressing, an address cannot per se define the block the address belongs to. For example, the address 230.8.24.56 can belong to many blocks. Some of them are shown below with the value of the prefix associated with that block.

Prefix length:16	→	Block:	230.8.0.0	to	230.8.255.255
Prefix length:20	→	Block:	230.8.16.0	to	230.8.31.255
Prefix length:26	→	Block:	230.8.24.0	to	230.8.24.63
Prefix length:27	→	Block:	230.8.24.32	to	230.8.24.63
Prefix length:29	→	Block:	230.8.24.56	to	230.8.24.63
Prefix length:31	→	Block:	230.8.24.56	to	230.8.24.57

Figure 4.35: Network address



Example 4.4

An ISP has requested a block of 1000 addresses. Since 1000 is not a power of 2, 1024 addresses are granted. The prefix length is calculated as $n = 32 - \log_2 1024 = 22$. An available block, 18.14.12.0/**22**, is granted to the ISP. It can be seen that the first address in decimal is 302,910,464, which is divisible by 1024.

Example 4.5

An organization is granted a block of addresses with the beginning address 14.24.74.0/24. The organization needs to have 3 subblocks of addresses to use in its three subnets: one subblock of 10 addresses, one subblock of 60 addresses, and one subblock of 120 addresses. Design the subblocks.

Solution

There are $2^{32-24} = 256$ addresses in this block. The first address is 14.24.74.0/24; the last address is 14.24.74.255/24. To satisfy the third requirement, we assign addresses to subblocks, starting with the largest and ending with the smallest one.

Example 4.5 (continued)

a. The number of addresses in the largest subblock, which requires 120 addresses, is not a power of 2. We allocate 128 addresses. The subnet mask for this subnet can be found as $n_1 = 32 - \log_2 128 = 25$. The first address in this block is 14.24.74.0/**25**; the last address is 14.24.74.127/**25**.

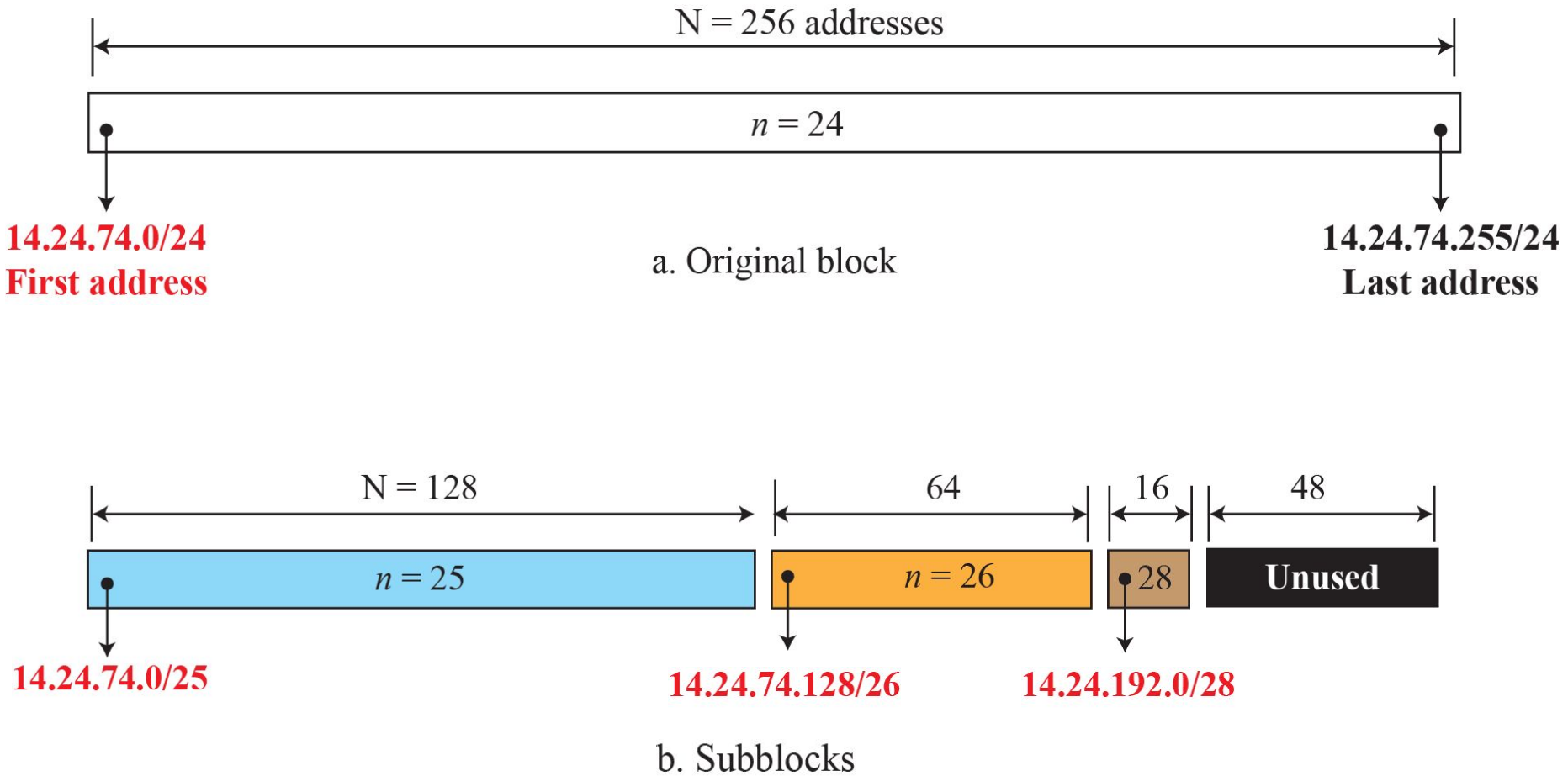
b. The number of addresses in the second largest subblock, which requires 60 addresses, is not a power of 2 either. We allocate 64 addresses. The subnet mask for this subnet can be found as $n_2 = 32 - \log_2 64 = 26$. The first address in this block is 14.24.74.128/**26**; the last address is 14.24.74.191/**26**.

Example 4.5 (continued)

c. The number of addresses in the largest subblock, which requires 120 addresses, is not a power of 2. We allocate 128 addresses. The subnet mask for this subnet can be found as $n_1 = 32 - \log_2 128 = 25$. The first address in this block is 14.24.74.0/**25**; the last address is 14.24.74.127/**25**.

If we add all addresses in the previous subblocks, the result is 208 addresses, which means 48 addresses are left in reserve. The first address in this range is 14.24.74.208. The last address is 14.24.74.255. We don't know about the prefix length yet. Figure 4.36 shows the configuration of blocks. We have shown the first address in each block.

Figure 4.36: *Solution to Example 4.5*



Example 4.6

Figure 4.37 shows how four small blocks of addresses are assigned to four organizations by an ISP. The ISP combines these four blocks into one single block and advertises the larger block to the rest of the world. Any packet destined for this larger block should be sent to this ISP. It is the responsibility of the ISP to forward the packet to the appropriate organization. This is similar to routing we can find in a postal network. All packages coming from outside a country are sent first to the capital and then distributed to the corresponding destination.

Figure 4.37: Example of address aggregation

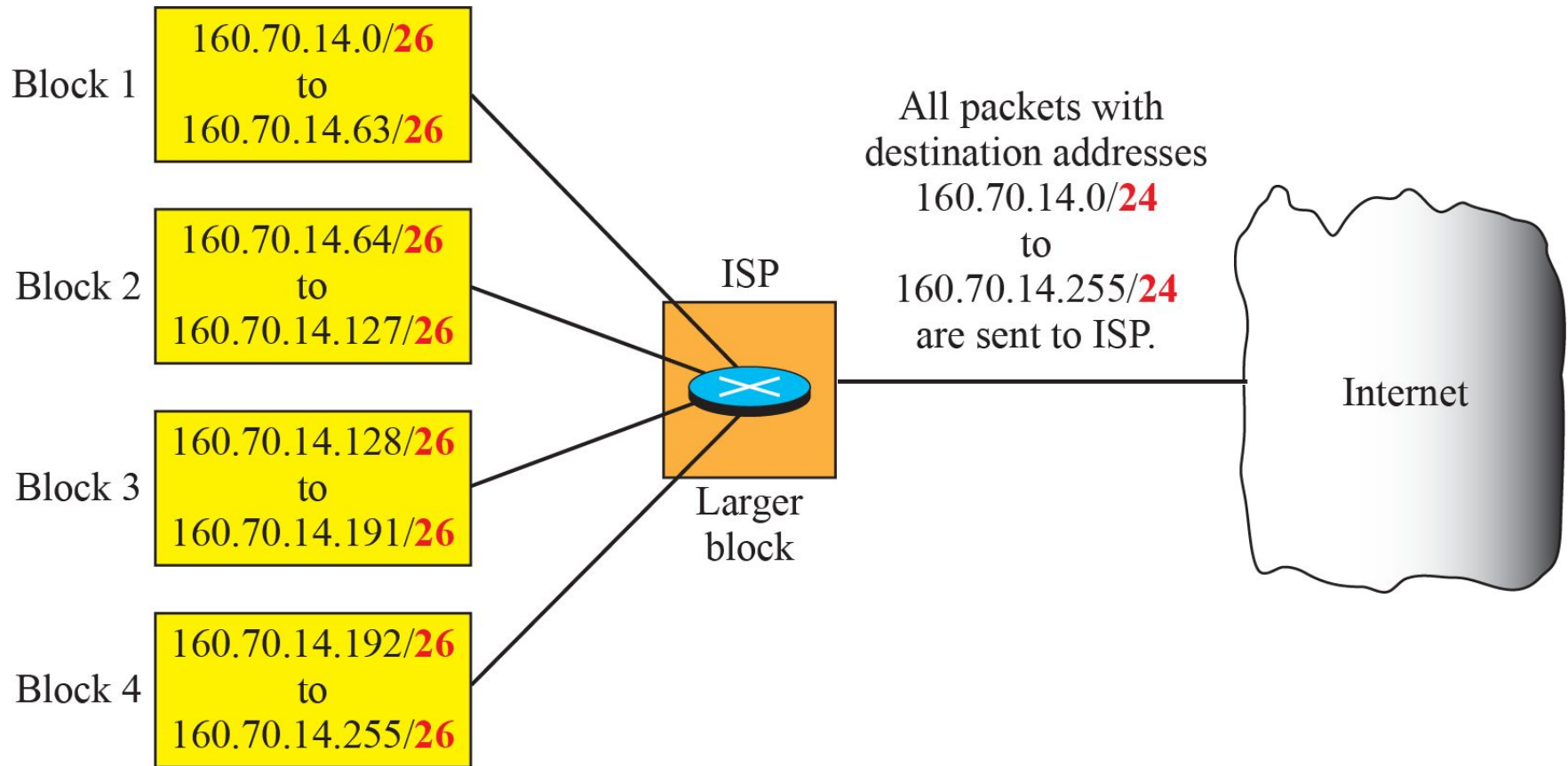
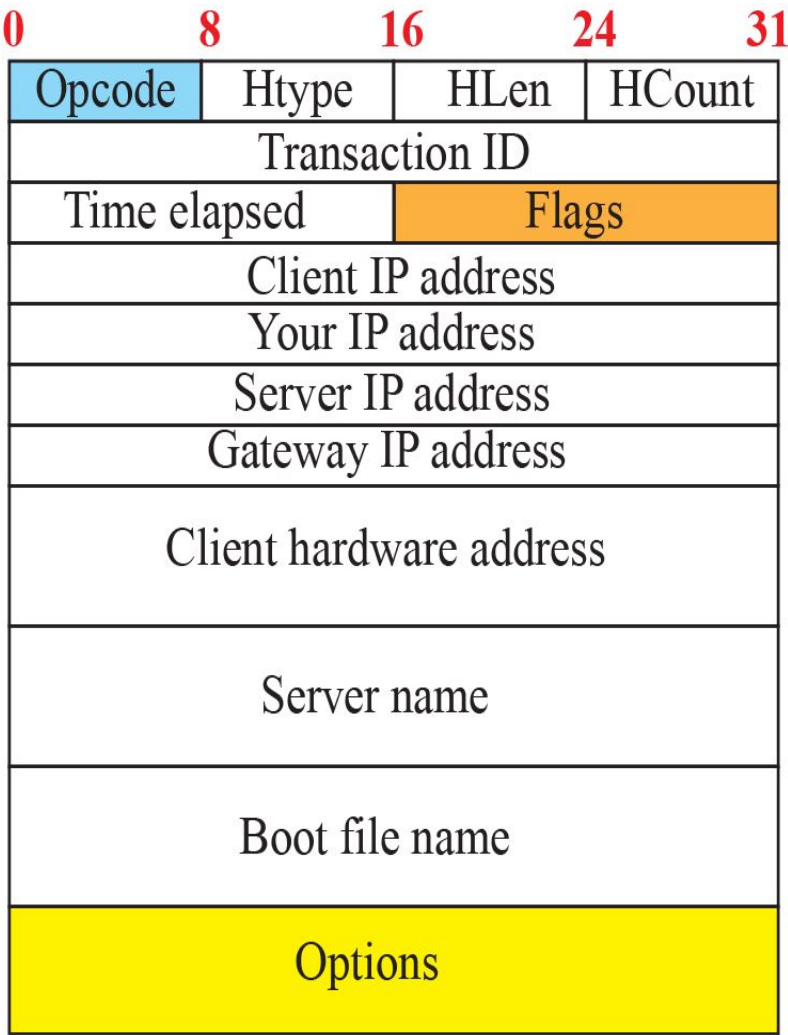


Figure 4.38: *DHCP message format*



Fields:

Opcode: Operation code, request (1) or reply (2)

Htype: Hardware type (Ethernet, ...)

HLen: Length of hardware address

HCount: Maximum number of hops the packet can travel

Transaction ID: An integer set by client and repeated by the server

Time elapsed: The number of seconds since the client started to boot

Flags: First bit defines unicast (0) or multicast (1); other 15 bits not used

Client IP address: Set to 0 if the client does not know it

Your IP address: The client IP address sent by the server

Server IP address: A broadcast IP address if client does not know it

Gateway IP address: The address of default router

Server name: A 64-byte domain name of the server

Boot file name: A 128-byte file name holding extra information

Options: A 64-byte field with dual purpose described in text

Figure 4.39: Option format

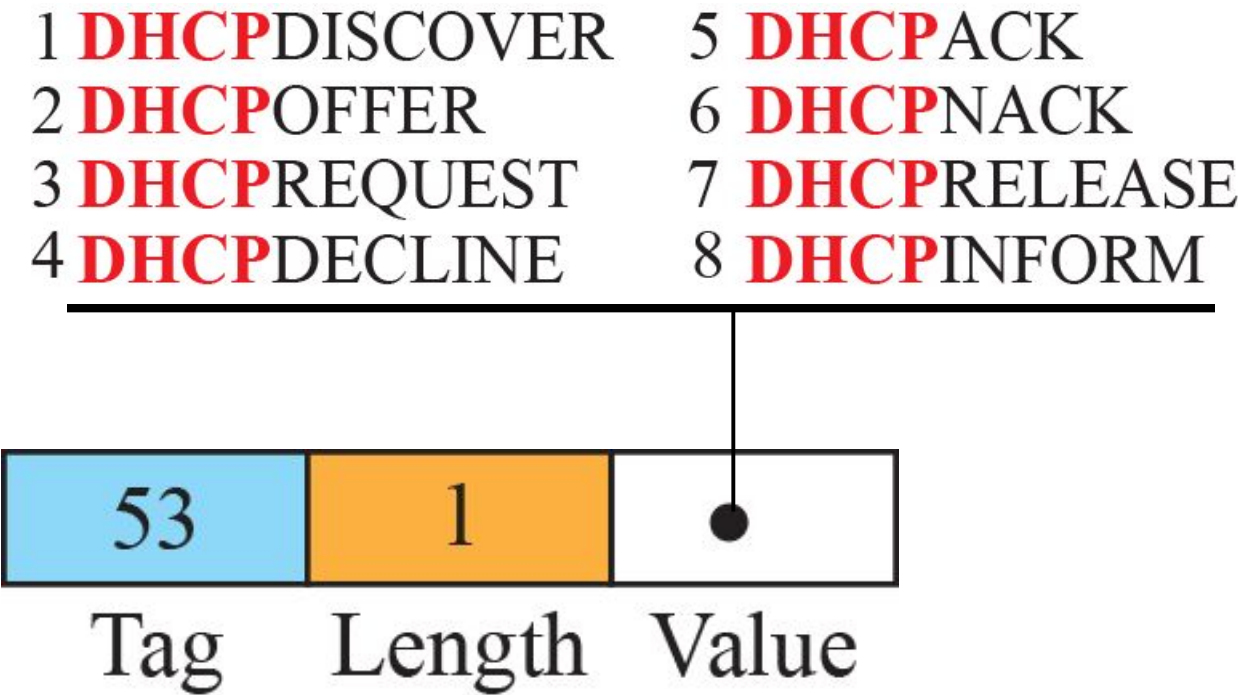


Figure 4.40: Operation of DHCP

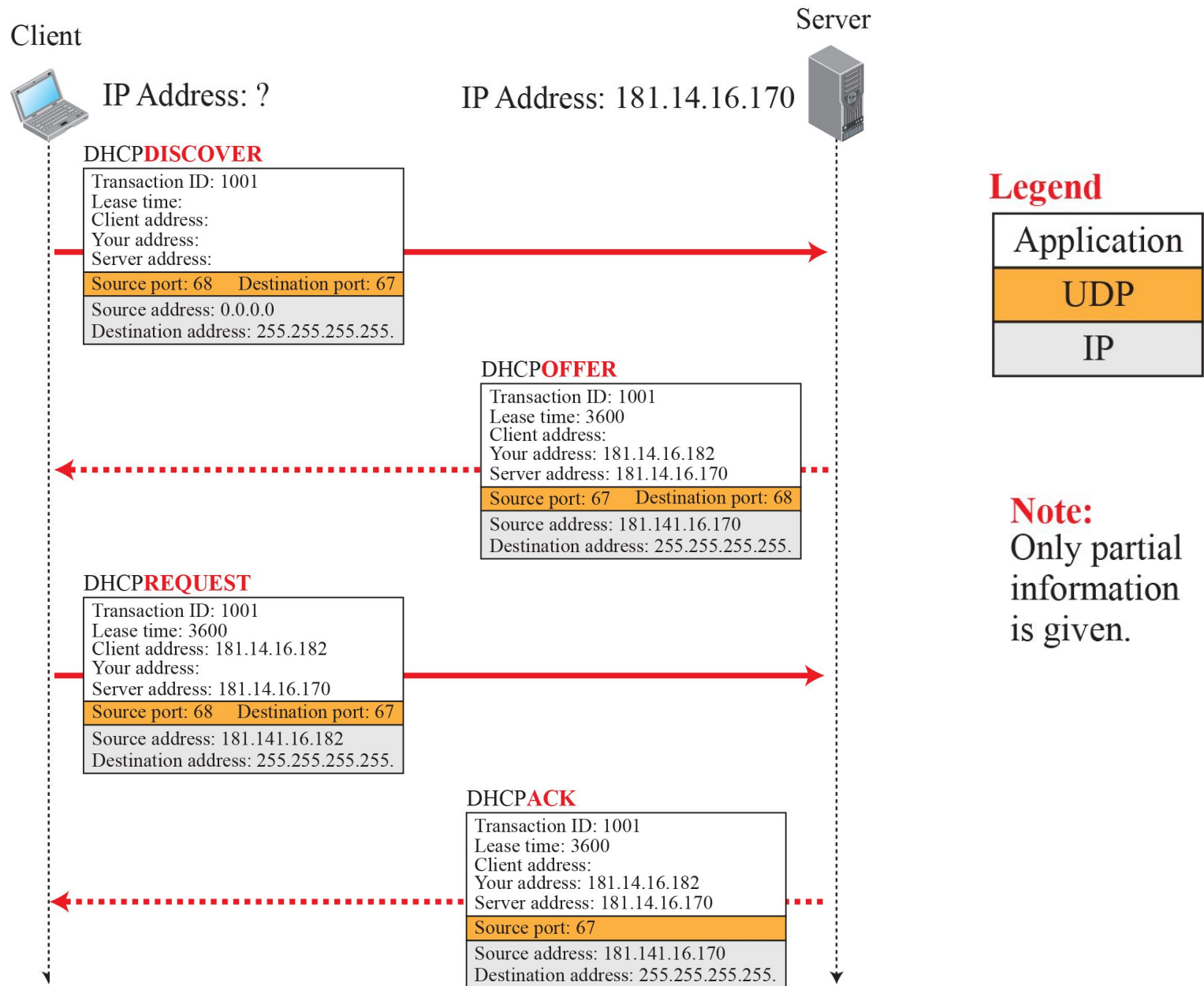


Figure 4.41: *FSM for the DHCP client*

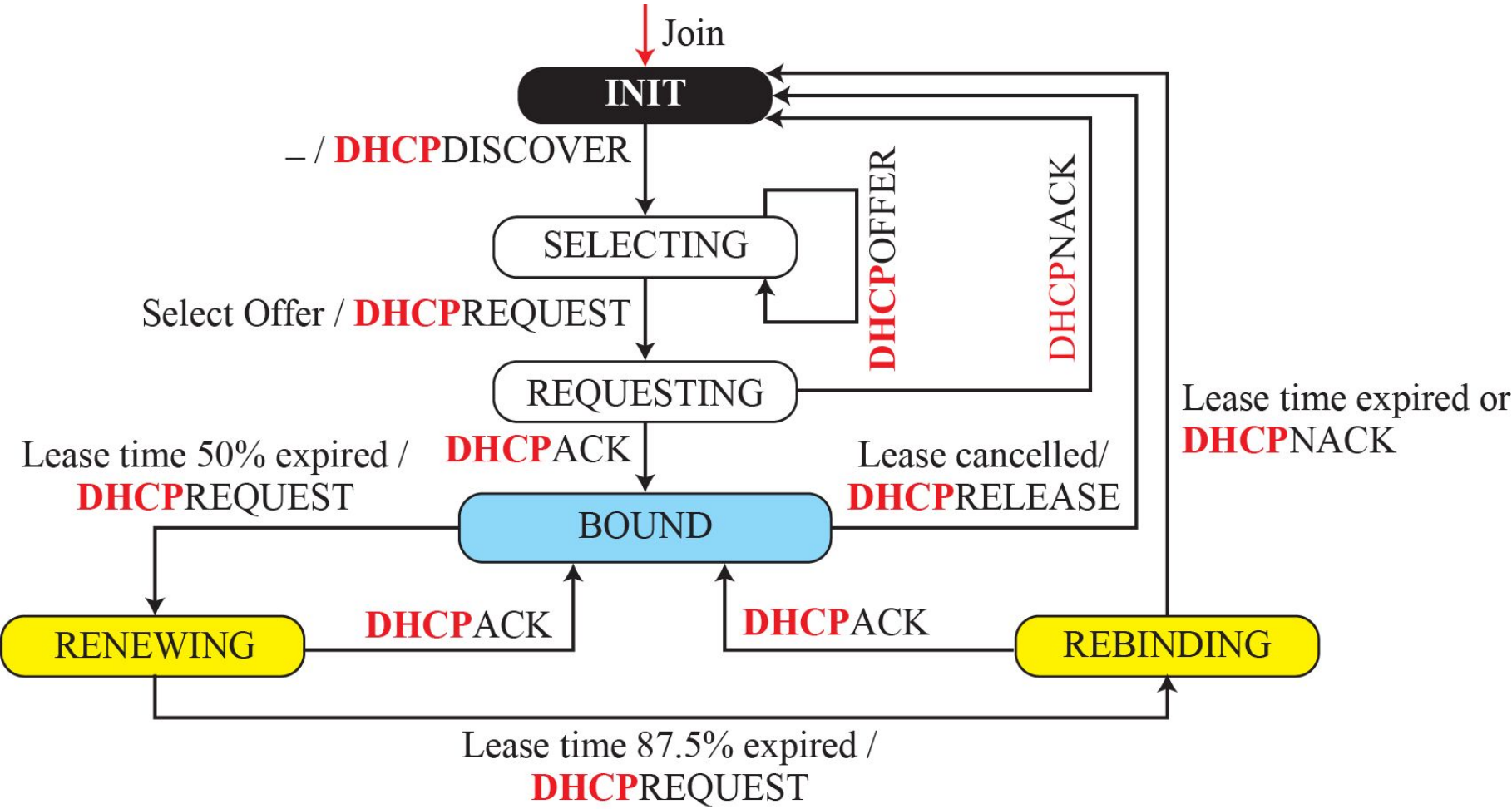


Figure 4.42: NAT

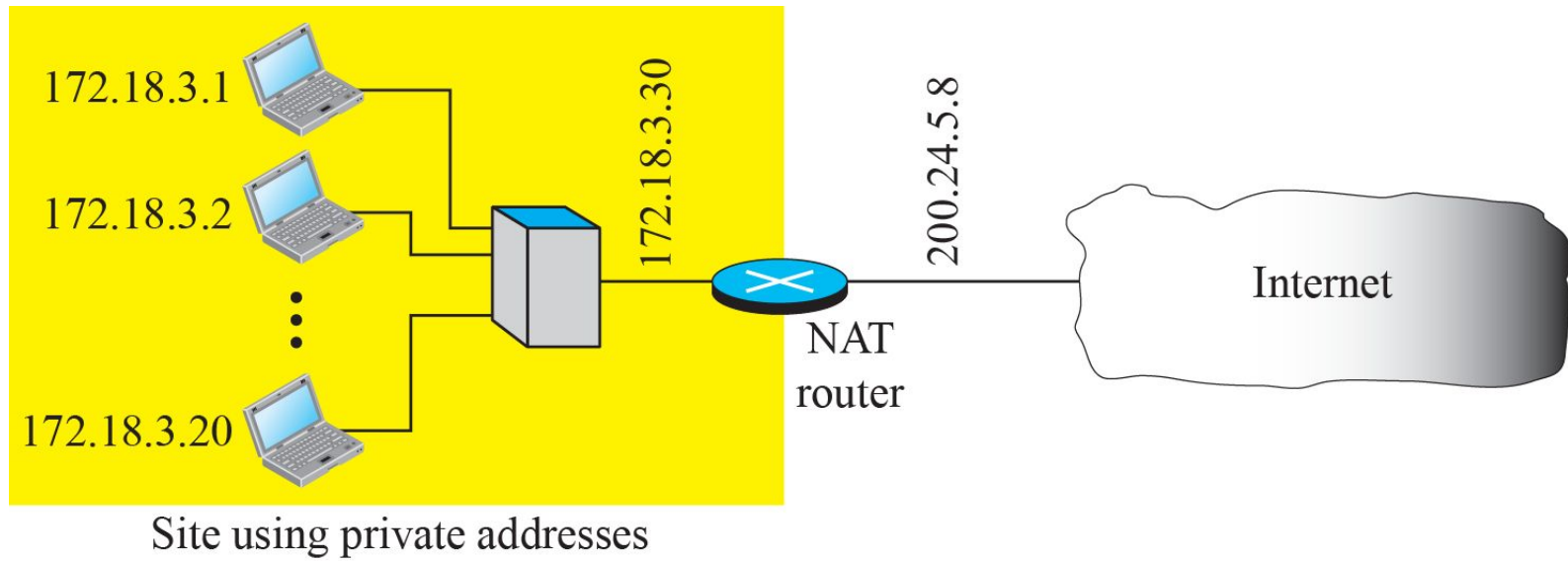


Figure 4.43: Address translation

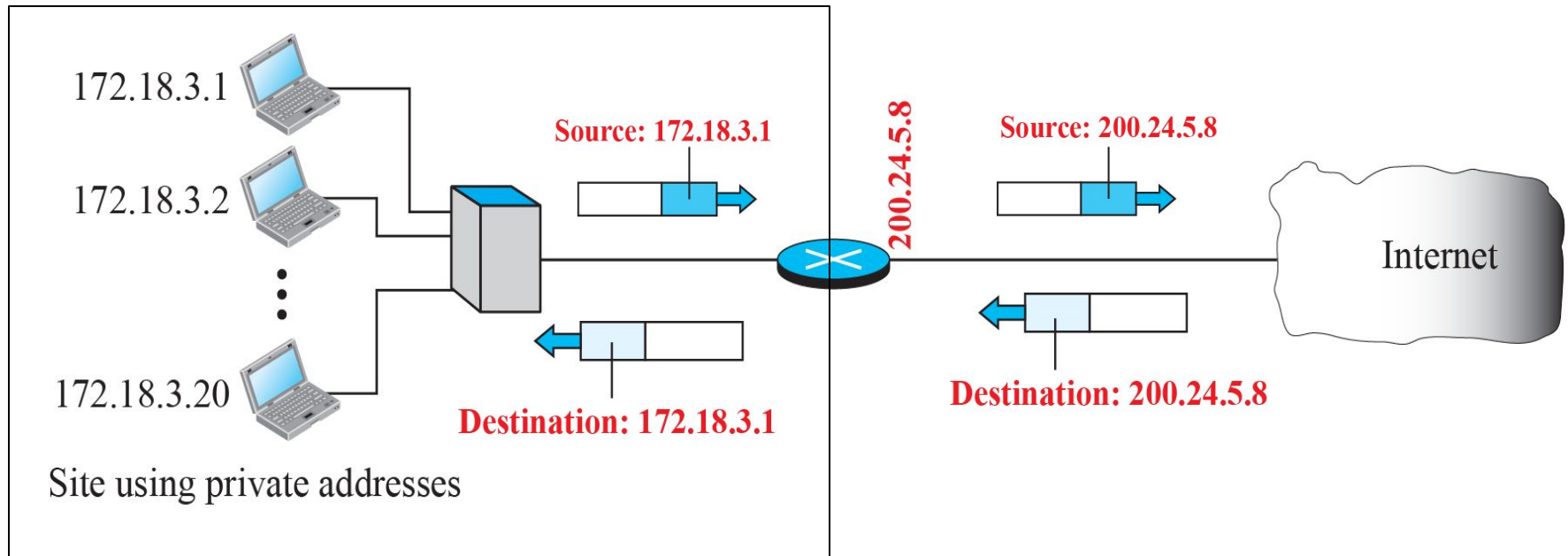
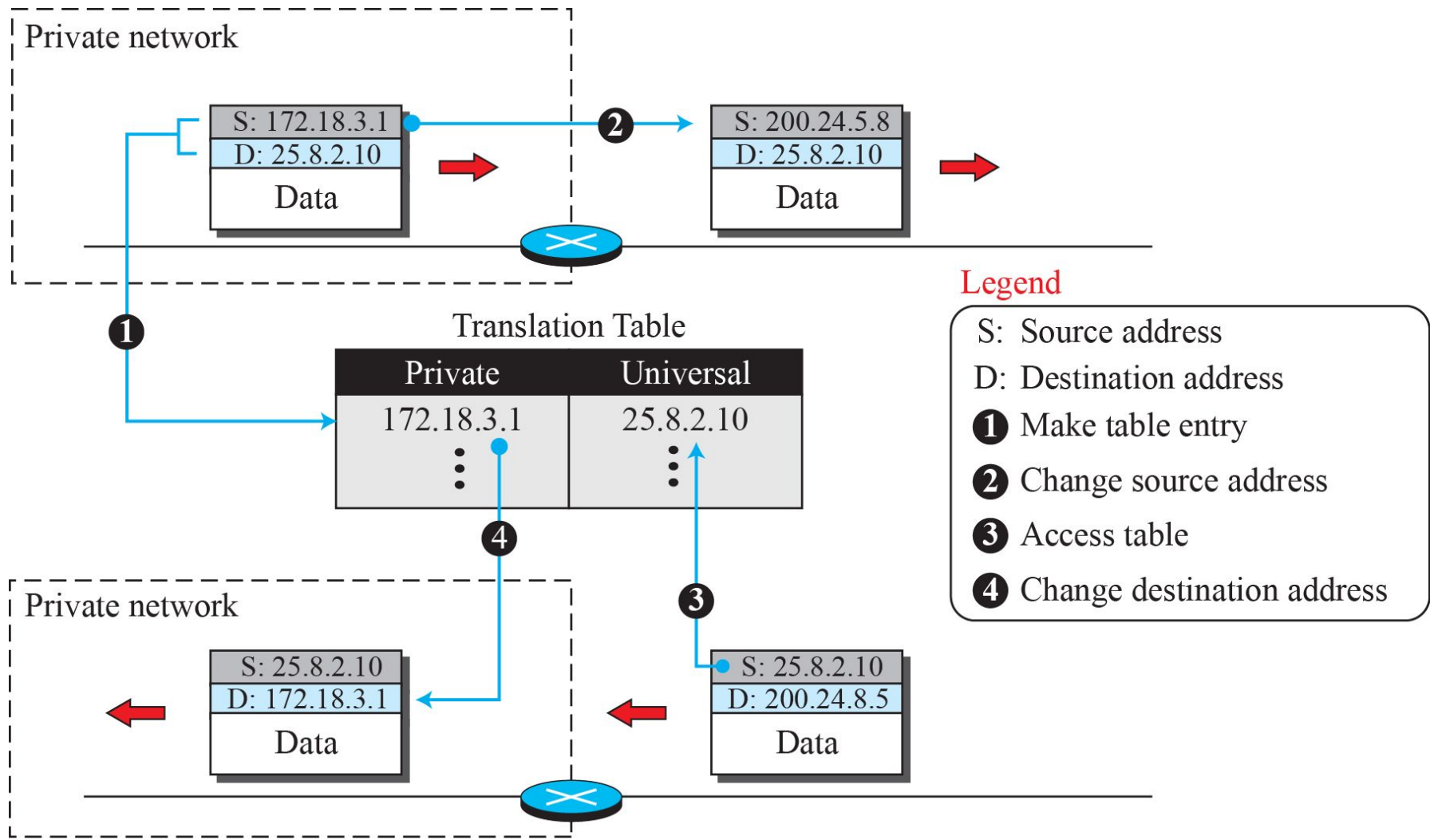


Figure 4.44: Translation



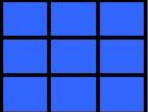
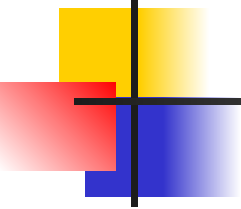


Table 4.1: Five-column translation table

<i>Private address</i>	<i>Private port</i>	<i>External address</i>	<i>External port</i>	<i>Transport protocol</i>
172.18.3.1	1400	25.8.3.2	80	TCP
172.18.3.2	1401	25.8.3.2	80	TCP
⋮	⋮	⋮	⋮	⋮



4.2.3 Forwarding of IP Packets

We discussed the concept of forwarding at the network layer earlier in this chapter. In this section, we extend the concept to include the role of IP addresses in forwarding. As we discussed before, forwarding means to place the packet in its route to its destination. Since the Internet today is made of a combination of links (networks), forwarding means to deliver the packet to the next hop (which can be the final destination or the intermediate connecting device).



4.2.3 (continued)

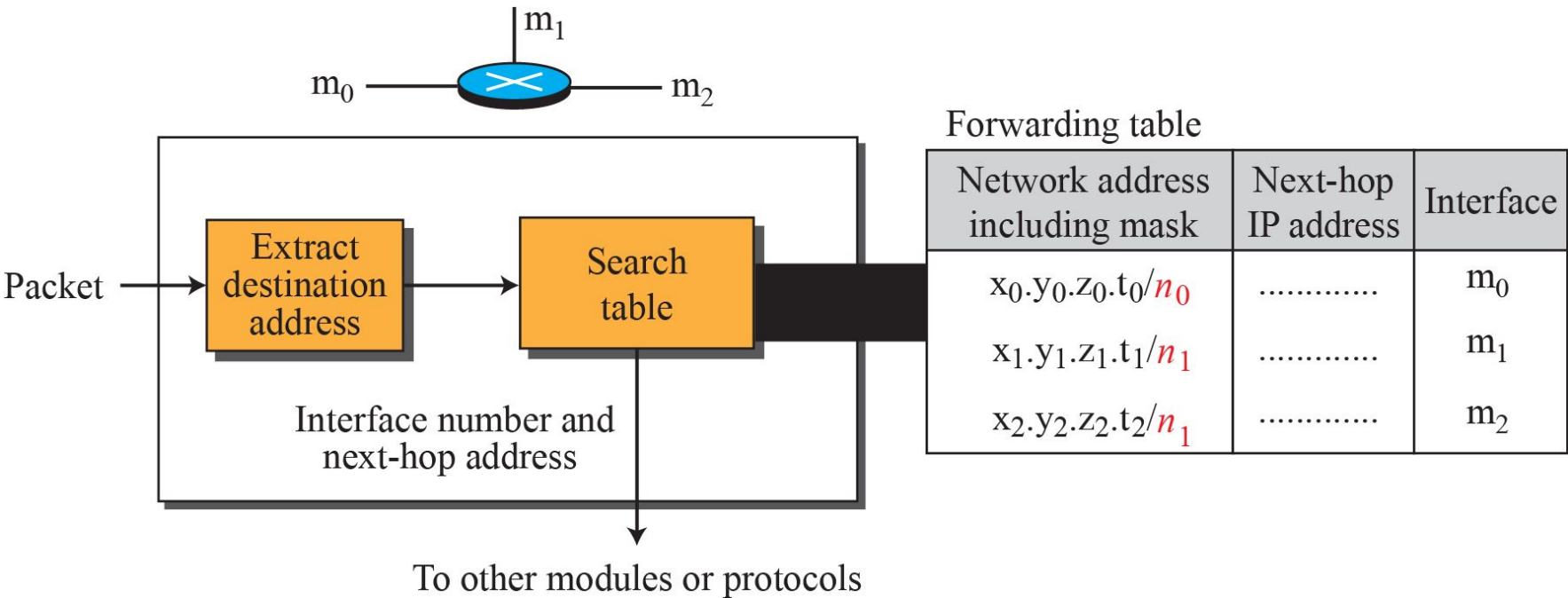
Forwarding Based On Destination Address

- ❖ *Address Aggregation*
- ❖ *Longest Mask Matching*
- ❖ *Hierarchical Routing*
- ❖ *Geographical Routing*
- ❖ *Forwarding Table Search Algorithms*

Forwarding Based on Label

- ❖ *Multi-Protocol Label Switching (MPLS)*
- ❖ *A New Header*
- ❖ *Hierarchical Routing*

Figure 4.45: *Simplified forwarding module in classless address*



Example 4.7

Make a forwarding table for router R1 using the configuration in Figure 4.46.

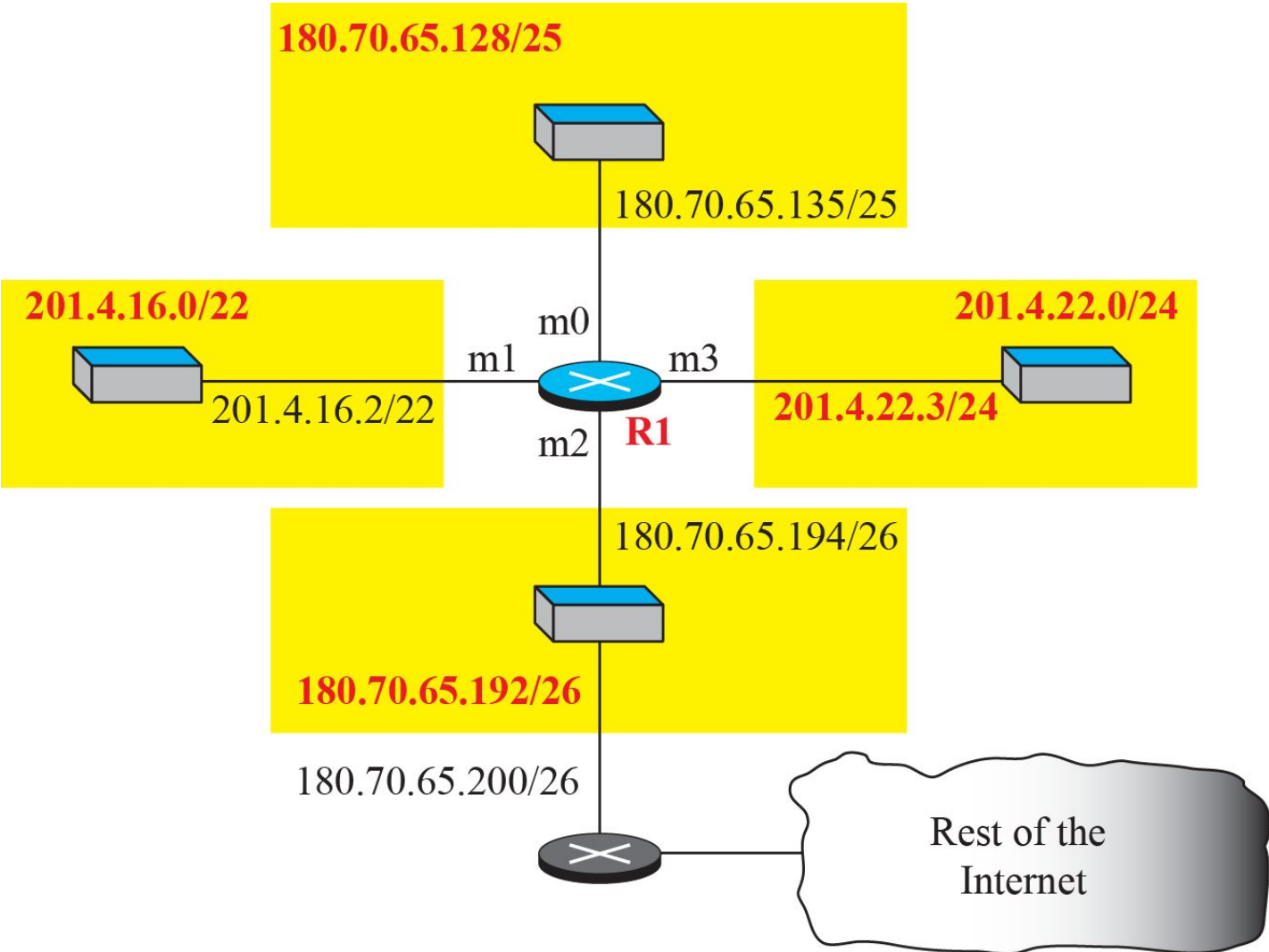
Solution

Table 4.2 shows the corresponding table.

Table 4.2: Forwarding table for router R1 in Figure 4.46

<i>Network address/mask</i>	<i>Next hop</i>	<i>Interface</i>
180.70.65.192/ 26	—	m2
180.70.65.128/ 25	—	m0
201.4.22.0/ 24	—	m3
201.4.16.0/ 22	—	m1
Default	180.70.65.200	m2

Figure 4.46: Configuration for Example 4.7



Example 4.8

Instead of Table 4.2, we can use Table 4.3, in which the network address/mask is given in bits.

Table 4.3: Forwarding table for router R1 using prefix bits

<i>Leftmost bits in the destination address</i>	<i>Next hop</i>	<i>Interface</i>
10110100 01000110 01000001 11	—	m2
10110100 01000110 01000001 1	—	m0
11001001 00000100 00011100	—	m3
11001001 00000100 000100	—	m1
Default	180.70.65.200	m2

When a packet arrives whose leftmost 26 bits in the destination address match the bits in the first row, the packet is sent out from interface m2. And so on.

Example 4.9

Show the forwarding process if a packet arrives at R1 in Figure 4.46 with the destination address 180.70.65.140.

Solution

The router performs the following steps:

1. The first mask (/26) is applied to the destination address. The result is 180.70.65.128, which does not match the corresponding network address.
2. The second mask (/25) is applied to the destination address. The result is 180.70.65.128, which matches the corresponding network address. The next-hop address and the interface number m0 are extracted for forwarding the packet (see Chapter 5).

Figure 4.47: Address aggregation

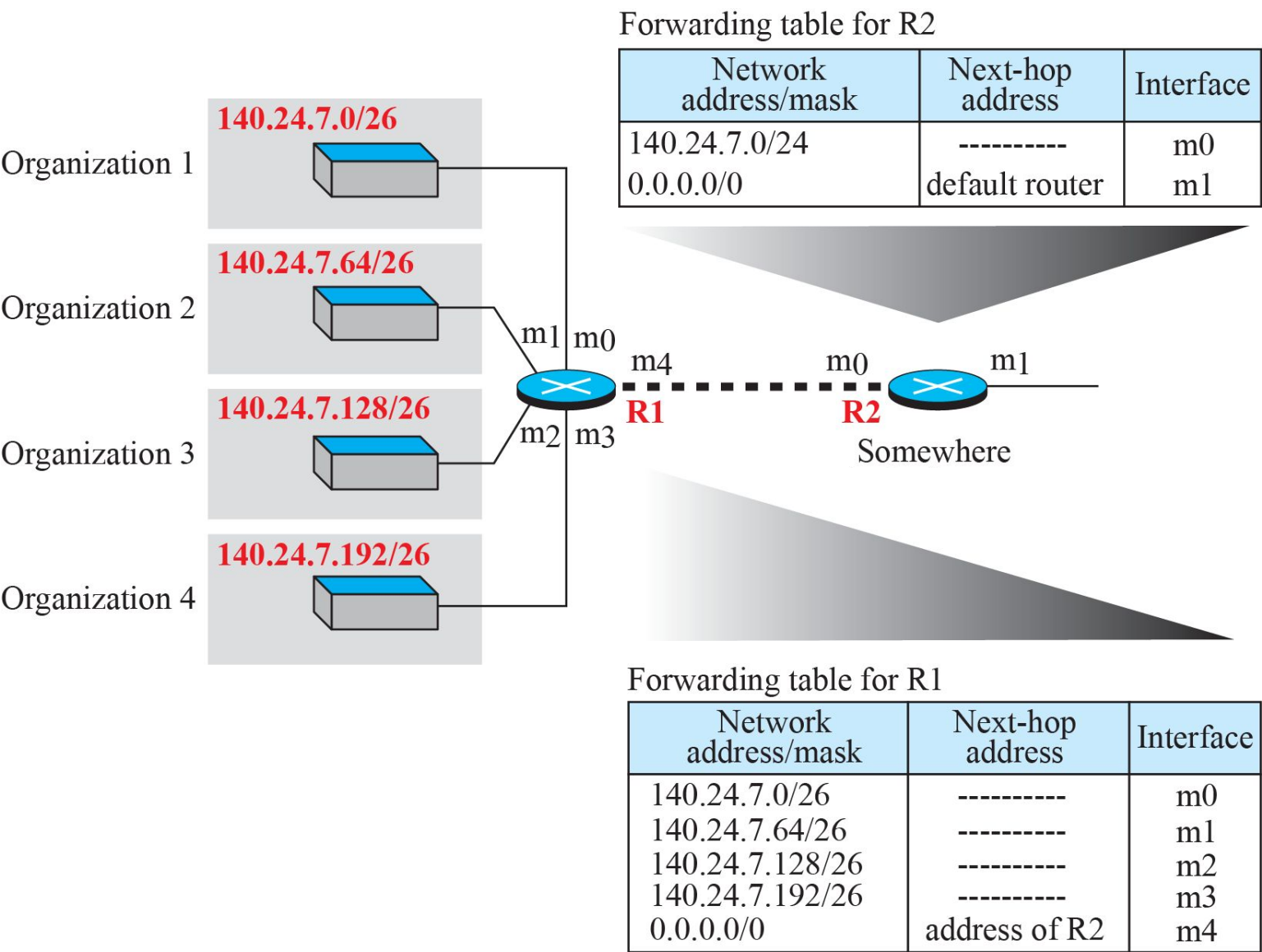
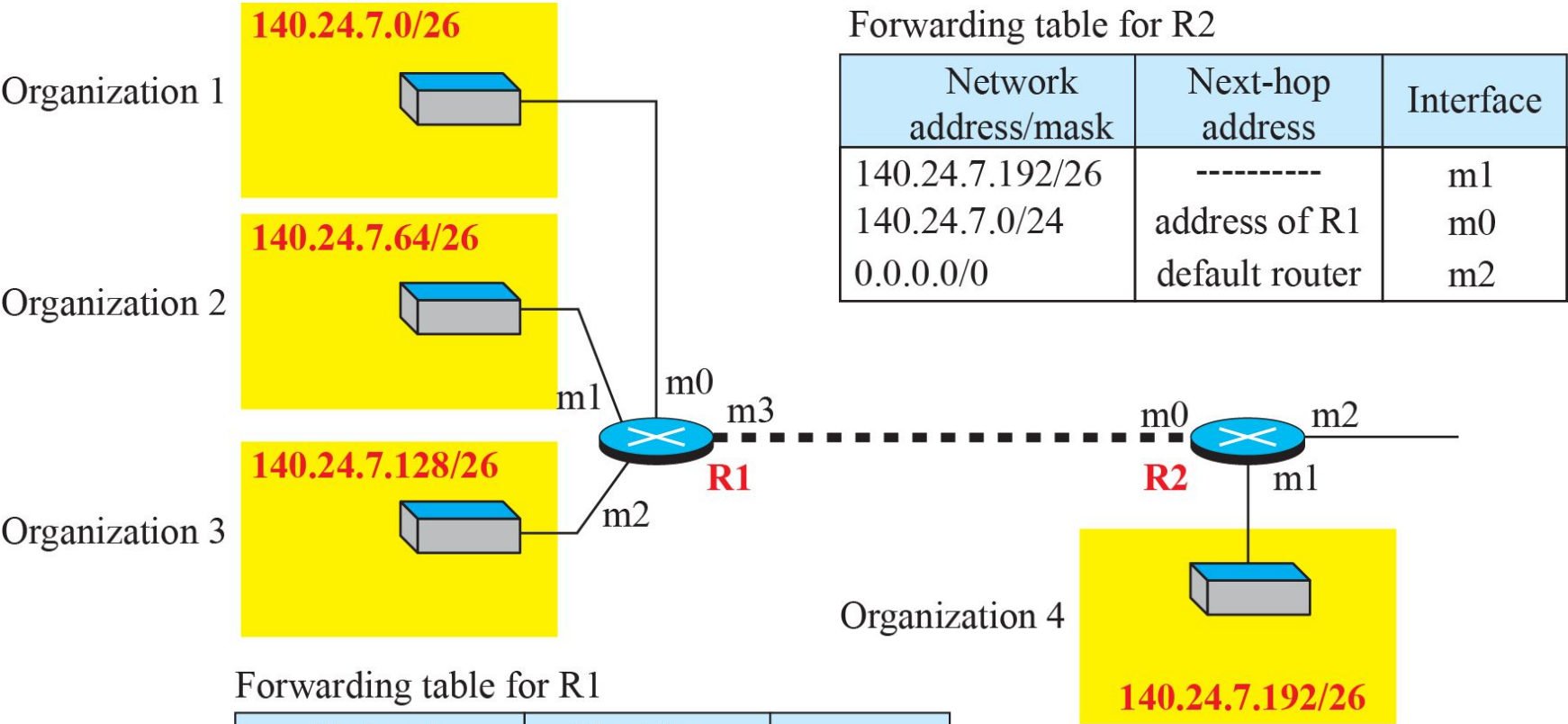


Figure 4.48: Longest mask matching



Forwarding table for R1

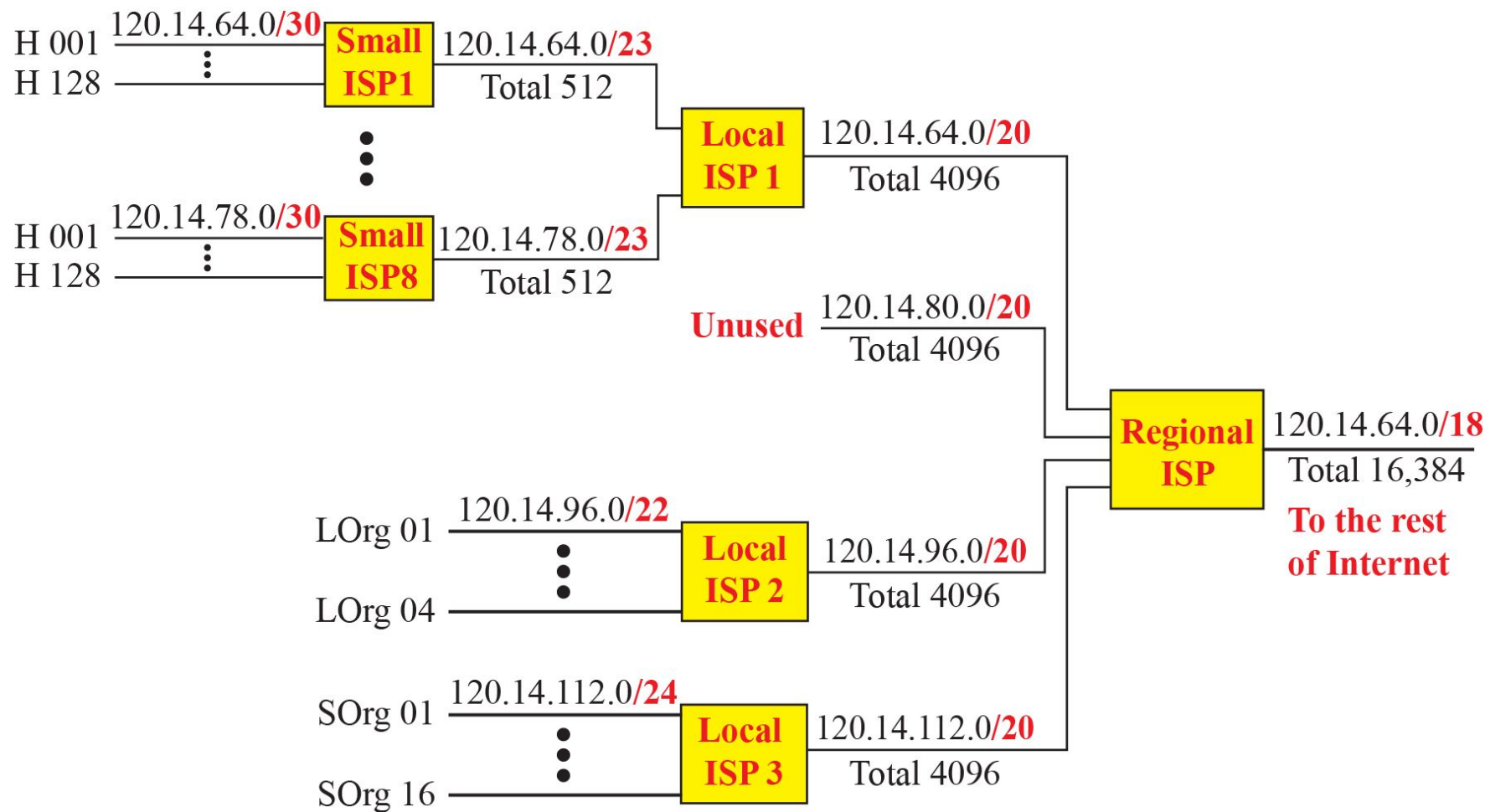
Network address/mask	Next-hop address	Interface
140.24.7.0/26	-----	m0
140.24.7.64/26	-----	m1
140.24.7.128/26	-----	m2
0.0.0.0/0	default router	m3

Example 4.10

As an example of hierarchical routing, let us consider Figure 4.49. A regional ISP is granted 16,384 addresses starting from 120.14.64.0. The regional ISP has decided to divide this block into 4 subblocks, each with 4096 addresses. Three of these subblocks are assigned to three local ISPs, the second subblock is reserved for future use. Note that the mask for each block is **/20** because the original block with mask /18 is divided into 4 blocks.

The figure also shows how local and small ISPs have assigned addresses.

Figure 4.49: Hierarchical routing with ISPs

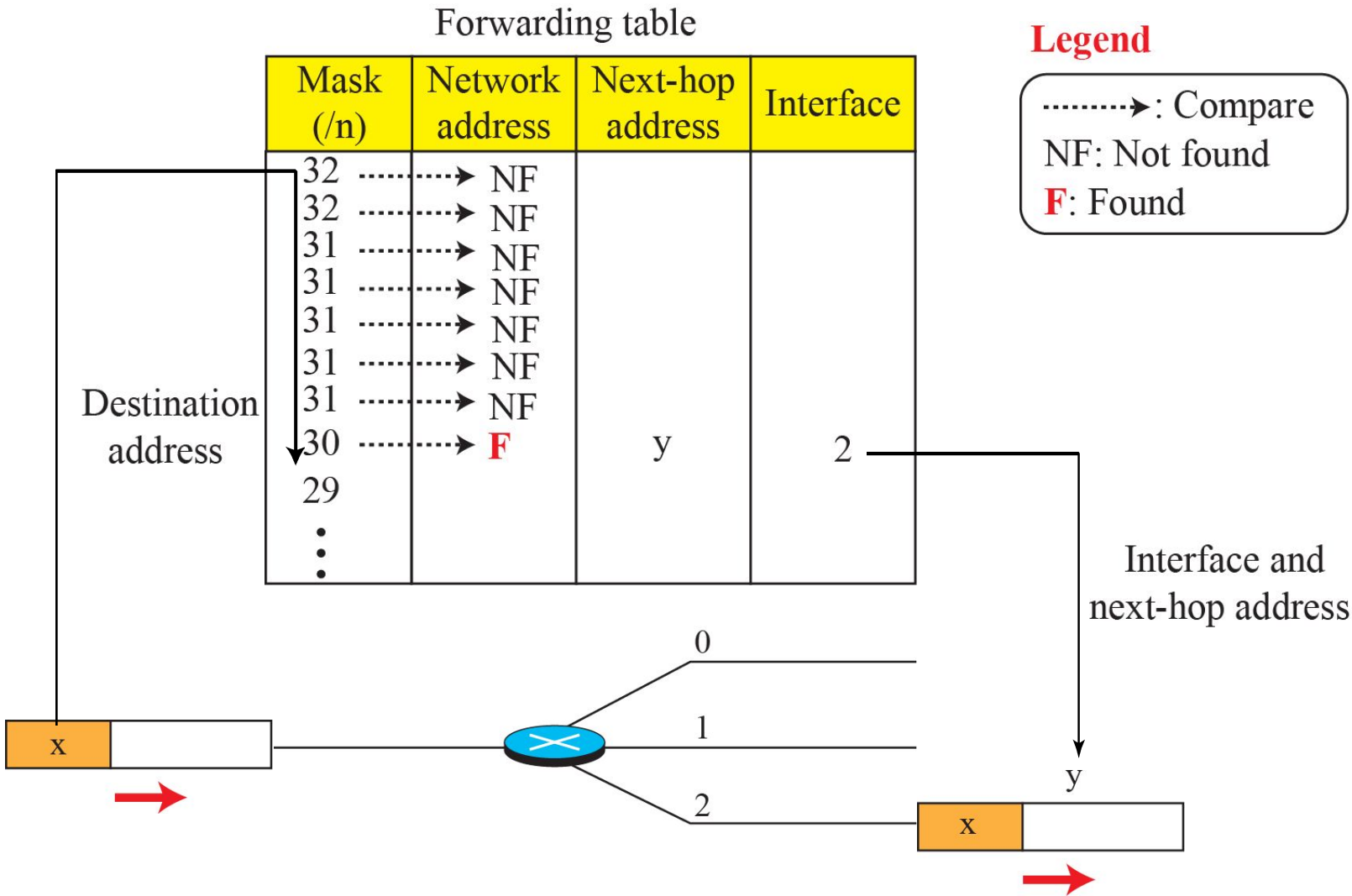


Example 4.11

Figure 4.50 shows a simple example of searching in a forwarding table using the longest mask algorithm. Although there are some more efficient algorithms today, the principle is the same.

When the forwarding algorithm gets the destination address of the packet, it needs to delve into the mask column. For each entry, it needs to apply the mask to find the destination network address. It then needs to check the network addresses in the table until it finds the match. The router then extracts the next-hop address and the interface number to be delivered to the data-link layer.

Figure 4.50: Example 4.11: Forwarding based on destination address



Example 4.12

Figure 4.51 shows a simple example of using a label to access a switching table. Since the labels are used as the index to the table, finding the information in the table is immediate.

Figure 4.51: Example 4.12: Forwarding based on label

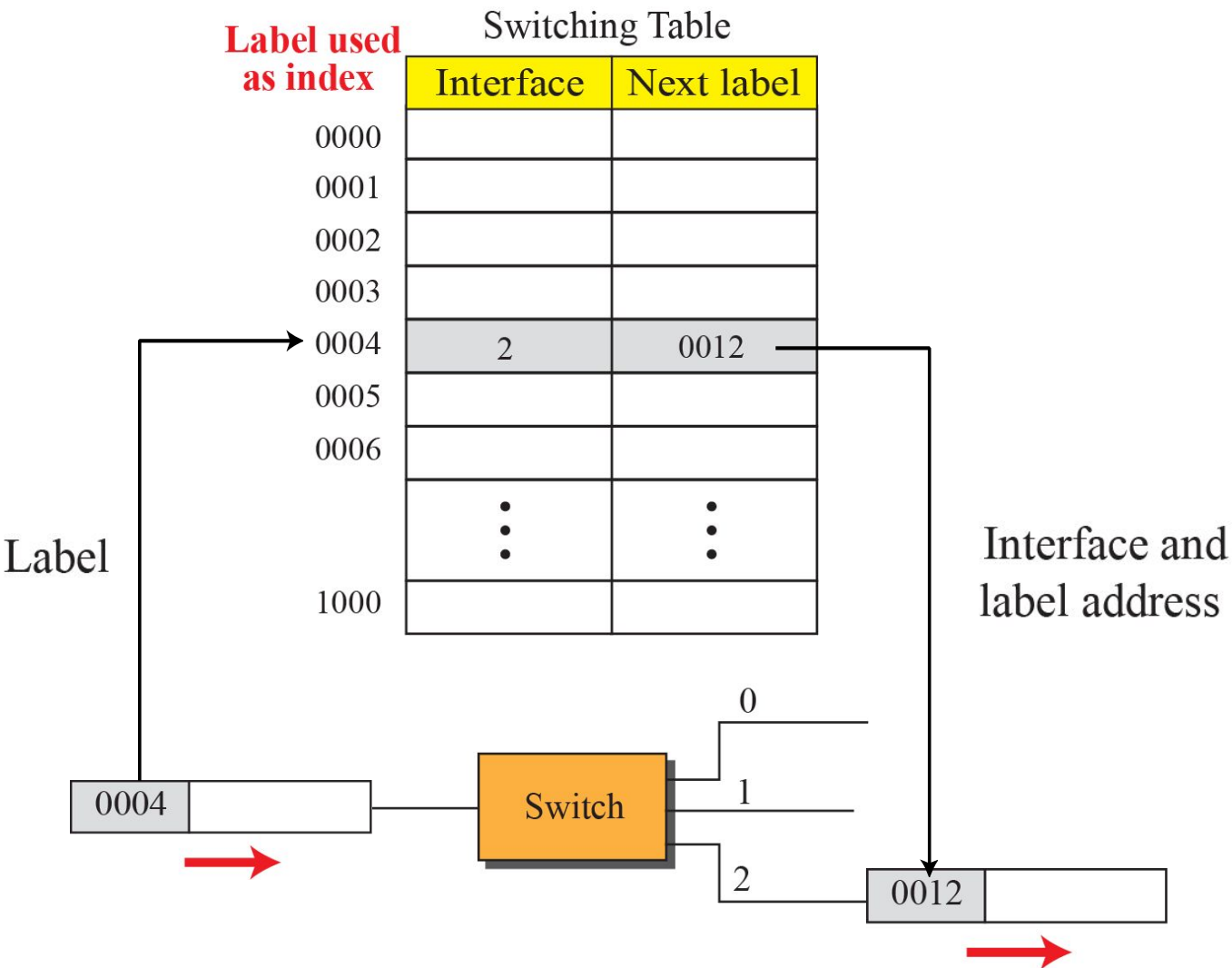


Figure 4.52: *MPLS header added to an IP packet*

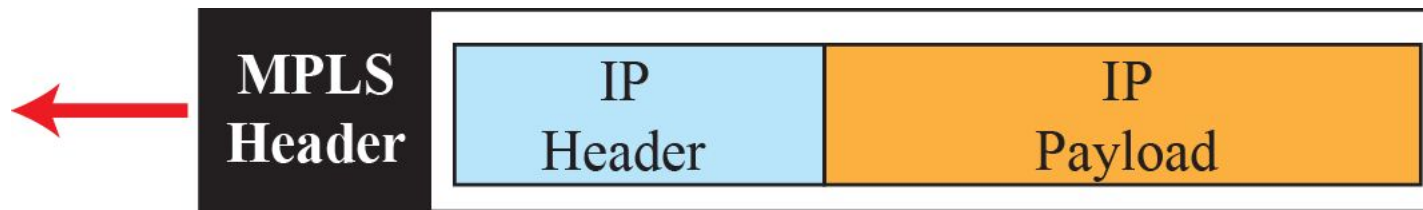
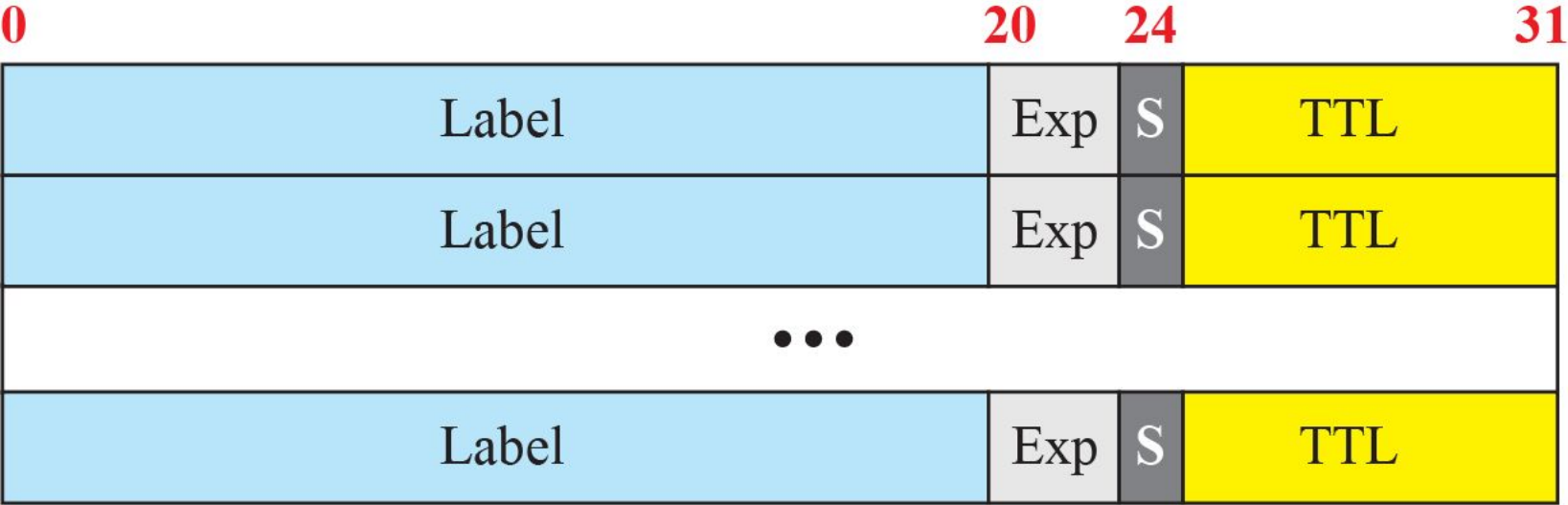


Figure 4.53: *MPLS header made of a stack of labels*



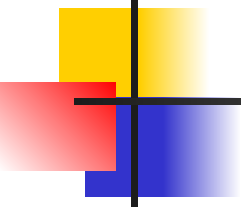


4.2.4 ICMPv4

The IPv4 has no error-reporting or error-correcting mechanism. What happens if something goes wrong? There are examples of situations where an error has occurred and the IP protocol has no built-in mechanism to notify the original host.

The IP protocol also lacks a mechanism for host and management queries.

The Internet Control Message Protocol version 4 (ICMPv4) has been designed to compensate for the above two deficiencies.

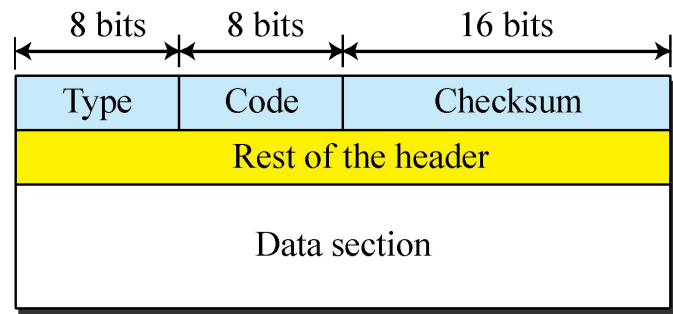


4.2.4 (continued)

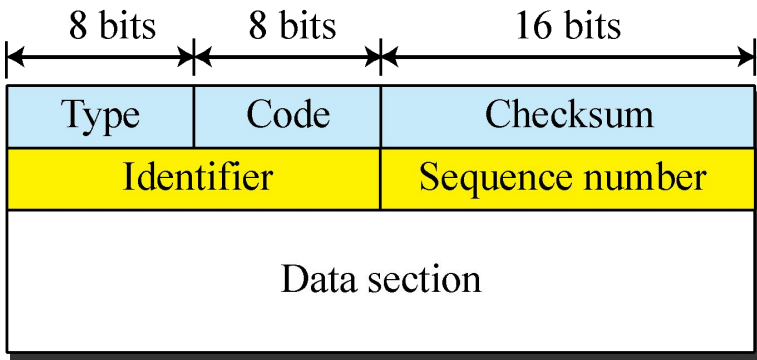
Messages

- ◆ *Message Format*
- ◆ *Error Reporting Messages*
- ◆ *Query Messages*

Figure 4.54: General format of ICMP messages



Error-reporting messages



Query messages

Type and code values

Error-reporting messages

- 03: Destination unreachable (codes 0 to 15)
- 04: Source quench (only code 0)
- 05: Redirection (codes 0 to 3)
- 11: Time exceeded (codes 0 and 1)
- 12: Parameter problem (codes 0 and 1)

Query messages

- 08 and 00: Echo request and reply (only code 0)
- 13 and 14: Timestamp request and reply (only code 0)

Note: See the book website for more explanation about the code values.

Example 4.13

One of the tools that a host can use to test the liveness of another host is the ping program. The ping program takes advantage of the ICMP echo request and echo reply messages. A host can send an echo request (type 8, code 0) message to another host, which, if alive, can send back an echo reply (type 0, code 0) message. To some extent, the ping program can also measure the reliability and congestion of the router between the two hosts by sending a set of request-reply messages.

Example 4.13 (continued)

The following shows how we send a ping message to the *auniversity.edu* site.

```
$ ping auniversity.edu
```

```
PING auniversity.edu (152.181.8.3) 56 (84) bytes of data.
```

```
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=0    ttl=62    time=1.91 ms
```

```
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=1    ttl=62    time=2.04 ms
```

```
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=2    ttl=62    time=1.90 ms
```

```
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=3    ttl=62    time=1.97 ms
```

```
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=4    ttl=62    time=1.93 ms
```

```
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=5    ttl=62    time=2.00 ms
```

```
--- auniversity.edu statistics ---
```

```
6 packets transmitted, 6 received, 0% packet loss
```

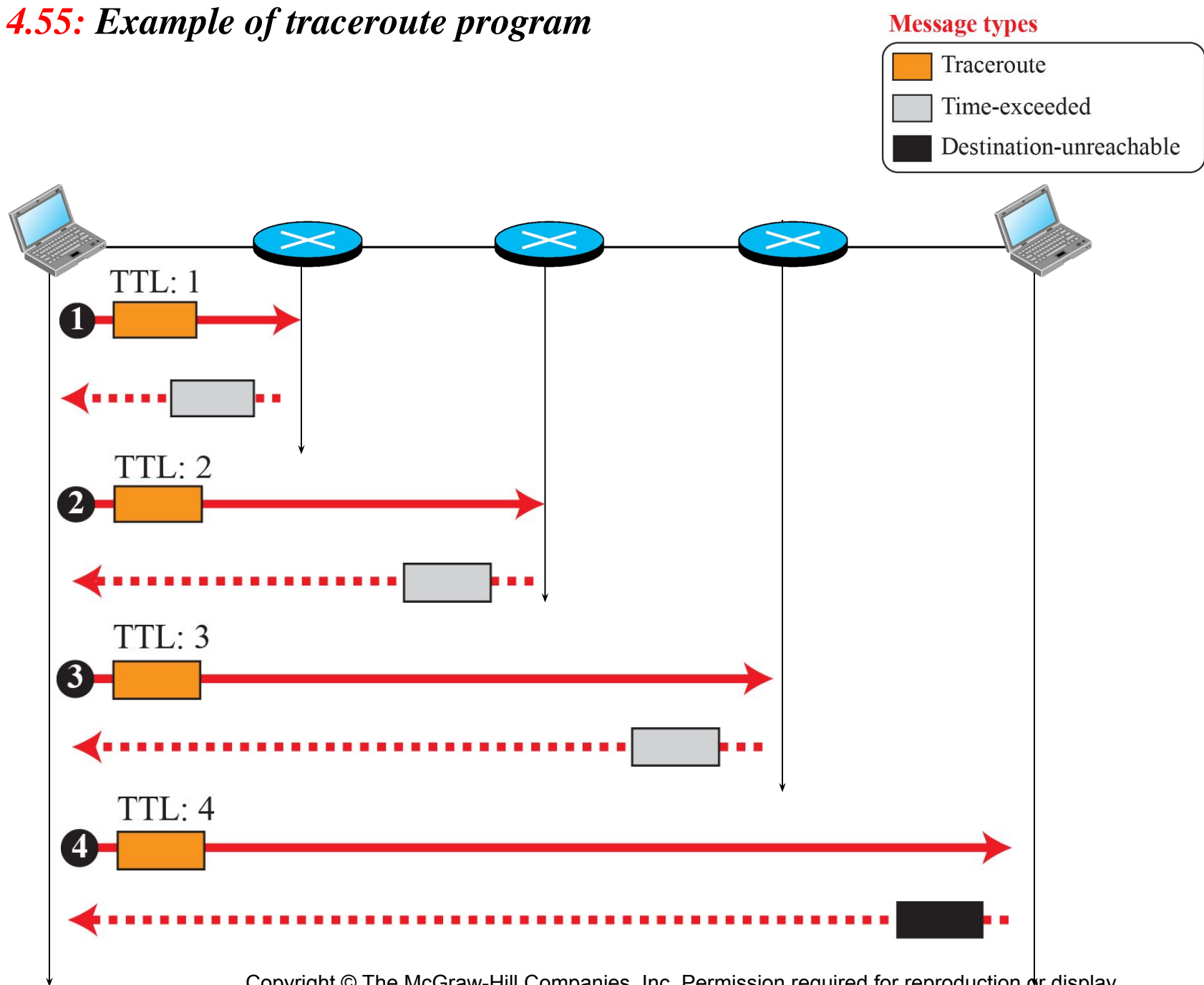
```
rtt min/avg/max = 1.90/1.95/2.04 ms
```

Example 4.14

The *traceroute* program in UNIX or *tracert* in Windows can be used to trace the path of a packet from a source to the destination. It can find the IP addresses of all the routers that are visited along the path. The program is usually set to check for the maximum of 30 hops (routers) to be visited. The number of hops in the Internet is normally less than this. The traceroute program is different from the ping program. The ping program gets help from two query messages; the traceroute program gets help from two error-reporting messages: time-exceeded and destination-unreachable

Figure 4.55 shows an example in which $n = 3$.

Figure 4.55: Example of traceroute program



Example 4.14 (continued)

The *traceroute* program also sets a timer to find the round-trip time for each router and the destination. Most *traceroute* programs send three messages to each device, with the same TTL value, to be able to find a better estimate for the round-trip time. The following shows an example of a *traceroute* program, which uses three probes for each device and gets three RTTs.

```
$ traceroute printers.com
```

```
traceroute to printers.com (13.1.69.93), 30 hops max, 38 byte packets
```

1	route.front.edu	(153.18.31.254)	0.622 ms	0.891 ms	0.875 ms
2	ceneric.net	(137.164.32.140)	3.069 ms	2.875 ms	2.930 ms
3	satire.net	(132.16.132.20)	3.071 ms	2.876 ms	2.929 ms
4	alpha.printers.com	(13.1.69.93)	5.922 ms	5.048 ms	4.922 ms

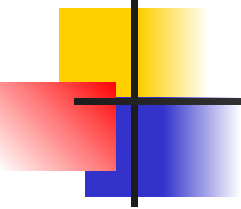
4-3 UNICAST ROUTING

In an internet, the goal of the network layer is to deliver a datagram from its source to its destination or destinations. If a datagram is destined for only one destination (one-to-one delivery), we have unicast routing. In this section and the next, we discuss only unicast routing; multicast and broadcast routing will be discussed later in the chapter.



4.3.1 General Idea

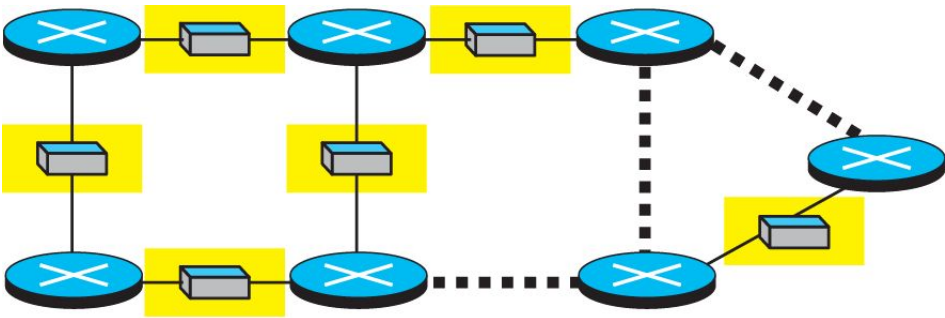
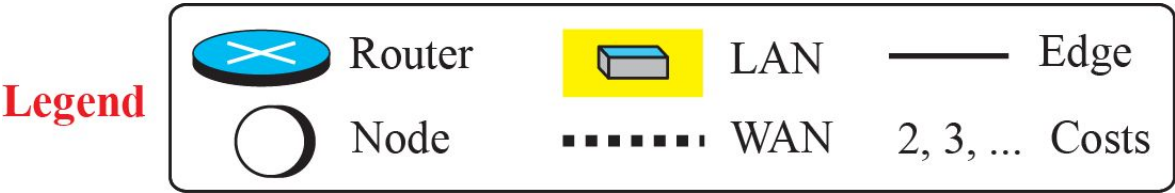
In unicast routing, a packet is routed, hop by hop, from its source to its destination by the help of forwarding tables. The source host needs no forwarding table because it delivers its packet to the default router in its local network. The destination host needs no forwarding table either because it receives the packet from its default router in its local network. This means that only the routers that glue together the networks in the internet need forwarding tables.



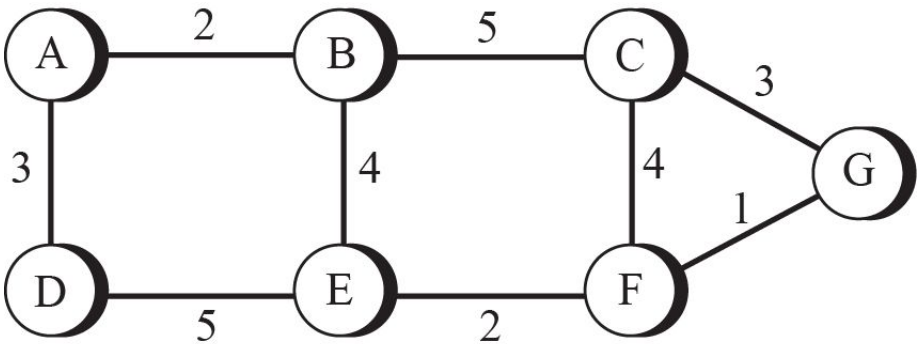
4.3.1 (continued)

- *An Internet as a Graph*
- *Least-Cost Routing*
- *Least-Cost Trees*

Figure 4.56: *An internet and its graphical representation*

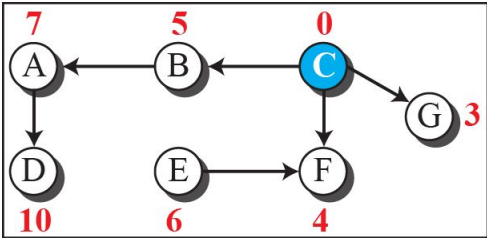
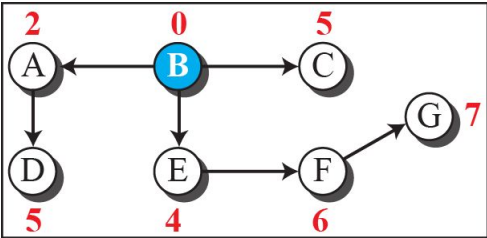
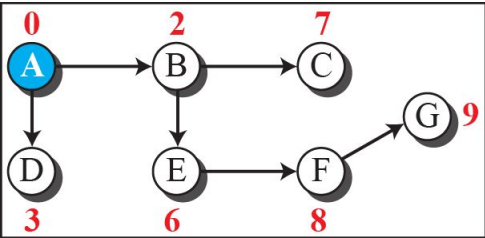


a. An internet



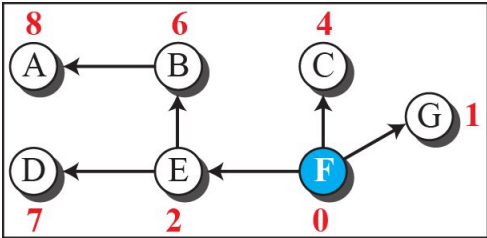
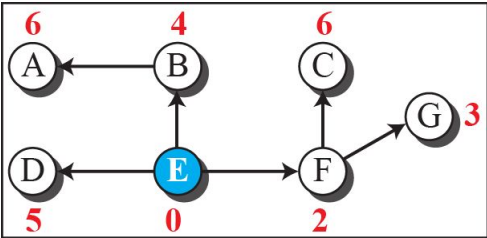
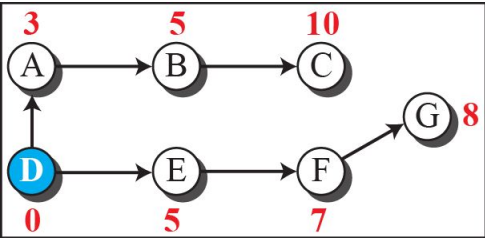
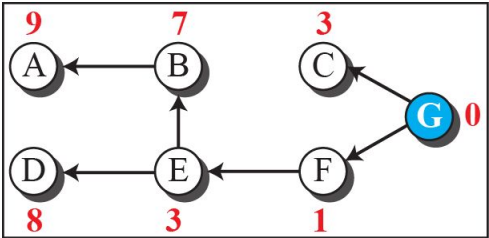
b. The weighted graph

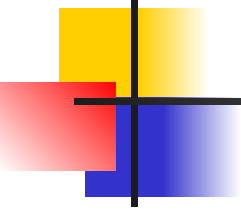
Figure 4.57: *Least-cost trees for nodes in the internet of Figure 4.56*



Legend

- Root of the tree
- Intermediate or end node
- 1, 2, ...** Total cost from the root





4.3.2 Routing Algorithm

After discussing the general idea behind least-cost trees and the forwarding tables that can be made from them, now we concentrate on the routing algorithms. Several routing algorithms have been designed in the past. The differences between these methods are in the way they interpret the least cost and the way they create the least-cost tree for each node. In this section, we discuss the common algorithm; later we show how a routing protocol in the Internet implements one of these algorithms.



4.3.2 (continued)

□ *Distance-Vector Routing*

- ◆ *Bellman-Ford Equation*
- ◆ *Distance Vectors*
- ◆ *Distance-Vector Routing Algorithm*
- ◆ *Count to Infinity*
- ◆ *Two-Node Loop*
 - *Split Horizon*
 - *Poisoned Reverse*
- ◆ *Three-Node Instability*



4.3.2 (continued)

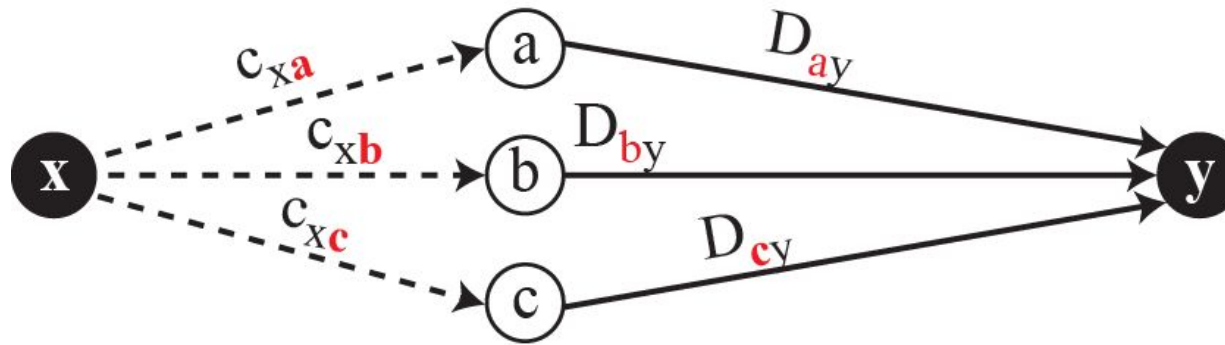
□ *Link-State Routing*

- ◆ *Link-State Database (LSDB)*
- ◆ *Least-Cost Trees (Dijkstra's Algorithm)*

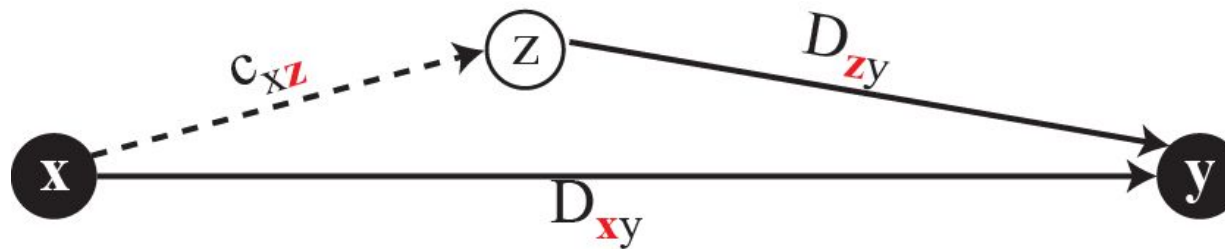
□ *Path-Vector Routing*

- ◆ *Spanning Trees*
- ◆ *Creation of Spanning Trees*
- ◆ *Path-Vector Algorithm*

Figure 4.58: Graphical idea behind Bellman-Ford equation

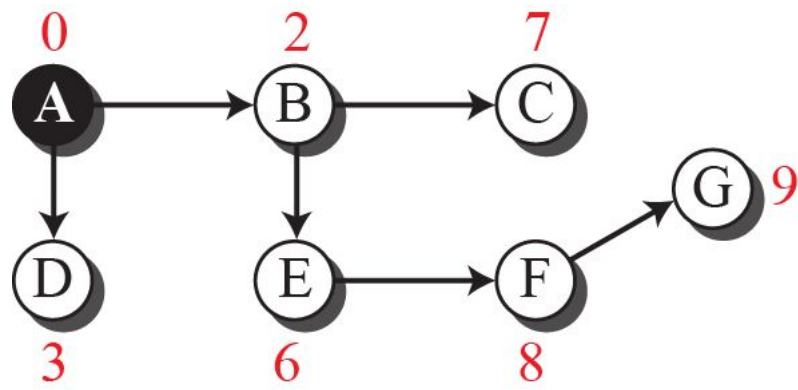


a. General case with three intermediate nodes



b. Updating a path with a new route

Figure 4.59: *The distance vector corresponding to a tree*



a. Tree for node A

	A
A	0
B	2
C	7
D	3
E	6
F	8
G	9

b. Distance vector for node A

Figure 4.60: *The first distance vector for an internet*

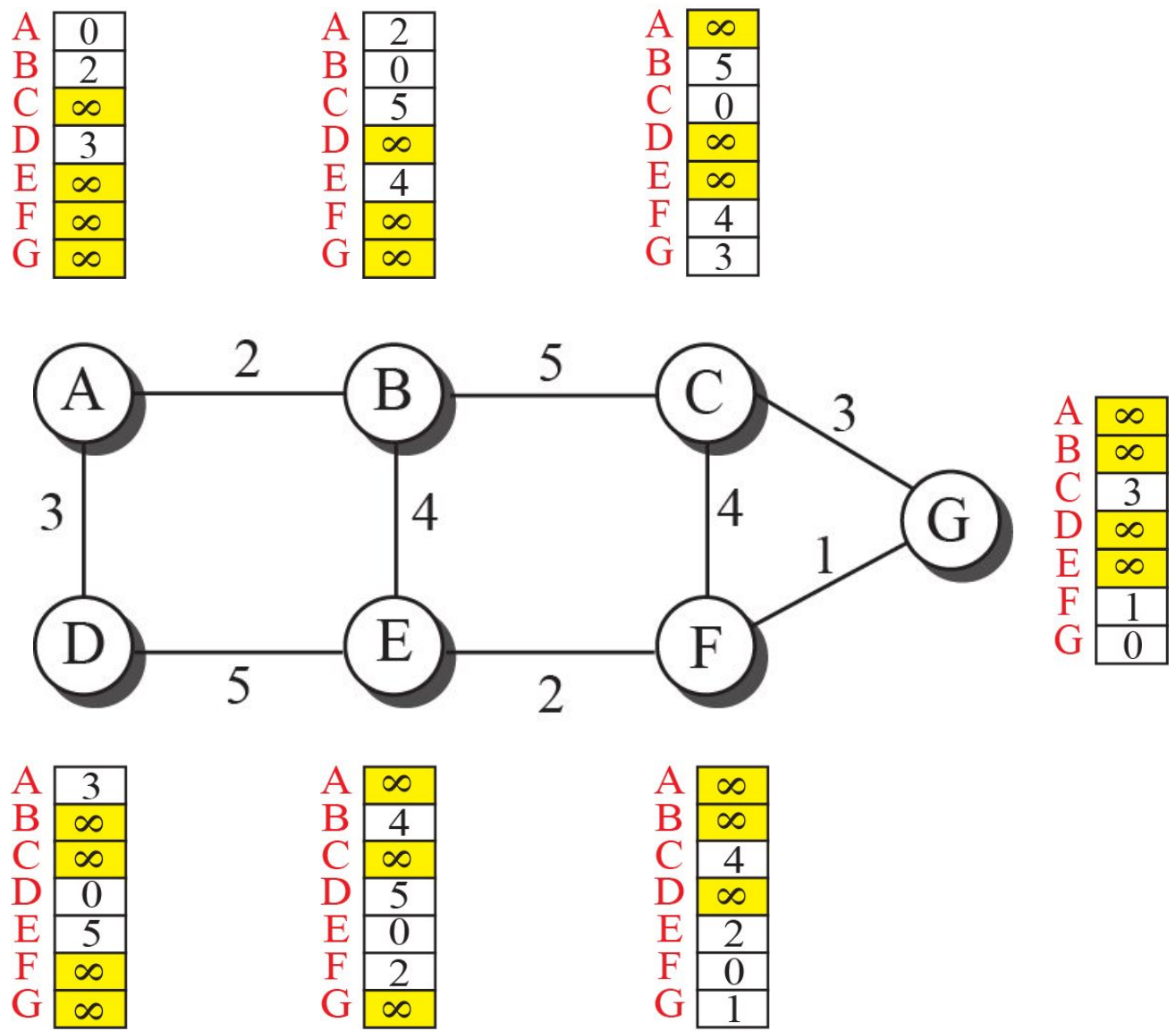


Figure 4.61: Updating distance vectors

New B		Old B		A	
A	2	A	2	A	0
B	0	B	0	B	2
C	5	C	5	C	∞
D	5	D	∞	D	3
E	4	E	4	E	∞
F	∞	F	∞	F	∞
G	∞	G	∞	G	∞

$B[] = \min (B[], 2 + A[])$

Note:

$X[]$: the whole vector

a. First event: B receives a copy of A's vector.

New B		Old B		E	
A	2	A	2	A	∞
B	0	B	0	B	4
C	5	C	5	C	∞
D	5	D	5	D	5
E	4	E	4	E	0
F	6	F	∞	F	2
G	∞	G	∞	G	∞

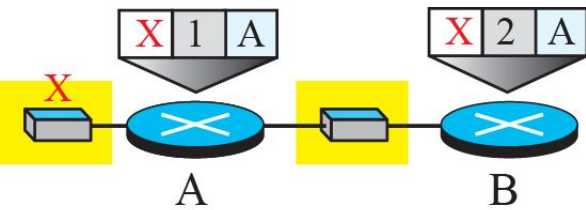
$B[] = \min (B[], 4 + E[])$

b. Second event: B receives a copy of E's vector.

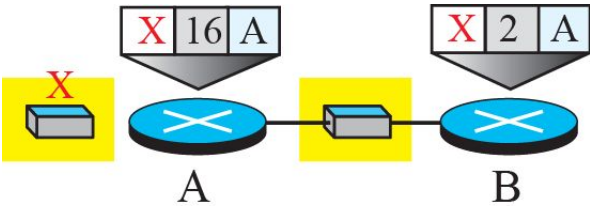
Table 4.4: Distance-Vector Routing Algorithm for A Node

```
1 Distance_Vector_Routing ( )
2 {
3     // Initialize (create initial vectors for the node)
4     D[myself] = 0
5     for (y = 1 to N)
6     {
7         if (y is a neighbor)
8             D[y] = c[myself][y]
9         else
10            D[y] =  $\infty$ 
11    }
12    send vector {D[1], D[2], ..., D[N]} to all neighbors
13    // Update (improve the vector with the vector received from a neighbor)
14    repeat (forever)
15    {
16        wait (for a vector  $D_w$  from a neighbor  $w$  or any change in the link)
17        for (y = 1 to N)
18        {
19            D[y] = min [D[y], (c[myself][w] +  $D_w$ [y])] // Bellman-Ford equation
20        }
21        if (any change in the vector)
22            send vector {D[1], D[2], ..., D[N]} to all neighbors
23    }
24 } // End of Distance Vector
```

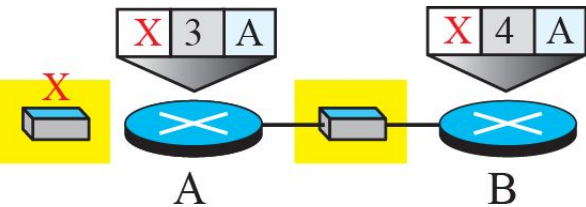
Figure 4.62: Two-node instability



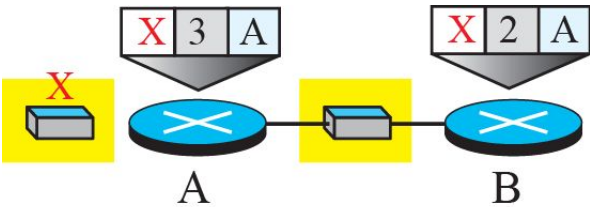
a. Before failure



b. After link failure

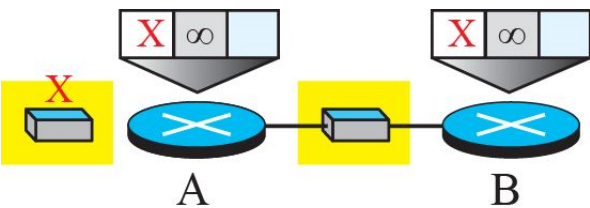


d. After B is updated by A



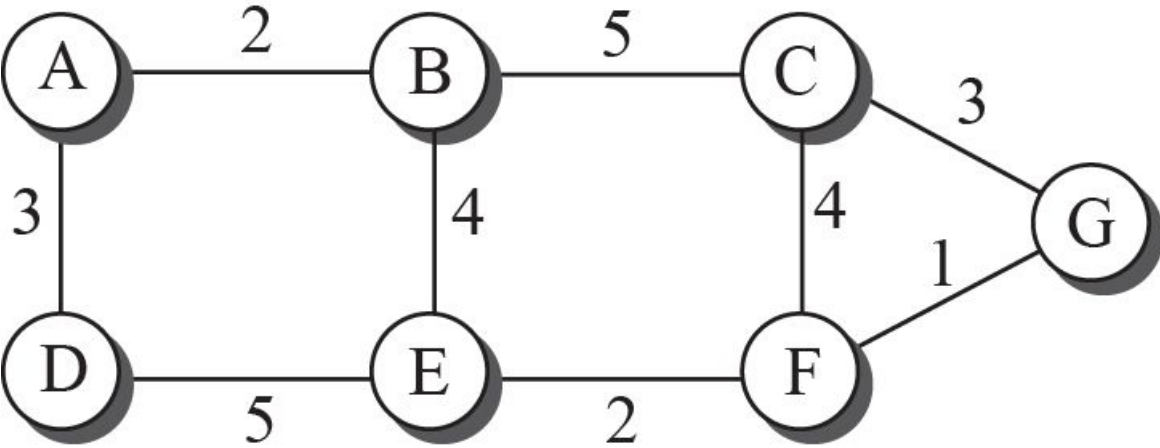
c. After A is updated by B

...



e. Finally

Figure 4.63: *Example of a link-state database*



a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

Figure 4.64: LSPs created and sent out by each node to build LSDB

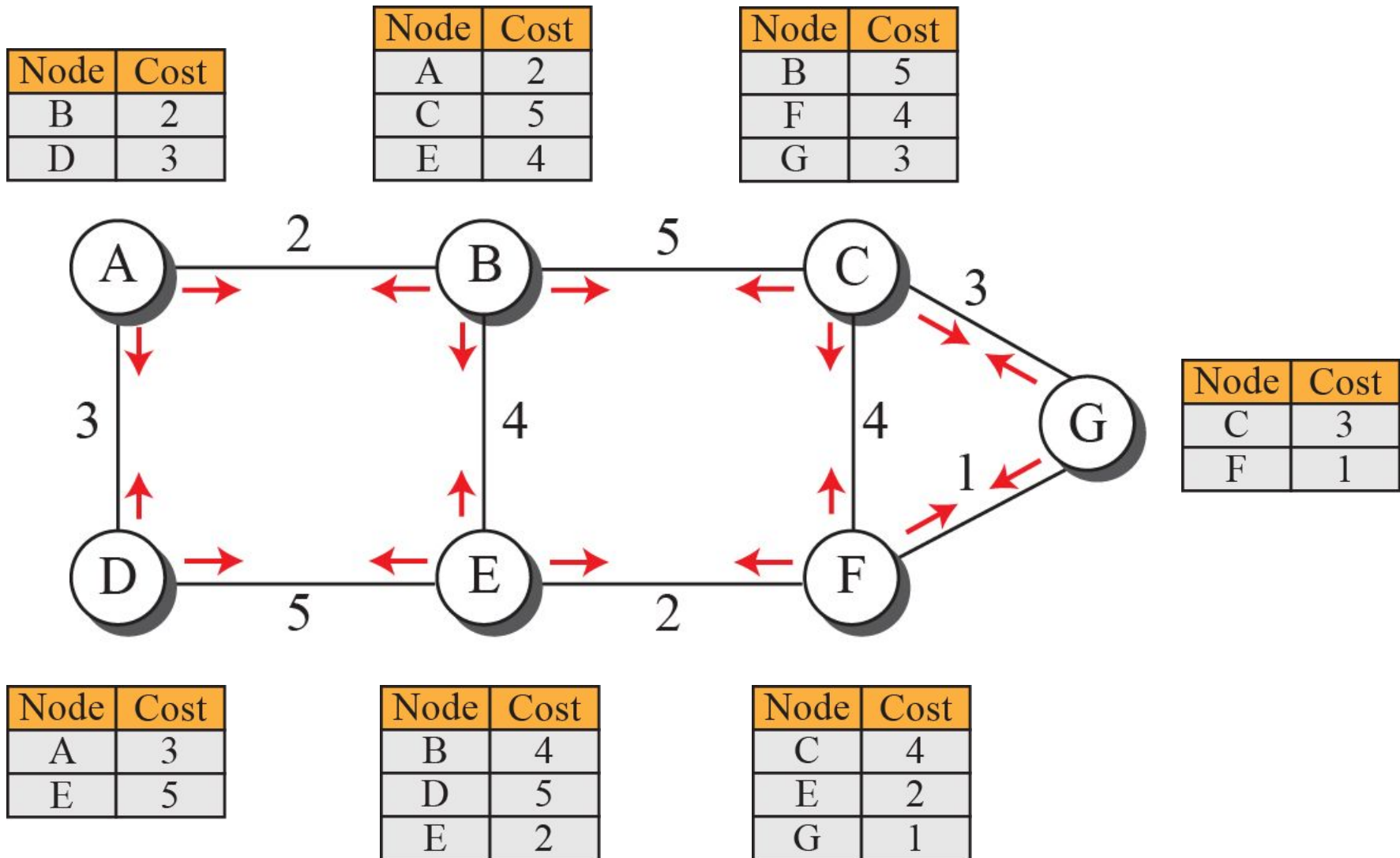


Table 4.5: Dijkstra's Algorithm

```
1  Dijkstra's Algorithm ( )
2  {
3      // Initialization
4      Tree = {root}                // Tree is made only of the root
5      for (y = 1 to N)             // N is the number of nodes
6      {
7          if (y is the root)
8              D[y] = 0              // D[y] is shortest distance from root to node y
9          else if (y is a neighbor)
10             D[y] = c[root][y]     // c[x][y] is cost between nodes x and y in LSDB
11          else
12             D[y] =  $\infty$ 
13      }
14      // Calculation
15      repeat
16      {
17          find a node w, with D[w] minimum among all nodes not in the Tree
18          Tree = Tree  $\cup$  {w}       // Add w to tree
19          // Update distances for all neighbor of w
20          for (every node x, which is neighbor of w and not in the Tree)
21          {
22              D[x] = min{D[x], (D[w] + c[w][x])}
23          }
24      } until (all nodes included in the Tree)
25  } // End of Dijkstra
```

Figure 4.65: Least-cost tree

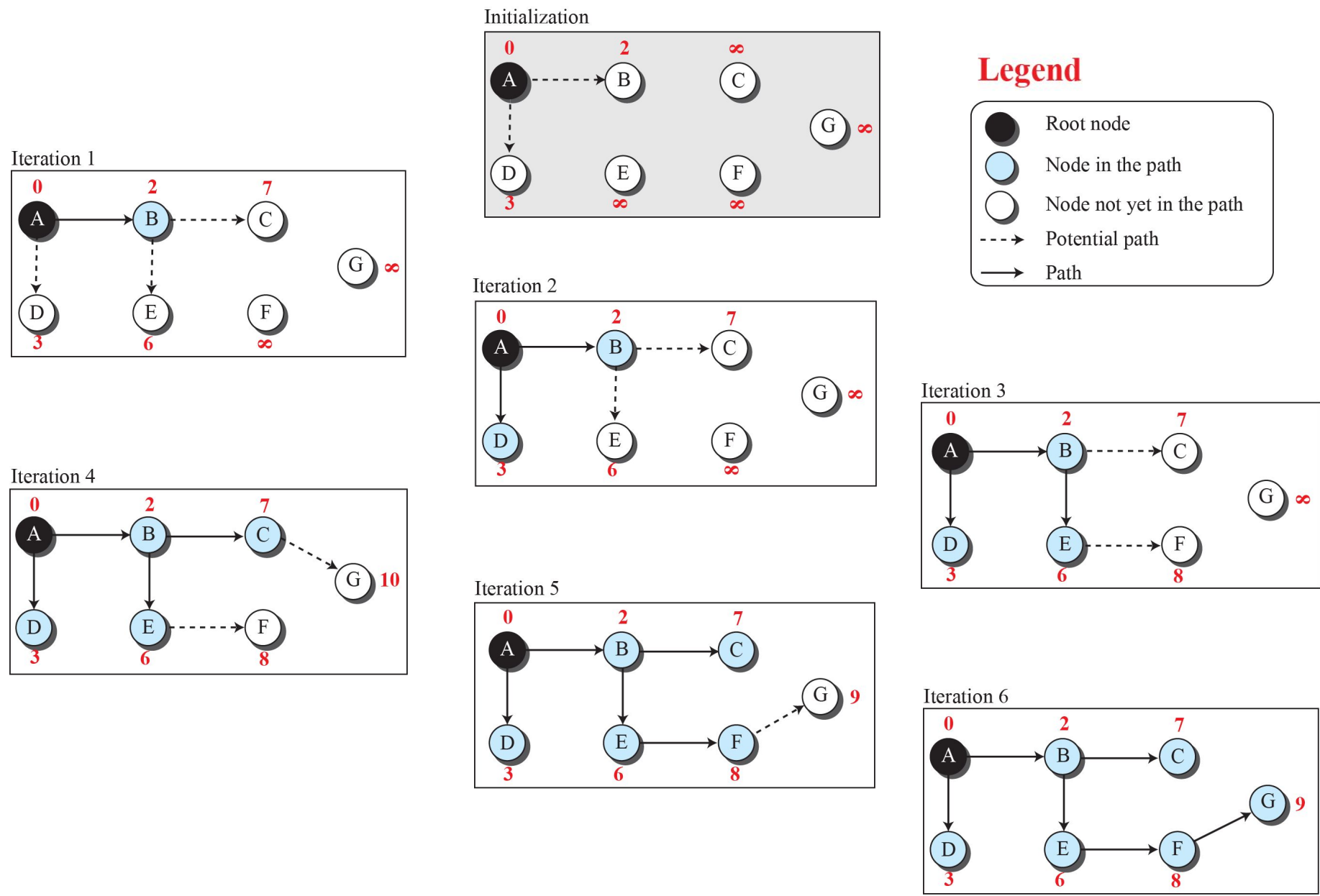
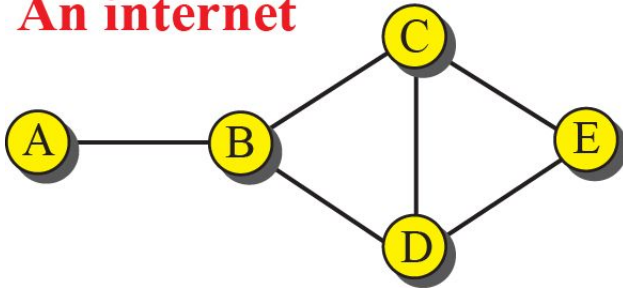
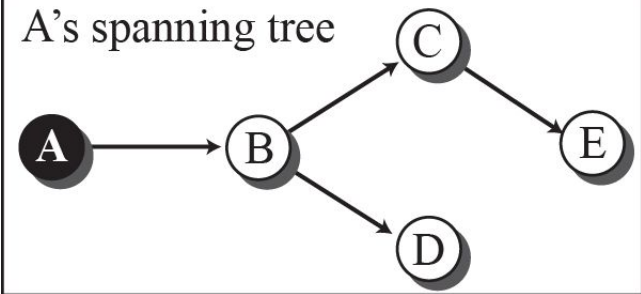


Figure 4.66: *Spanning trees in path-vector routing*

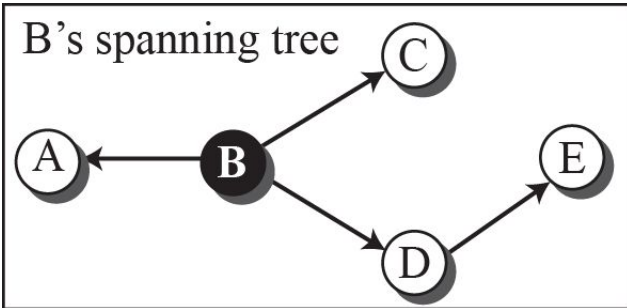
An internet



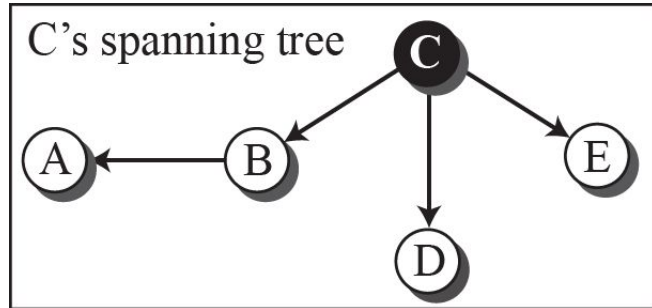
A's spanning tree



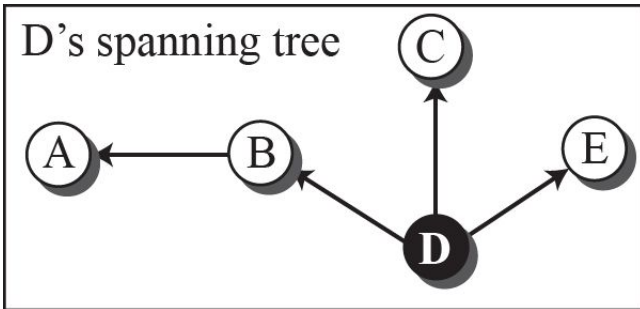
B's spanning tree



C's spanning tree



D's spanning tree



E's spanning tree

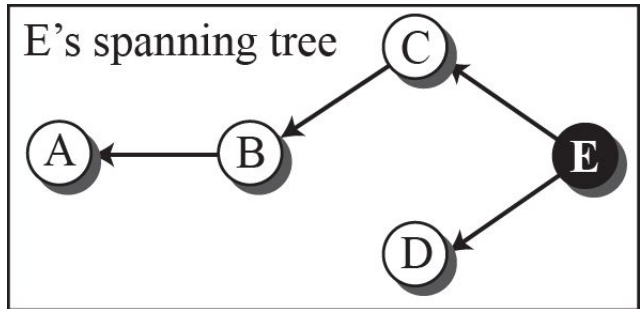


Figure 4.67: Path vectors made at booting time

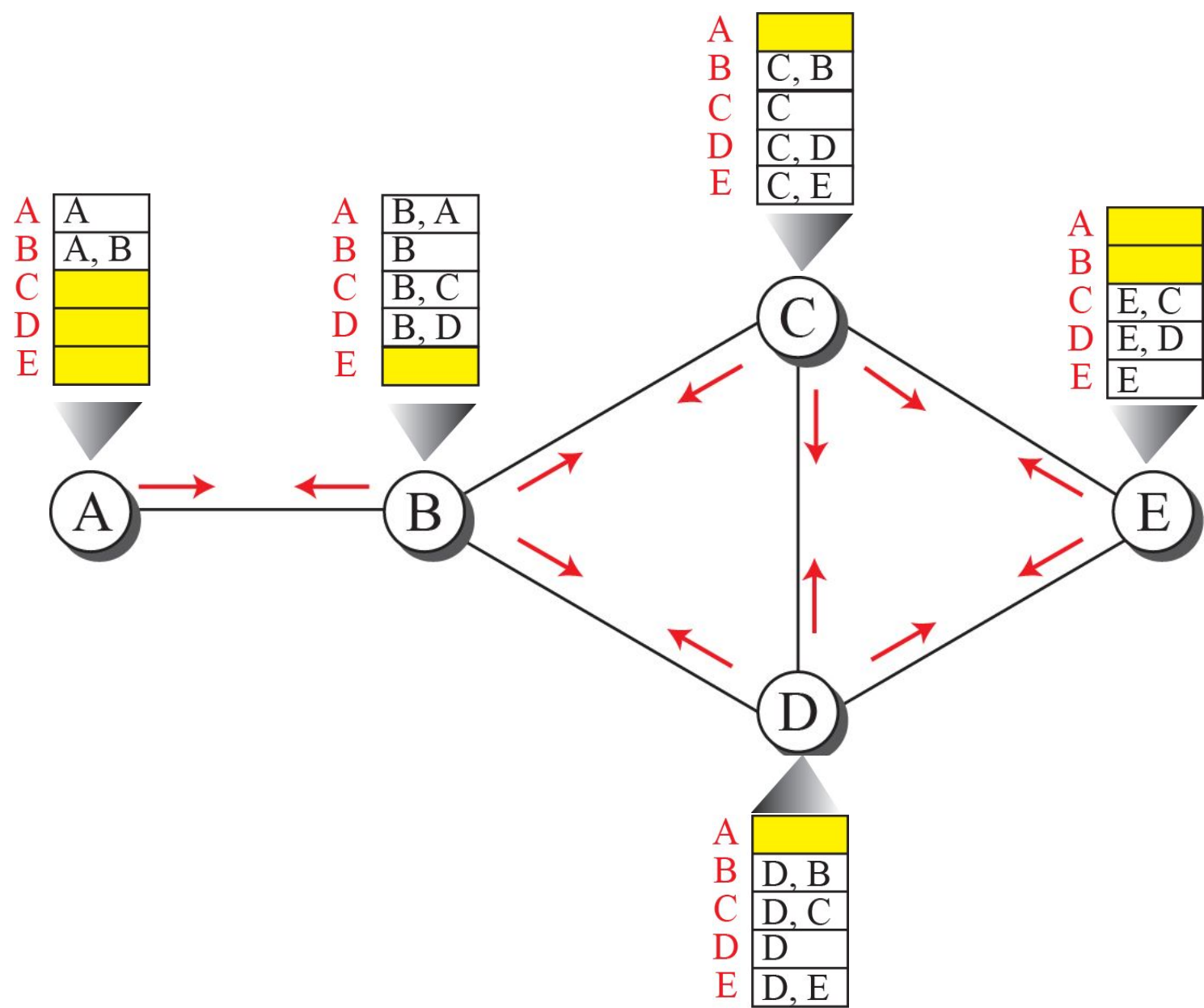
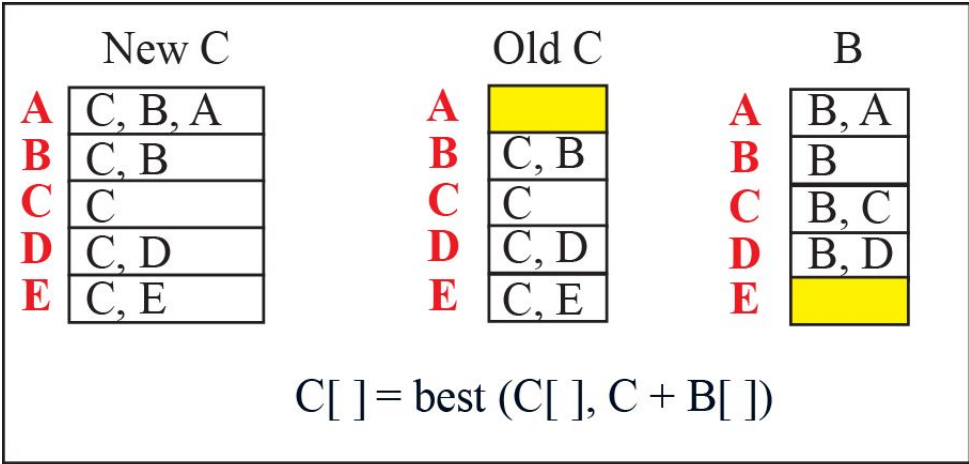
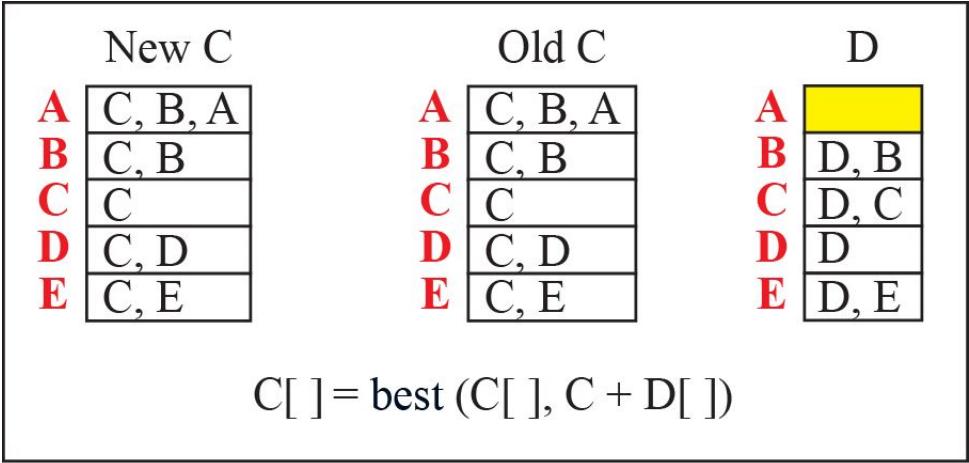


Figure 4.68: Updating path vectors



Note:
X []: vector X
Y: node Y

Event 1: C receives a copy of B's vector



Event 2: C receives a copy of D's vector

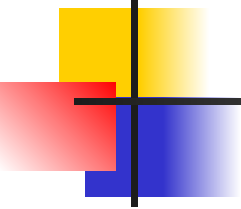
Table 4.6: Path-vector algorithm for a node

```
1 Path_Vector_Routing ( )
2 {
3     // Initialization
4     for (y = 1 to N)
5     {
6         if (y is myself)
7             Path[y] = myself
8         else if (y is a neighbor)
9             Path[y] = myself + neighbor node
10        else
11            Path[y] = empty
12    }
13    Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
14    // Update
15    repeat (forever)
16    {
17        wait (for a vector Pathw from a neighbor w)
18        for (y = 1 to N)
19        {
20            if (Pathw includes myself)
21                discard the path // Avoid any loop
22            else
23                Path[y] = best {Path[y], (myself + Pathw[y])}
24        }
25        If (there is a change in the vector)
26            Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
27    }
28 } // End of Path Vector
```




4.3.3 Unicast Routing Protocols

In the previous section, we discussed unicast routing algorithms; in this section, we discuss unicast routing protocols used in the Internet. Although three protocols we discuss here are based on the corresponding algorithms we discussed before, a protocol is more than an algorithm. A protocol needs to define its domain of operation, the messages exchanged, communication between routers, and interaction with protocols in other domains. After an introduction, we discuss three common protocols used in the Internet: RIP, OSPF, and BGP.



4.3.3 (continued)

□ *Internet Structure*

- ◆ *Hierarchical Routing*
- ◆ *Autonomous Systems*

□ *Routing Information Protocol (RIP)*

- ◆ *Hop Count*
- ◆ *Forwarding Tables*
- ◆ *RIP Implementation*
- ◆ *Performance*



4.3.3 (continued)

Open Shortest Path First (OSPF)

- ◆ *Metric*
- ◆ *Forwarding Tables*
- ◆ *Areas*
- ◆ *Link-State Advertisement*
- ◆ *OSPF Implementation*
- ◆ *Performance*

Border Gateway Protocol Version 4 (BGP4)

- ◆ *Introduction*
- ◆ *Path Attributes*
- ◆ *Route Selection*
- ◆ *Messages*
- ◆ *Performance*

Figure 4.69: Internet structure

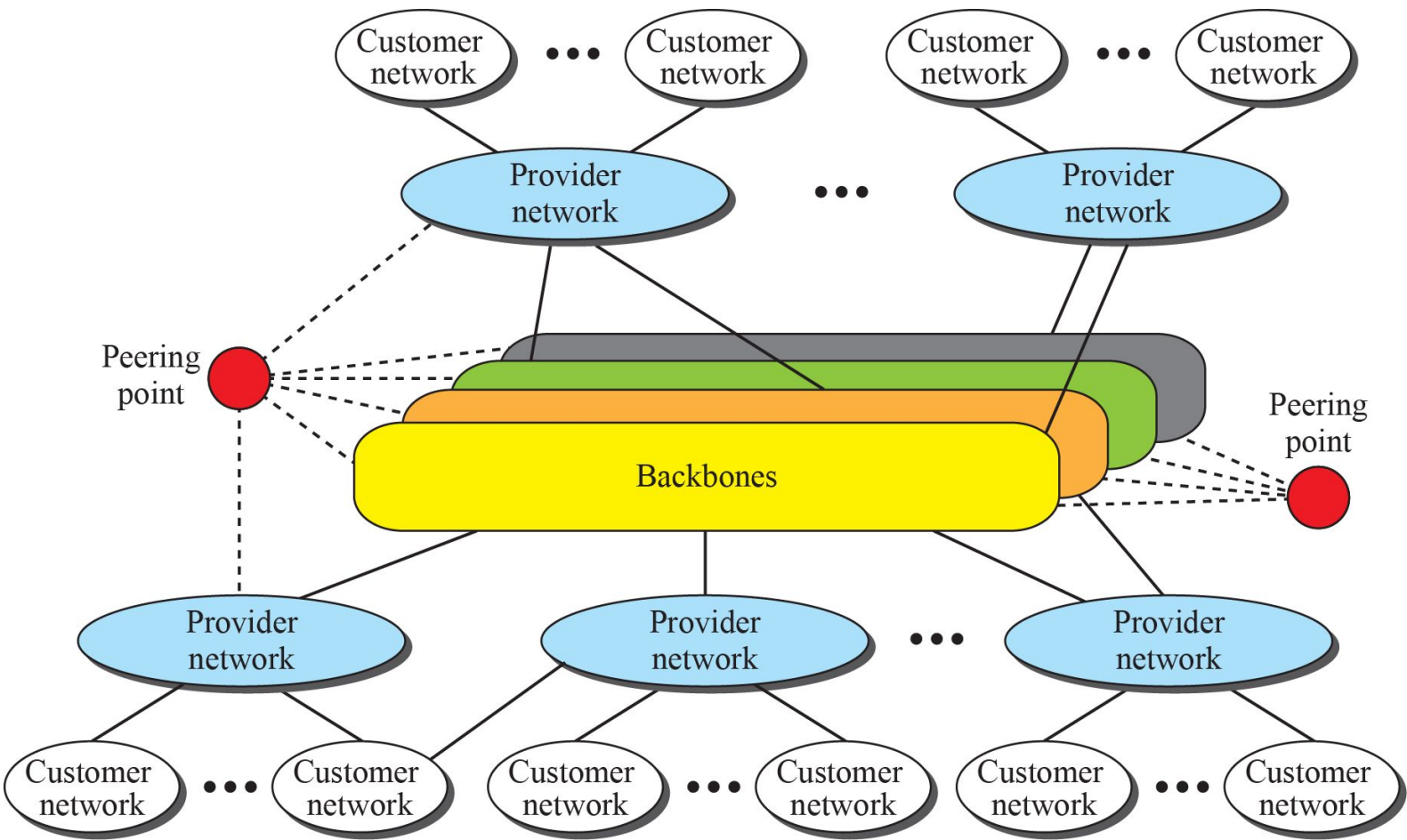


Figure 4.70: Hop counts in RIP

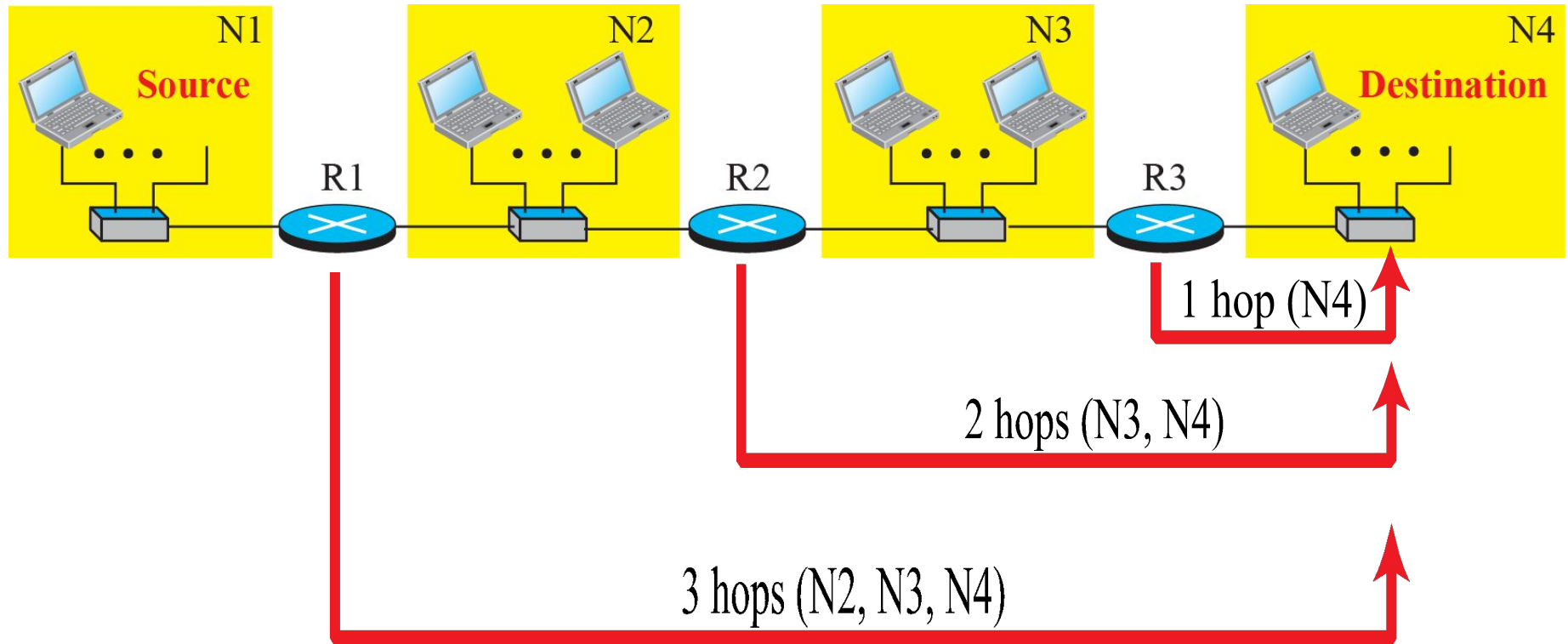


Figure 4.71: Forwarding tables

Forwarding table for R1

Destination network	Next router	Cost in hops
N1	—	1
N2	—	1
N3	R2	2
N4	R2	3

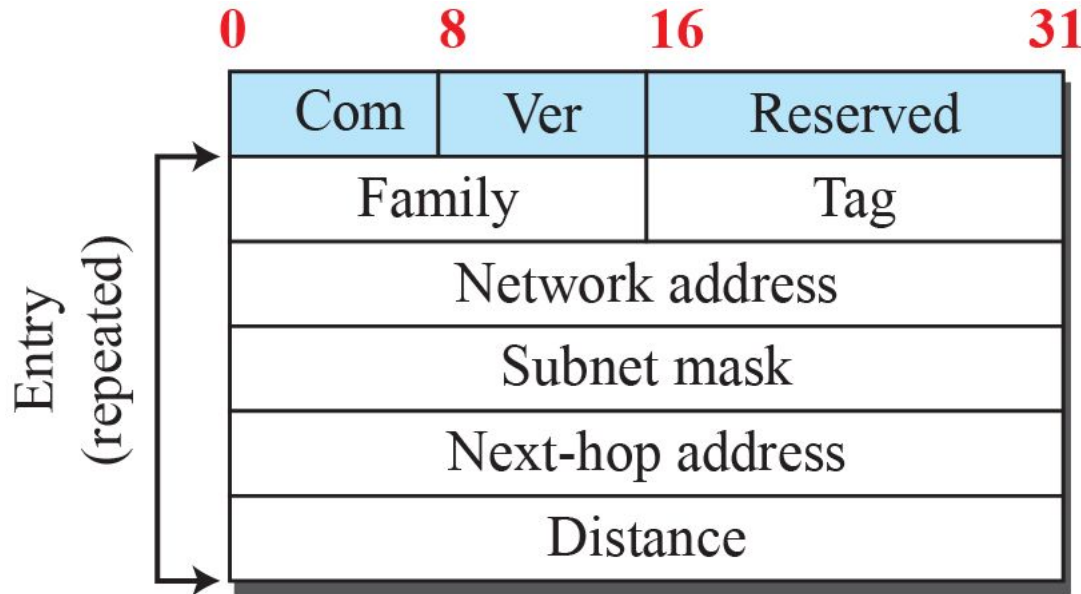
Forwarding table for R3

Destination network	Next router	Cost in hops
N1	R2	3
N2	R2	2
N3	—	1
N4	—	1

Forwarding table for R2

Destination network	Next router	Cost in hops
N1	R1	2
N2	—	1
N3	—	1
N4	R3	2

Figure 4.72: RIP message format



Fields

Com: Command, request (1), response (2)

Ver: Version, current version is 2

Family: Family of protocol, for TCP/IP value is 2

Tag: Information about autonomous system

Network address: Destination address

Subnet mask: Prefix length

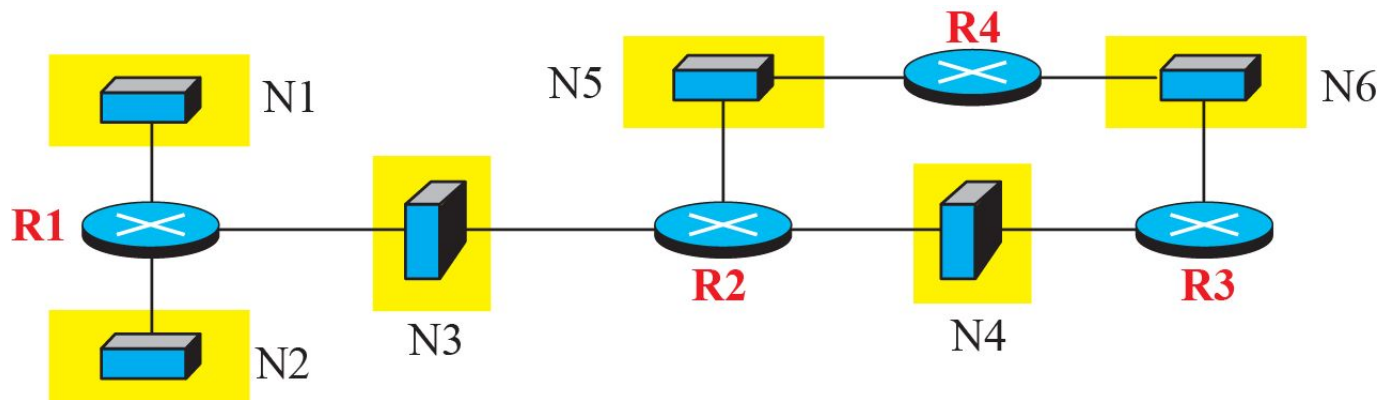
Next-hop address: Address length

Distance: Number of hops to the destination

Example 4.15

Figure 4.73 shows a more realistic example of the operation of RIP in an autonomous system. First, the figure shows all forwarding tables after all routers have been booted. Then we show changes in some tables when some update messages have been exchanged. Finally, we show the stabilized forwarding tables when there is no more change.

Figure 4.73: Example of an autonomous system using RIP (Part I)



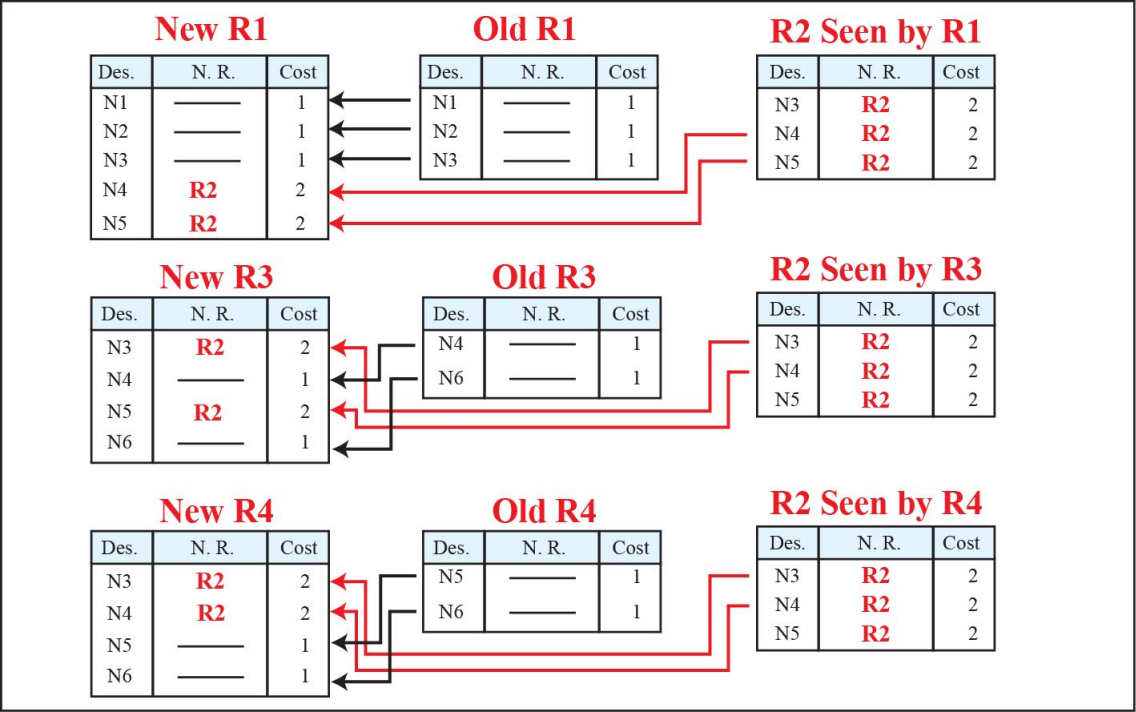
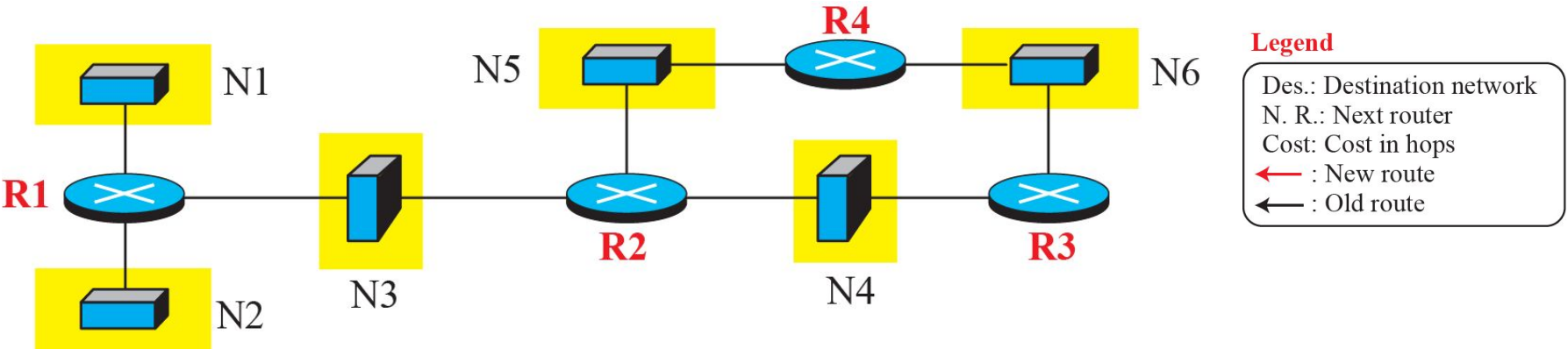
Legend

Des.: Destination network
N. R.: Next router
Cost: Cost in hops

R1			R2			R3			R4		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N1	_____	1	N3	_____	1	N4	_____	1	N5	_____	1
N2	_____	1	N4	_____	1	N6	_____	1	N6	_____	1
N3	_____	1	N5	_____	1						

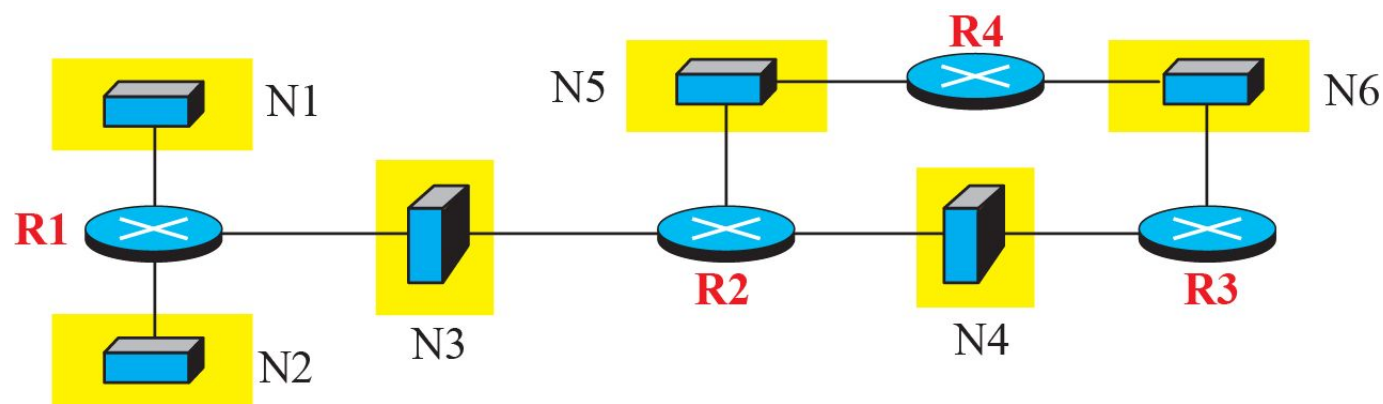
Forwarding tables after all routers booted

Figure 4.73: Example of an autonomous system using RIP (Part II)



Changes in the forwarding tables of R1, R3, and R4 after they receive a copy of R2's table

Figure 4.73: Example of an autonomous system using RIP (Part III)



Legend

Des.: Destination network

N. R.: Next router

Cost: Cost in hops

Forwarding tables for all routers
after they have been stablized

Final R1			Final R2			Final R3			Final R4		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N1	_____	1	N1	R1	2	N1	R2	3	N1	R2	3
N2	_____	1	N2	R1	2	N2	R2	3	N2	R2	3
N3	_____	1	N3	_____	1	N3	R2	2	N3	R2	2
N4	R2	2	N4	_____	1	N4	_____	1	N4	R2	2
N5	R2	2	N5	_____	1	N5	R2	2	N5	_____	1
N6	R2	3	N6	R3	2	N6	_____	1	N6	_____	1

Figure 4.74: Metric in OSPF

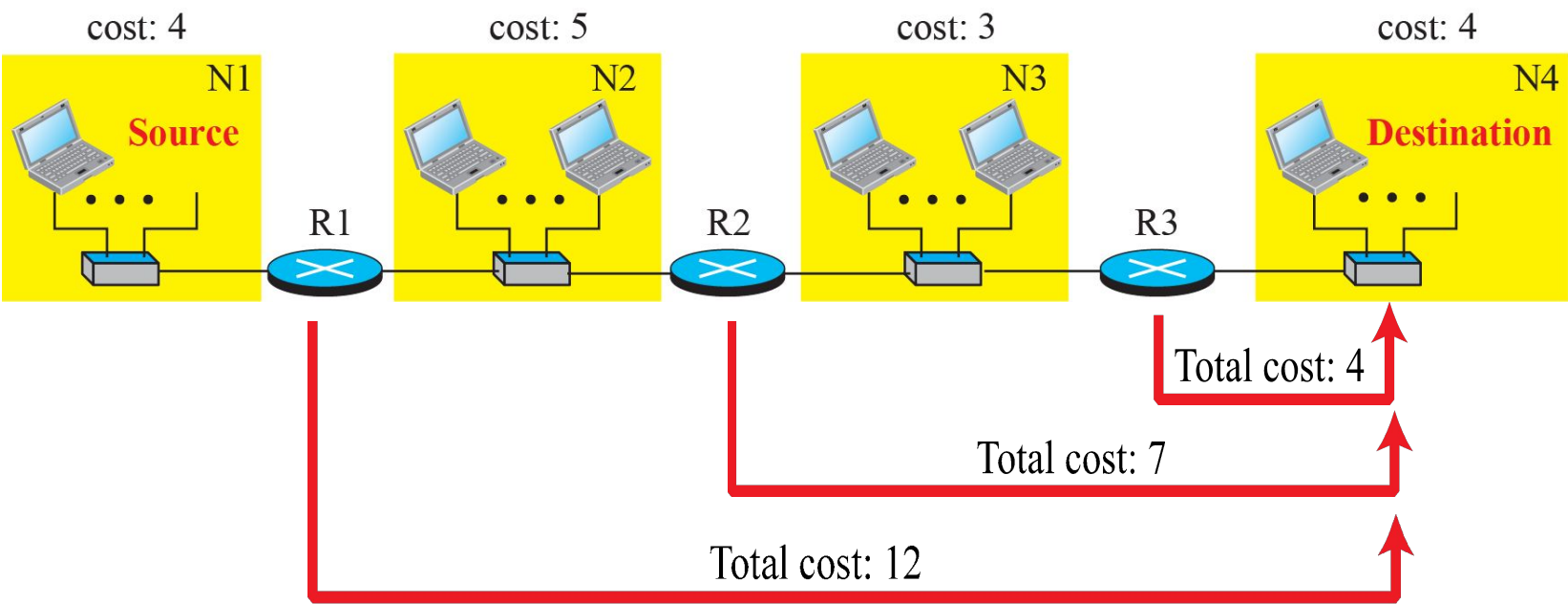
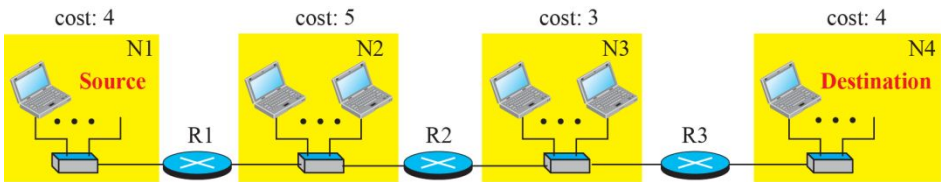


Figure 4.75: Forwarding tables in OSPF

Forwarding table for R1

Destination network	Next router	Cost
N1	—	
N2	—	
N3	R2	
N4	R2	12



The internet from previous figure

Forwarding table for R2

Destination network	Next router	Cost
N1	R1	9
N2	—	5
N3	—	3
N4	R3	7

Forwarding table for R3

Destination network	Next router	Cost
N1	R2	12
N2	R2	8
N3	—	3
N4	—	4

Figure 4.76: *Areas in an autonomous system*

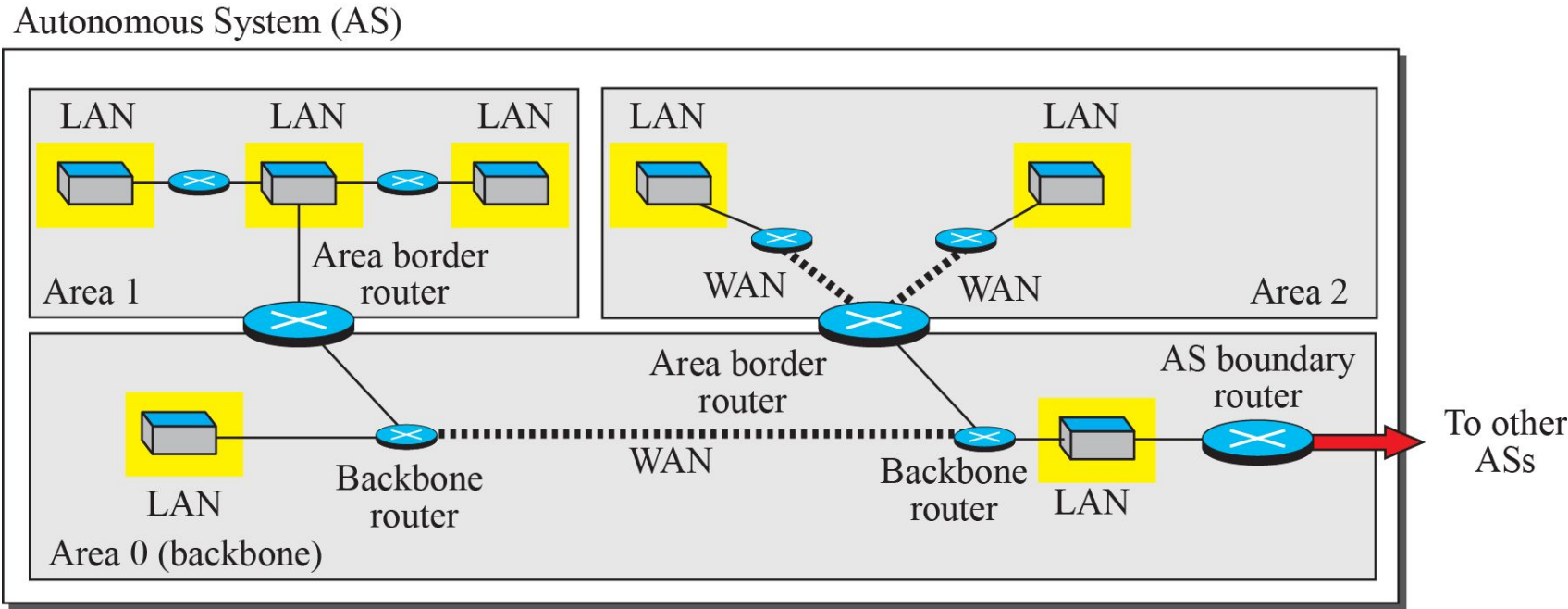
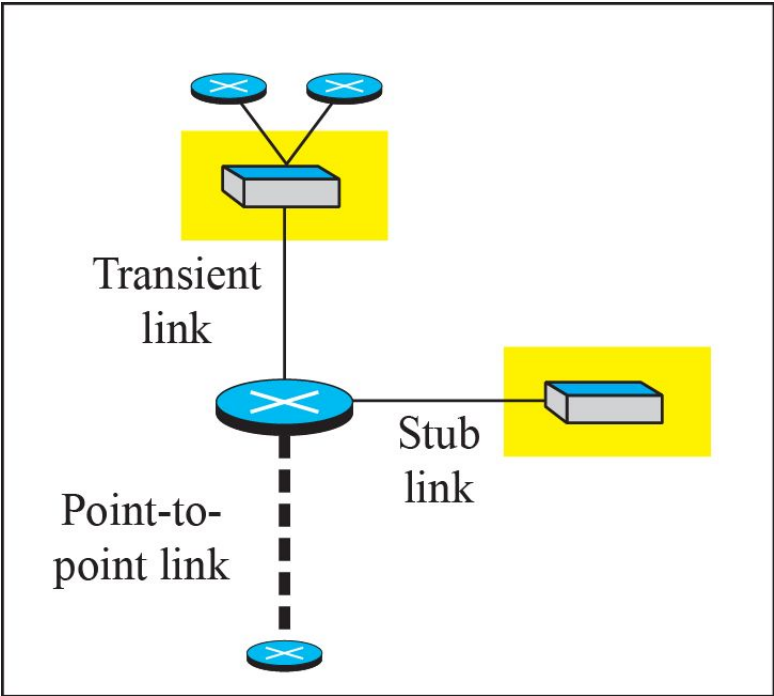
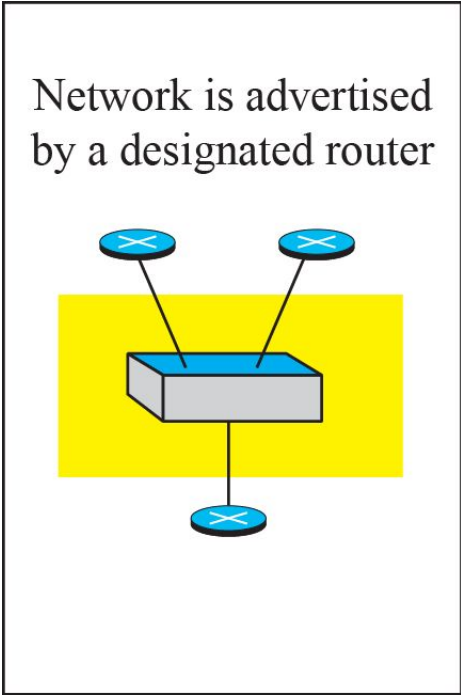


Figure 4.77: Five different LSPs (Part I)

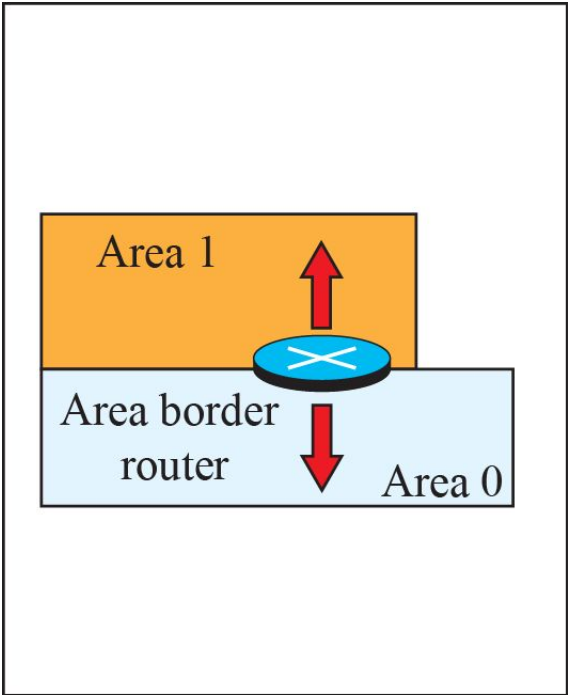


a. Router link

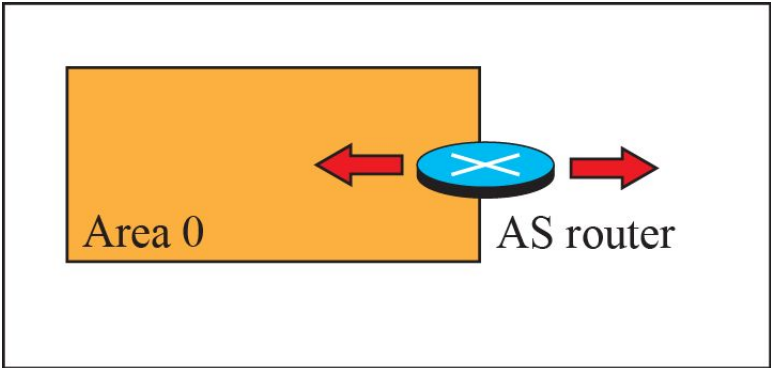


b. Network link

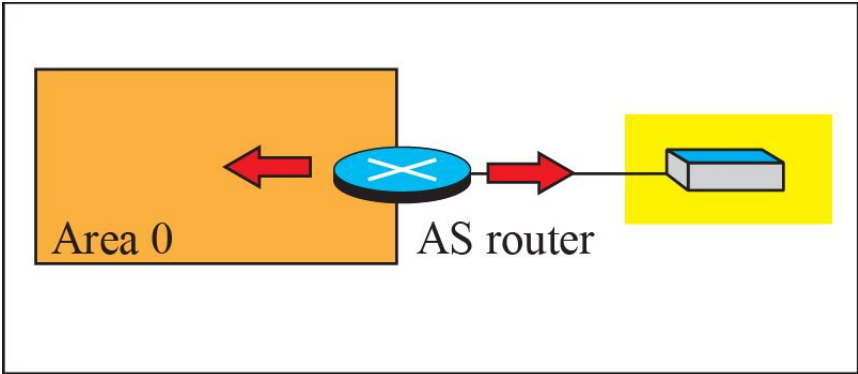
Figure 4.77: Five different LSPs (Part II)



c. Summary link to network



d. Summary link to AS



e. External link

Figure 4.78: OSPF message formats (Part I)

0	8	16	31
Version	Type	Message length	
Source router IP address			
Area Identification			
Checksum		Authentication type	
Authentication			

OSPF common header

Legend

E, T, B, I, M, MS: flags used by OSPF
Priority: used to define the designated router
Rep.: Repeated as required

Attention



Rep. [

OSPF common header (Type: 1)			
Network mask			
Hello interval		E	T
Dead interval			
Designated router IP address			
Backup designated router IP address			
Neighbor IP address			

OSPF common header (Type: 2)			
		E	B
		I	M
			S
Message sequence number			
Link-state general header			

Database description

Figure 4.78: OSPF message formats (Part II)

LS age		E	T	LS type
LS ID				
Advertising router				
LS sequence number				
LS checksum	Length			

Link-state general header

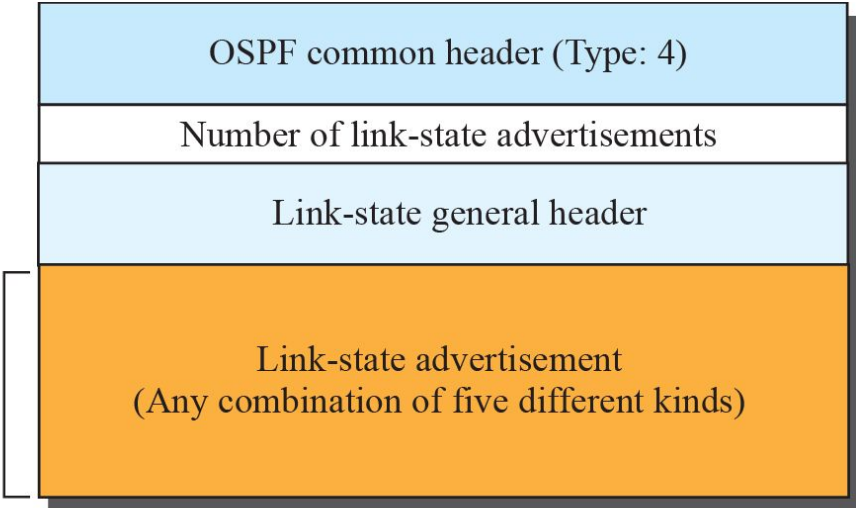
Legend

E, T, B, I, M, MS: flags used by OSPF
Priority: used to define the designated router
Rep.: Repeated as required

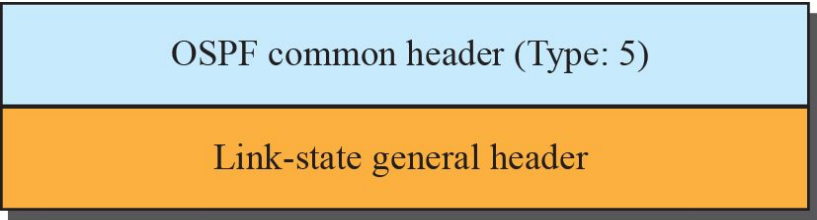
Attention



Rep.



Link-state update



Link-state acknowledgment

Figure 4.79: *A sample internet with four ASs*

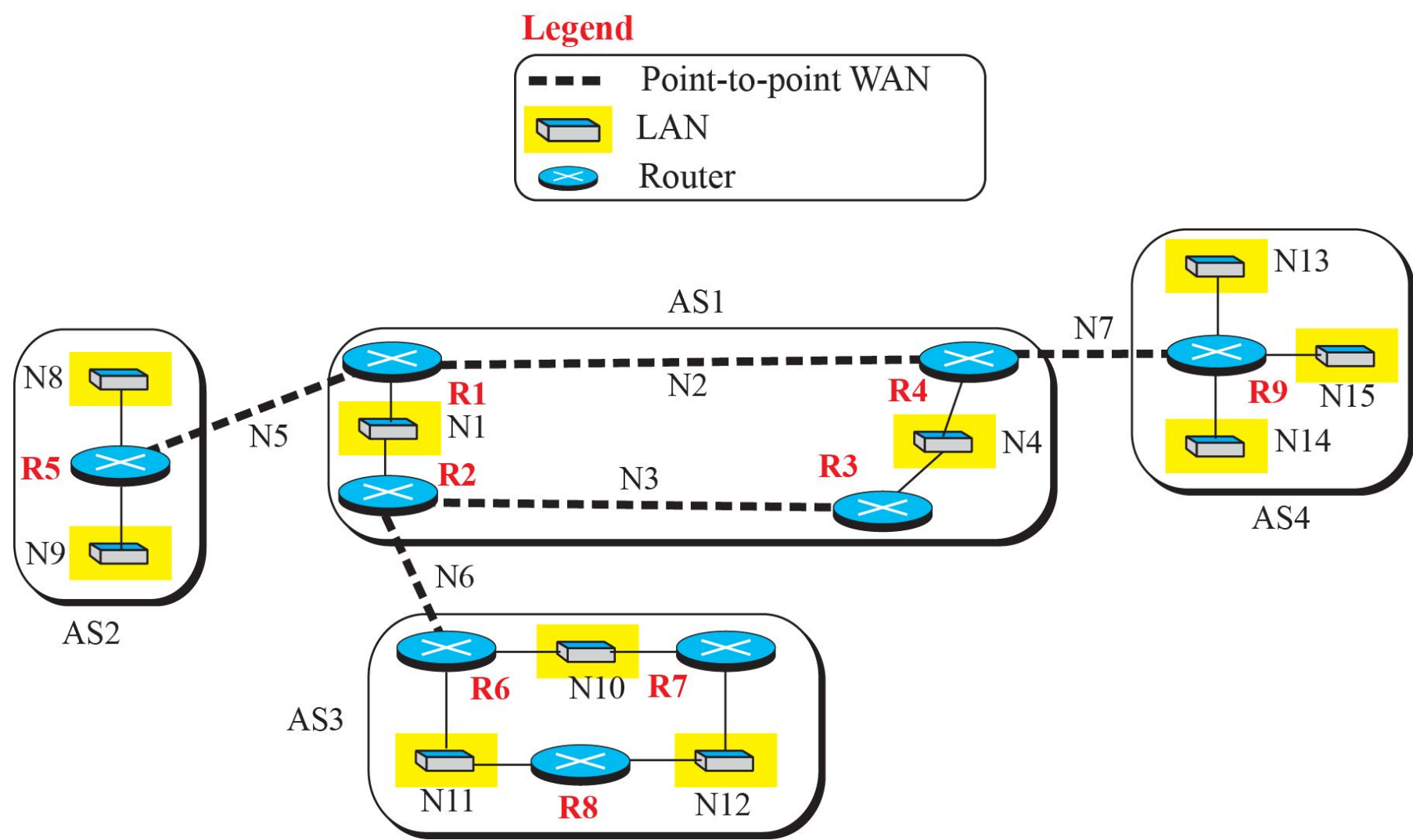


Figure 4.80: eBGP operation

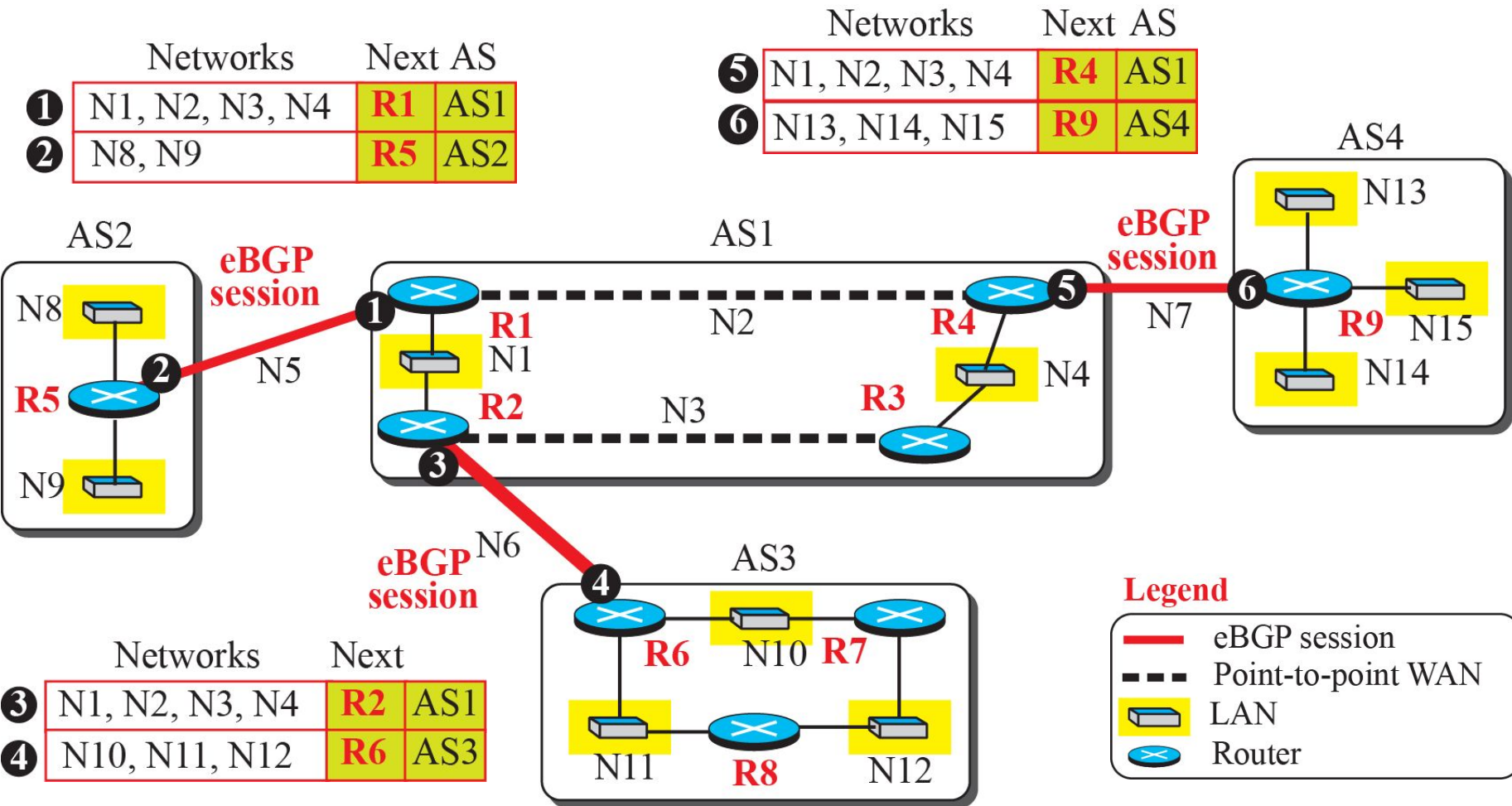


Figure 4.81: *Combination of eBGP and iBGP sessions in our internet*

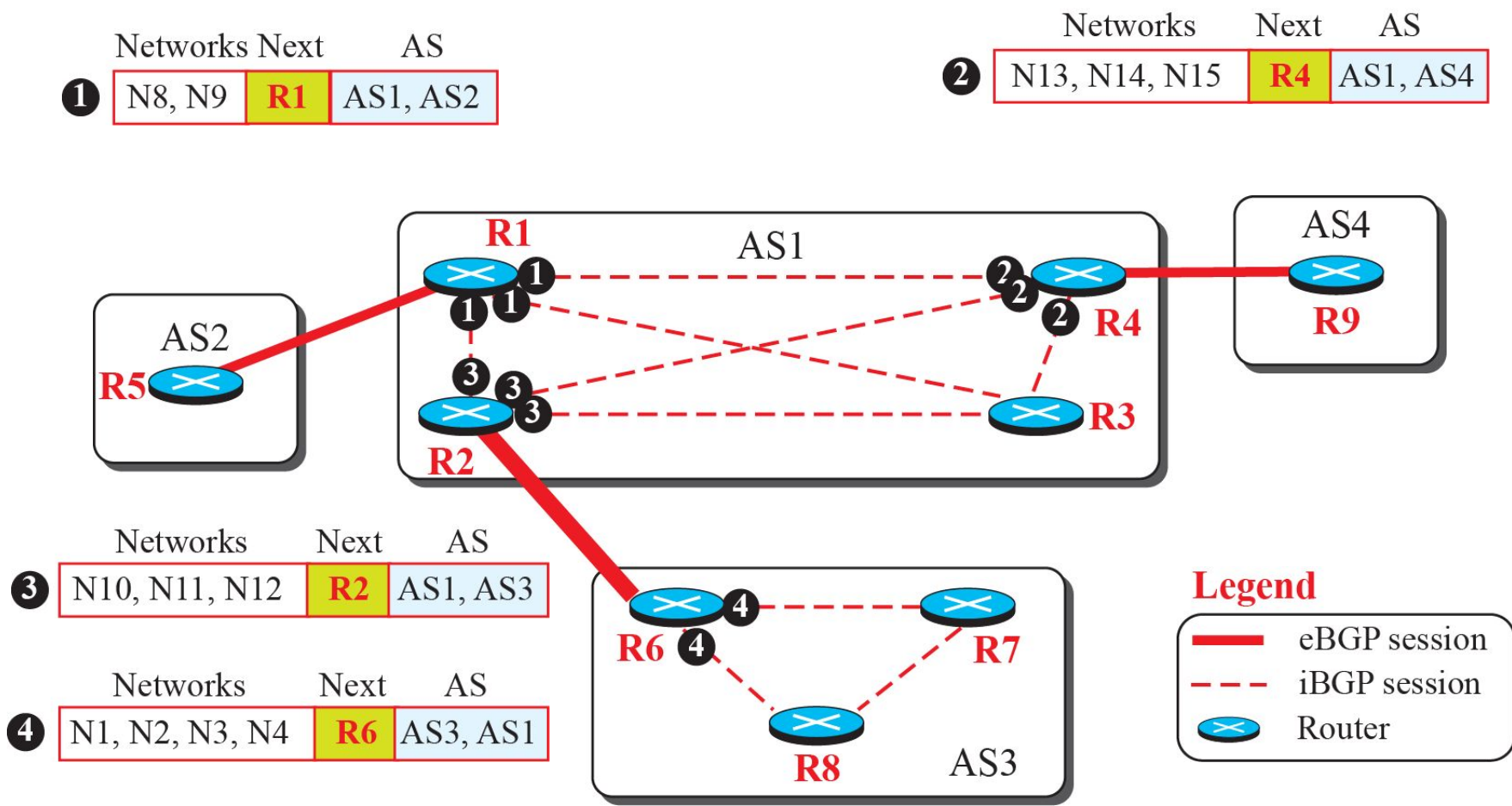
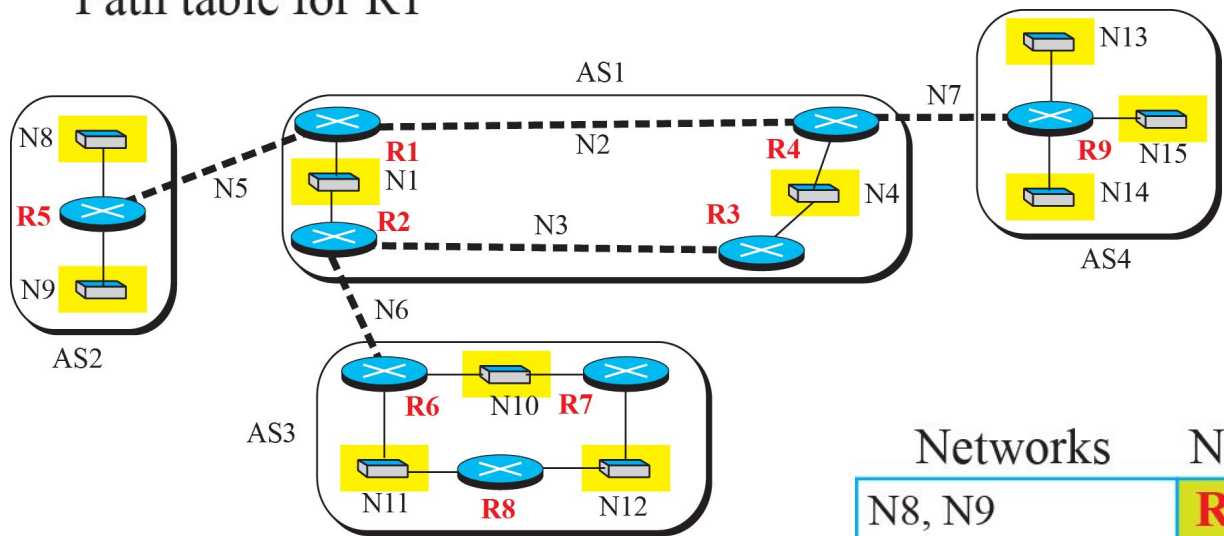


Figure 4.82: Finalized BGP path tables (Part I)

Networks	Next	Path
N8, N9	R5	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R1



Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R6	AS1, AS3
N13, N14, N15	R1	AS1, AS4

Path table for R2

Networks	Next	Path
N8, N9	R2	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R3

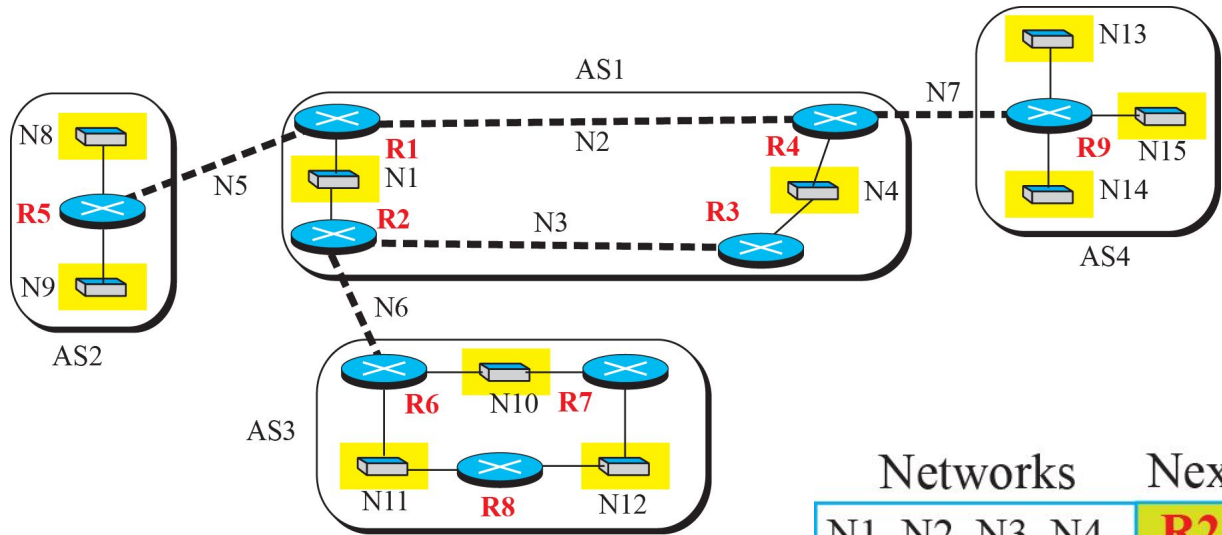
Figure 4.82: Finalized BGP path tables (Part II)

Networks	Next	Path
N1, N2, N3, N4	R1	AS2, AS1
N10, N11, N12	R1	AS2, AS1, AS3
N13, N14, N15	R1	AS2, AS1, AS4

Path table for R5

Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R1	AS1, AS3
N13, N14, N15	R9	AS1, AS4

Path table for R4



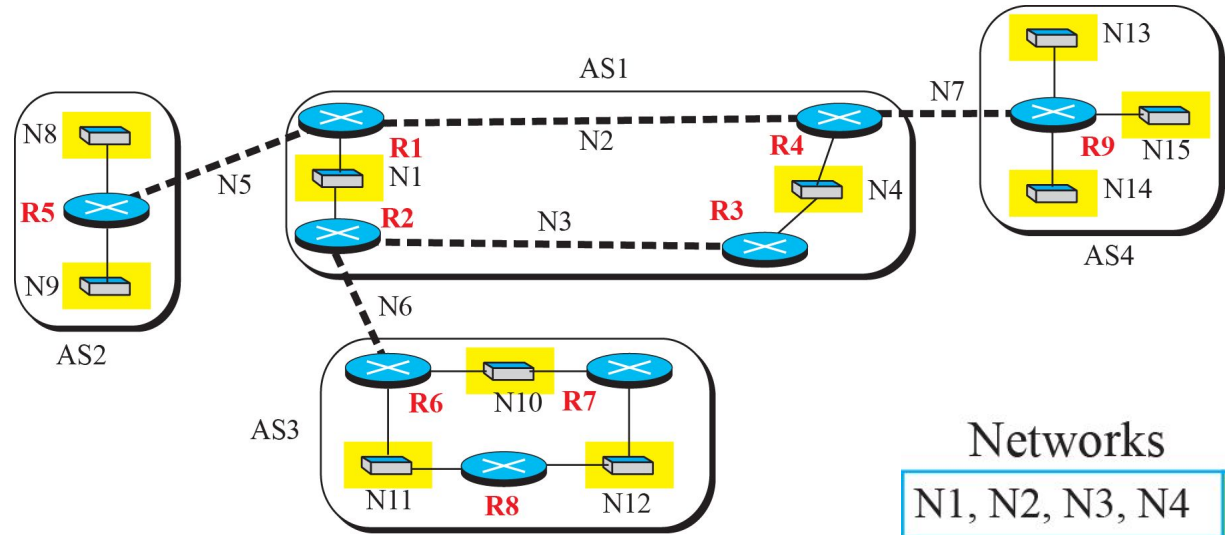
Networks	Next	Path
N1, N2, N3, N4	R2	AS3, AS1
N8, N9	R2	AS3, AS1, AS2
N13, N14, N15	R2	AS3, AS1, AS4

Path table for R6

Figure 4.82: Finalized BGP path tables (Part III)

Networks	Next	Path
N1, N2, N3, N4	R4	AS4, AS1
N8, N9	R4	AS4, AS1, AS2
N10, N11, N12	R4	AS4, AS1, AS3

Path table for R9



Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R7

Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R8

Figure 4.83: Forwarding tables after injection from BGP (Part I)

Des.	Next	Cost
N1	—	1
N4	R4	2
N8	R5	1
N9	R5	1
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R1

Des.	Next	Cost
N1	—	1
N4	R3	2
N8	R1	2
N9	R1	2
N10	R6	1
N11	R6	1
N12	R6	1
N13	R3	3
N14	R3	3
N15	R3	3

Table for R2

Des.	Next	Cost
N1	R2	2
N4	—	1
N8	R2	3
N9	R2	3
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R3

Des.	Next	Cost
N1	R1	2
N4	—	1
N8	R1	2
N9	R1	2
N10	R3	3
N11	R3	3
N12	R3	3
N13	R9	1
N14	R9	1
N15	R9	1

Table for R4

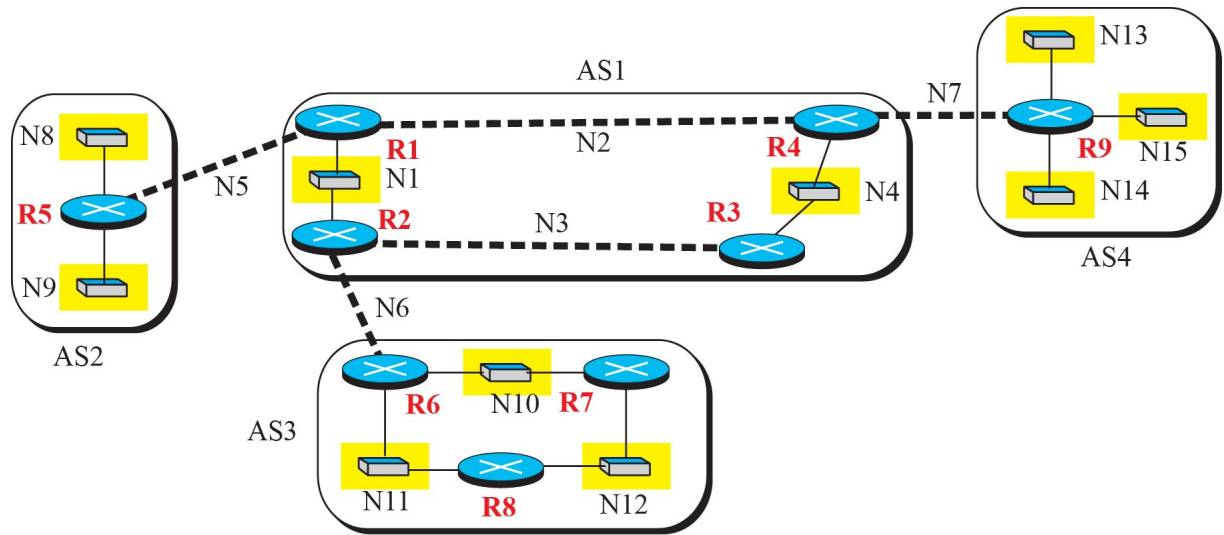


Figure 4.83: Forwarding tables after injection from BGP (Part II)

Des.	Next	Cost
N8	—	1
N9	—	1
0	R1	1

Table for R5

Des.	Next	Cost
N13	—	1
N14	—	1
N15	—	1
0	R4	1

Table for R9

Des.	Next	Cost
N10	—	1
N11	—	1
N12	R7	2
0	R2	1

Table for R6

Des.	Next	Cost
N10	R6	2
N11	—	1
N12	—	1
0	R6	2

Table for R8

Des.	Next	Cost
N10	—	1
N11	R6	2
N12	—	1
0	R6	2

Table for R7

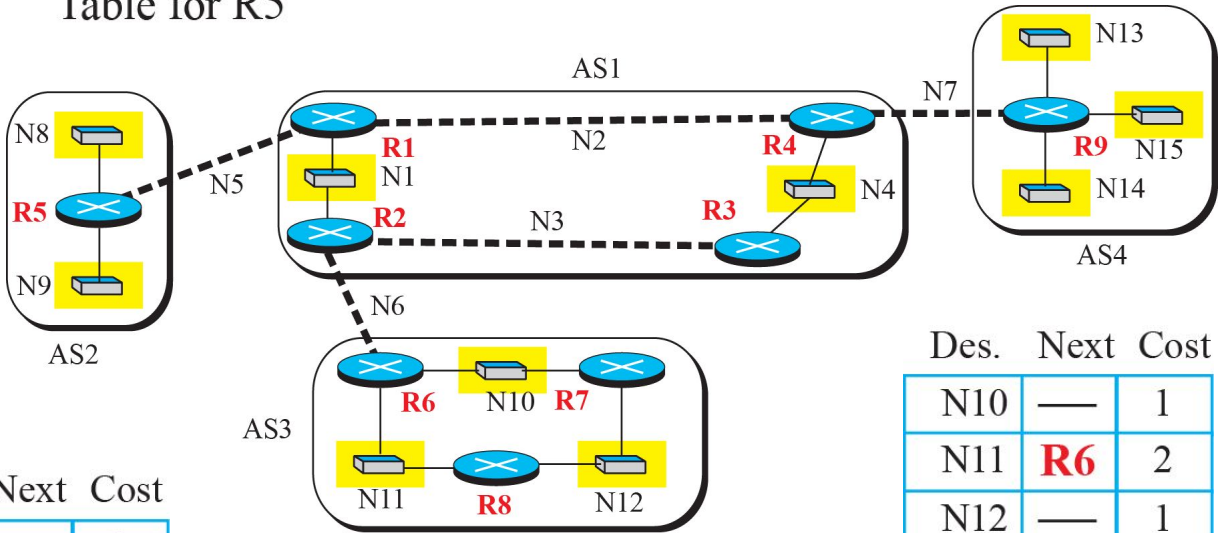


Figure 4.84: *Format of path attribute*

- O:** Optional bit (set if attribute is optional)
- P:** Partial bit (set if an optional attribute is lost in transit)
- T:** Transitive bit (set if attribute is transitive)
- E:** Extended bit (set if attribute length is two bytes)

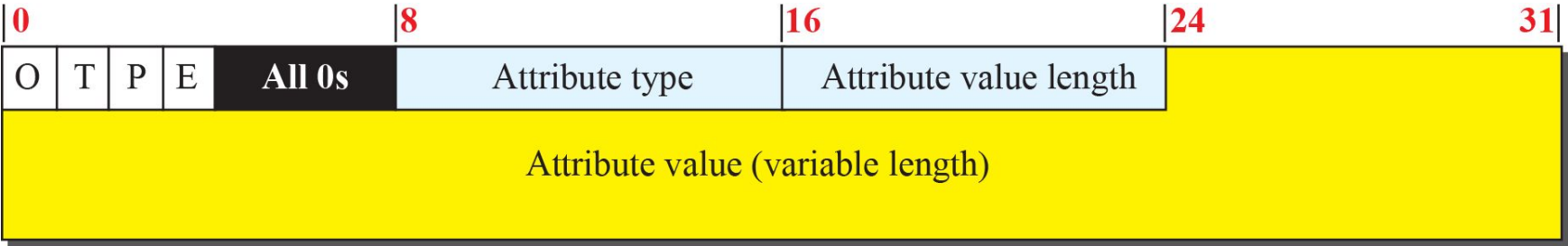


Figure 4.85: *Flow diagram for route selection*

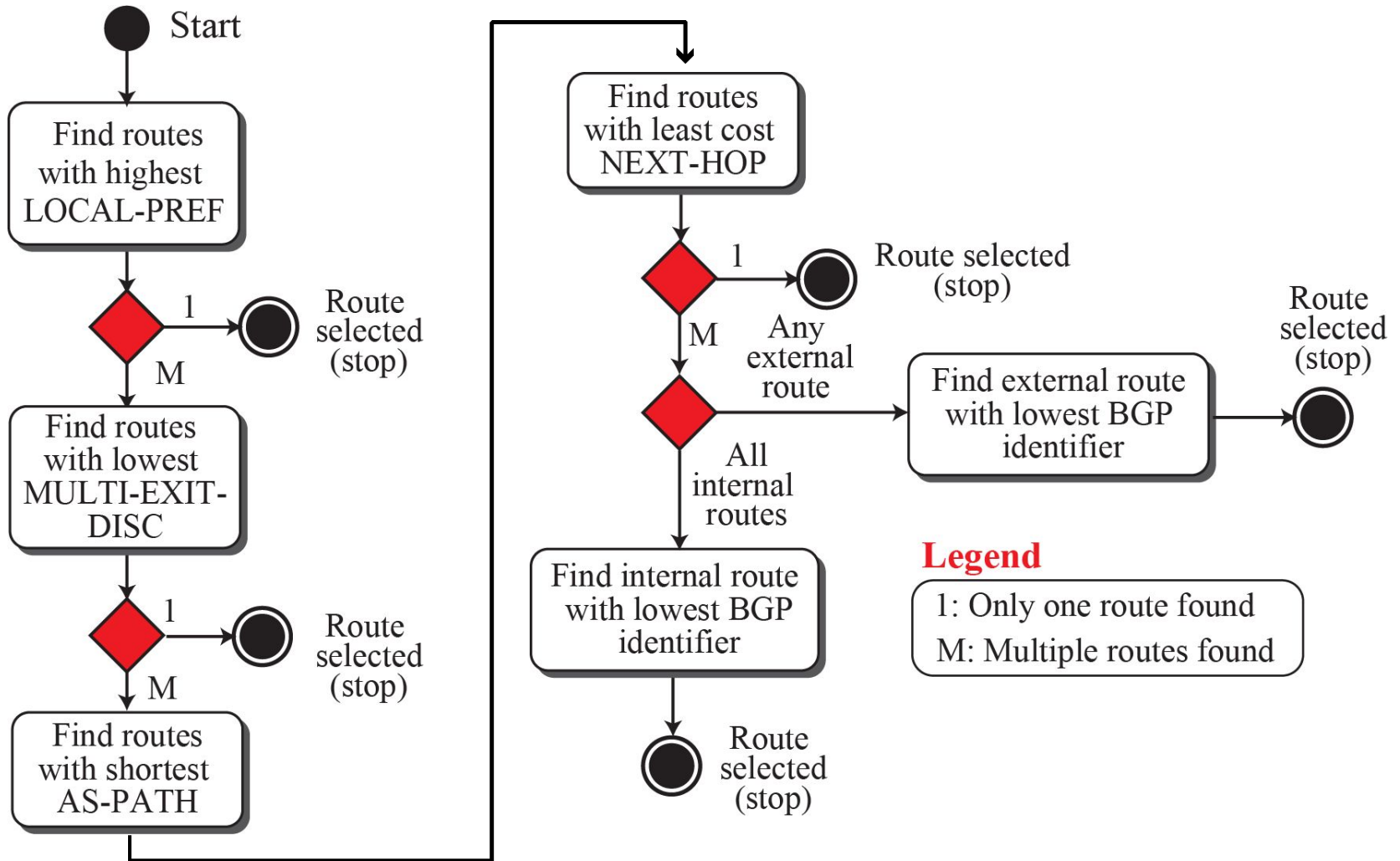
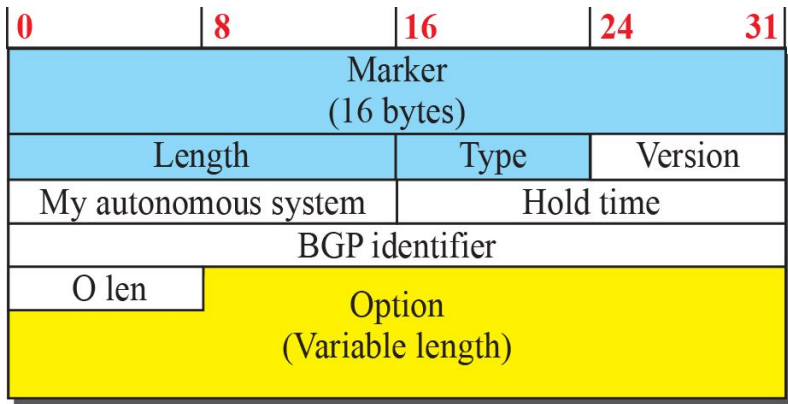
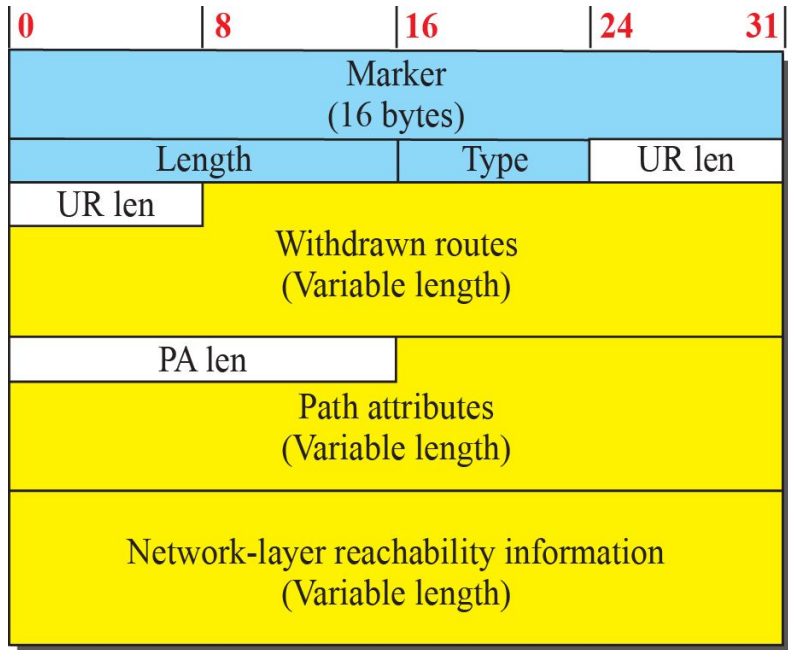


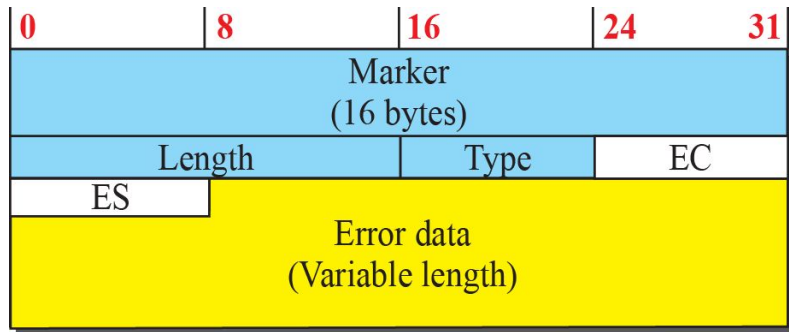
Figure 4.86: BGP messages



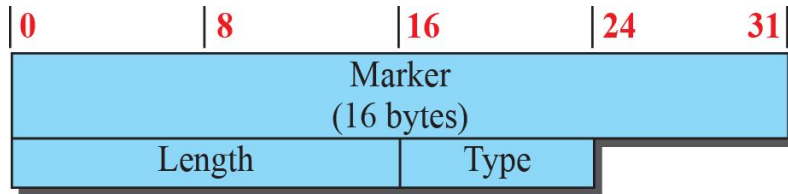
Open message (type 1)



Update message (type 2)



Notification message (type 3)



Keepalive message (type 4)

Fields in common header

- Marker: Reserved for authentication
- Length: Length of total message in bytes
- Type: Type of message (1 to 4)

Abbreviations

- O len: Option length
- EC: Error code
- ES: Error subcode
- UR len: Unfeasible route length
- PA len: Path attribute length

4-4 MULTICAST ROUTING

Communication in the Internet today is not only unicasting; multicasting communication is growing fast. In this section, we first discuss the general ideas behind unicasting, multicasting, and broadcasting. We then talk about some basic issues in multicast routing. Finally, we discuss multicasting routing protocols in the Internet.



4.4.1 Introduction

From the previous sections, we have learned that forwarding a datagram by a router is normally based on the prefix of the destination address in the datagram, which defines the network to which the destination host is connected. Understanding the above forwarding principle, we can now define unicasting, multicasting, and broadcasting. Let us clarify these terms as they relate to the Internet.



4.4.1 (continued)

□ *Unicasting*

□ *Multicasting*

- ◆ *Multicasting versus Multiple Unicasting*
- ◆ *Emulation of Multicasting with Unicasting*
- ◆ *Multicast Applications*

□ *Broadcasting*

Figure 4.87: Unicast

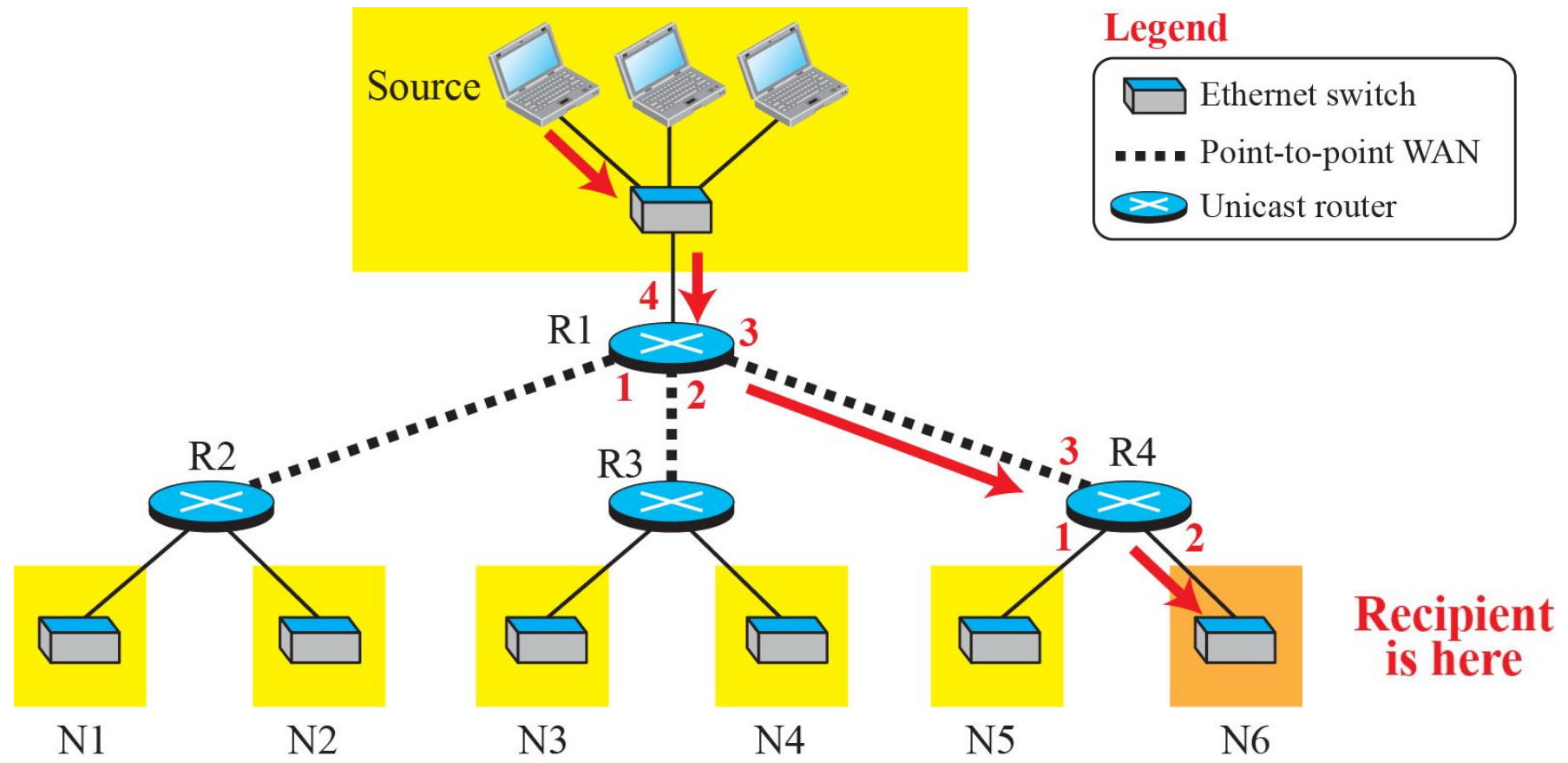


Figure 4.88: Multicasting

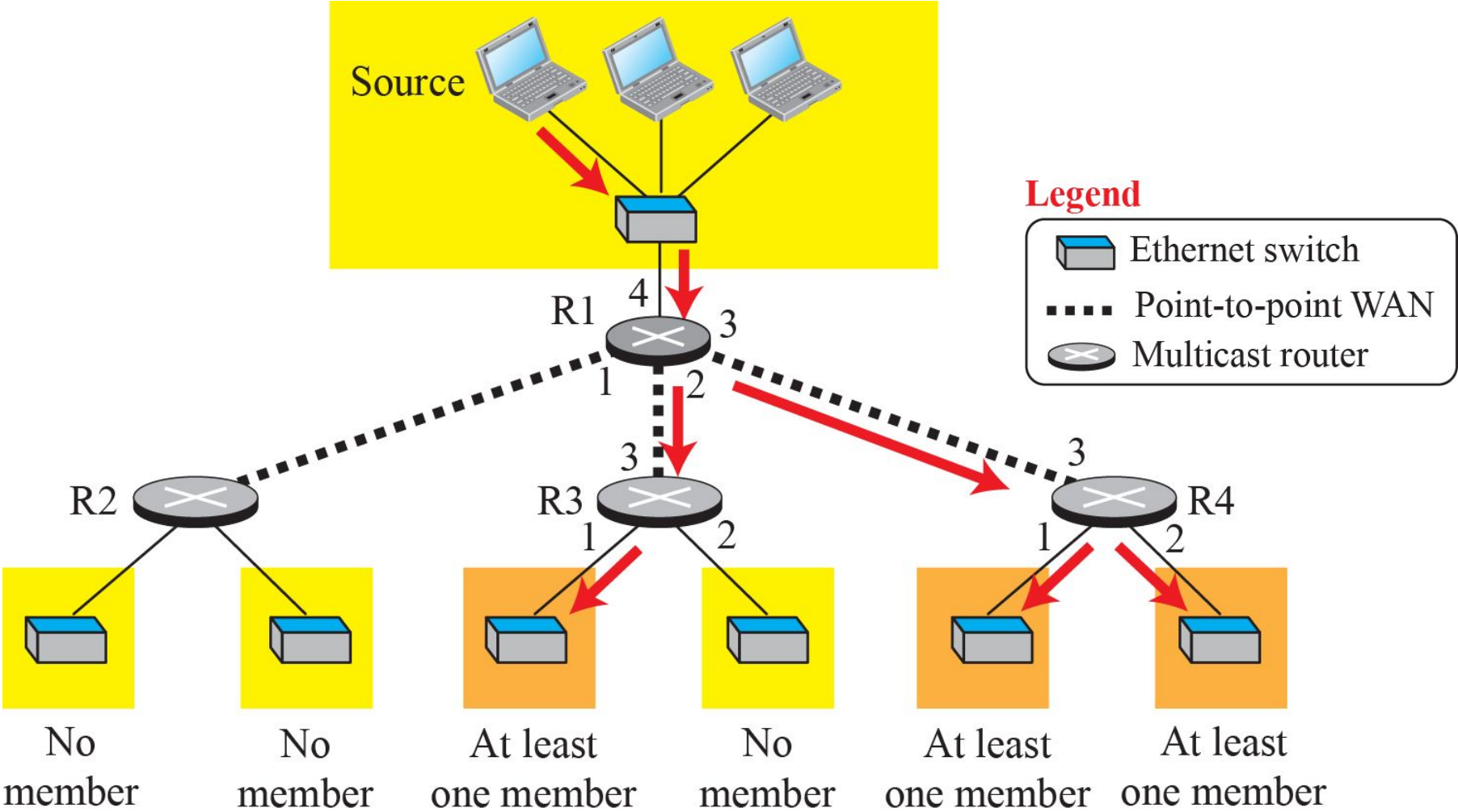
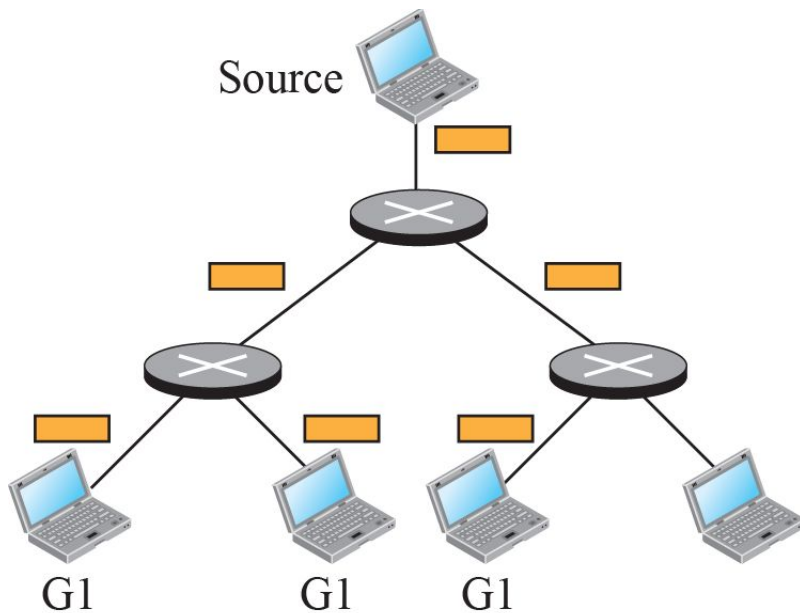


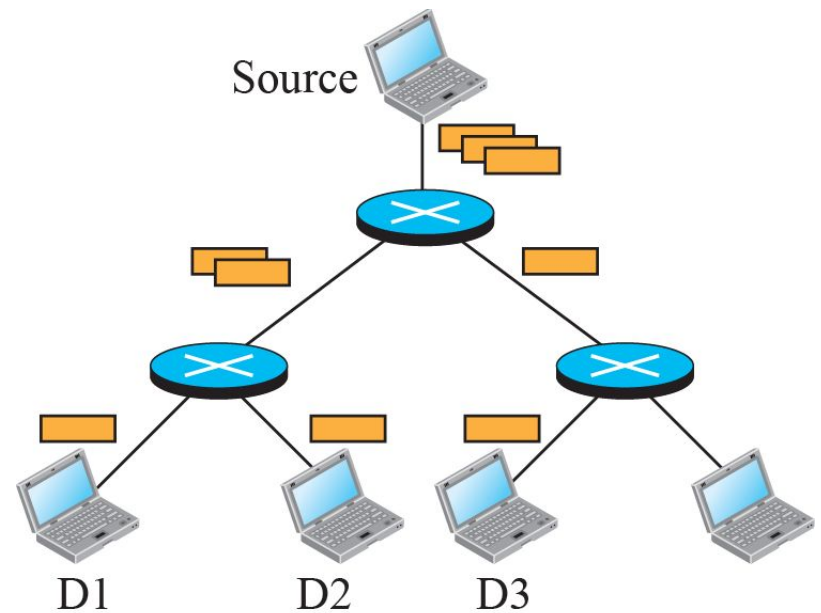
Figure 4.89: Multicasting versus multiple unicasting

Legend

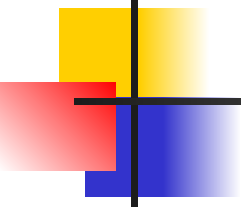
-  Multicast router
-  Unicast router
- D_i Unicast destination
- G_i Group member



a. Multicasting



b. Multiple unicasting



4.4.2 Multicasting Basics

Before discussing multicast routing protocols in the Internet, we need to discuss some multicasting basics: multicast addressing, collecting information about multicast groups, and multicast optimal trees.

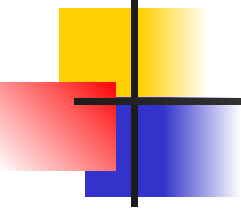
□ *Multicast Addresses*

◆ *Multicast Addresses in IPv4*

- *Local Network Control Block*
- *Internetwork Control Block.*
- *Source-Specific Multicast (SSM) Block.*
- *GLOP Block.*
- *Administratively Scoped Block.*

◆ *Selecting Multicast Address*

- *Limited Group*
- *Larger Group*



4.4.2 (continued)

Collecting Information about Groups

- ❖ *Internet Group Management Protocol (IGMP)*

Multicast Forwarding

Two Approaches to Multicasting

- ❖ *Source-Based Tree Approach*

- ❖ *Group-Shared Tree Approach*

Figure 4.90: Needs for multicast addresses

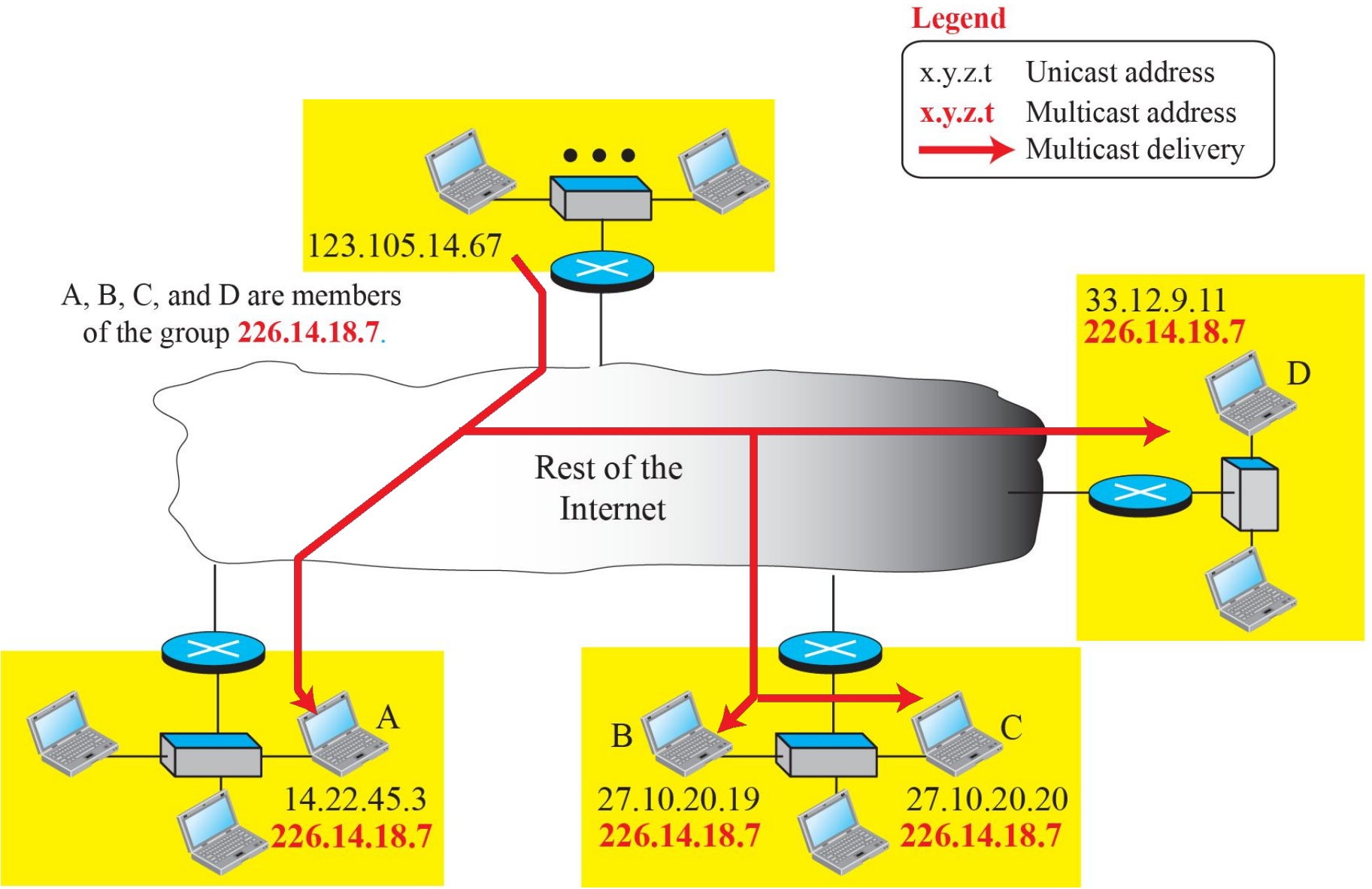


Figure 4.91: *A multicast address in binary*

Block: 224.0.0.0/4

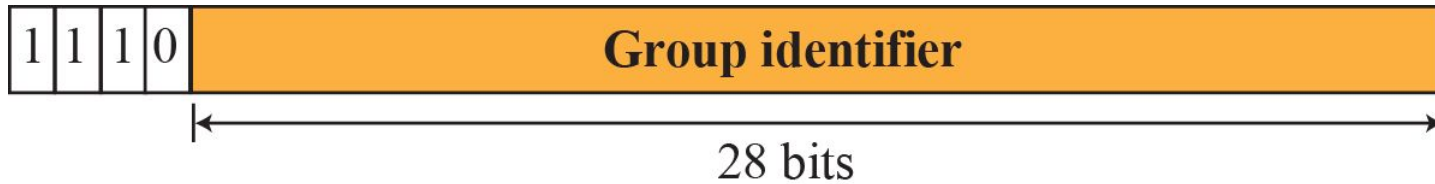


Figure 4.92: *Unicast versus multicast advertisement*

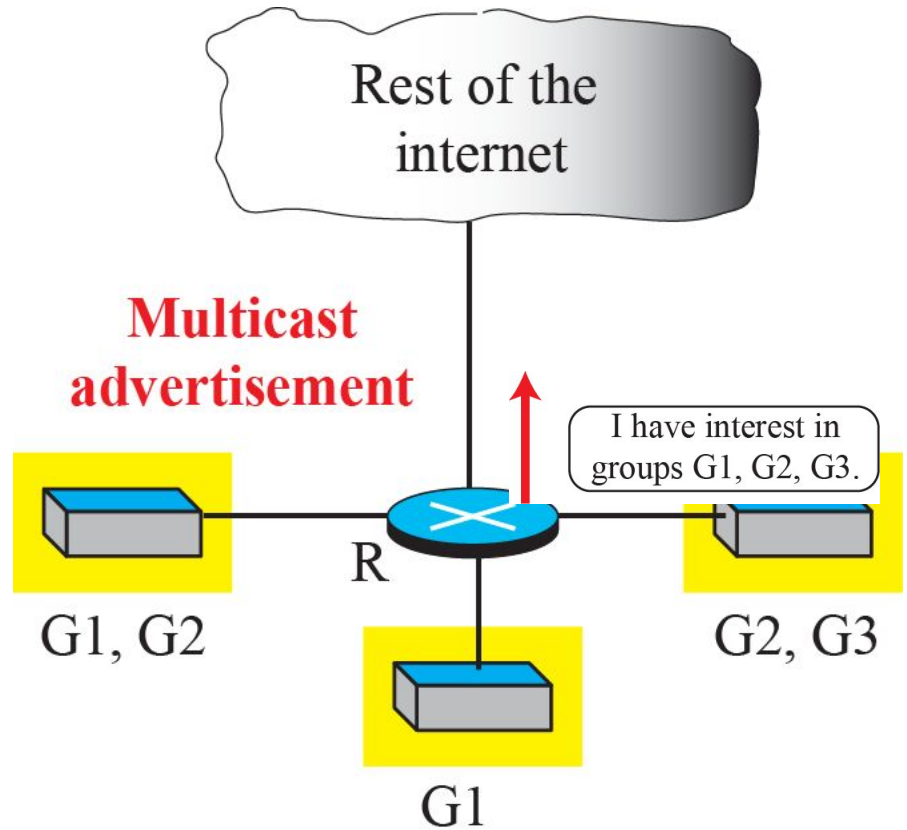
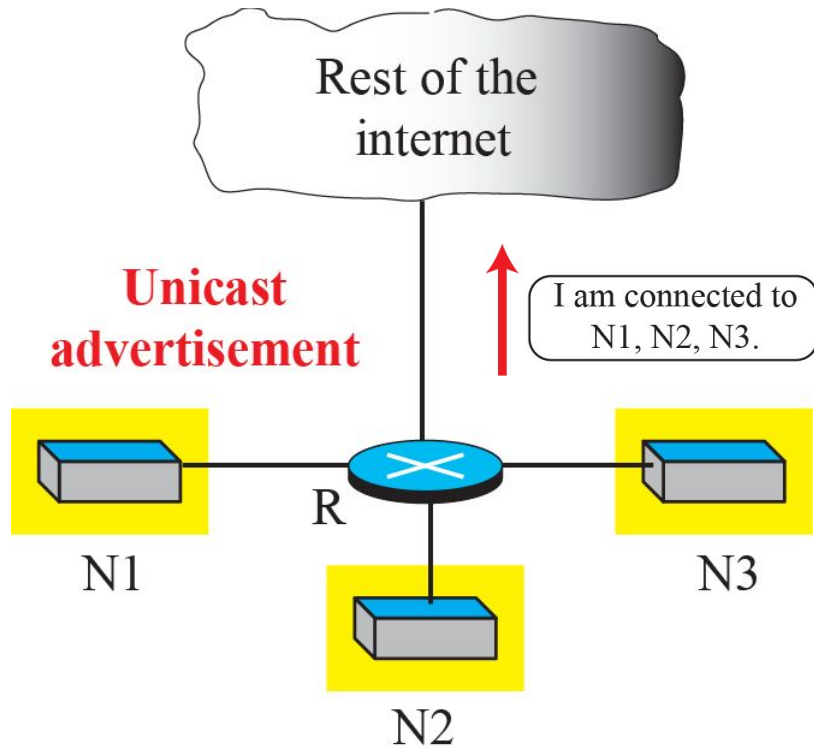


Figure 4.93: *IGMP operation*

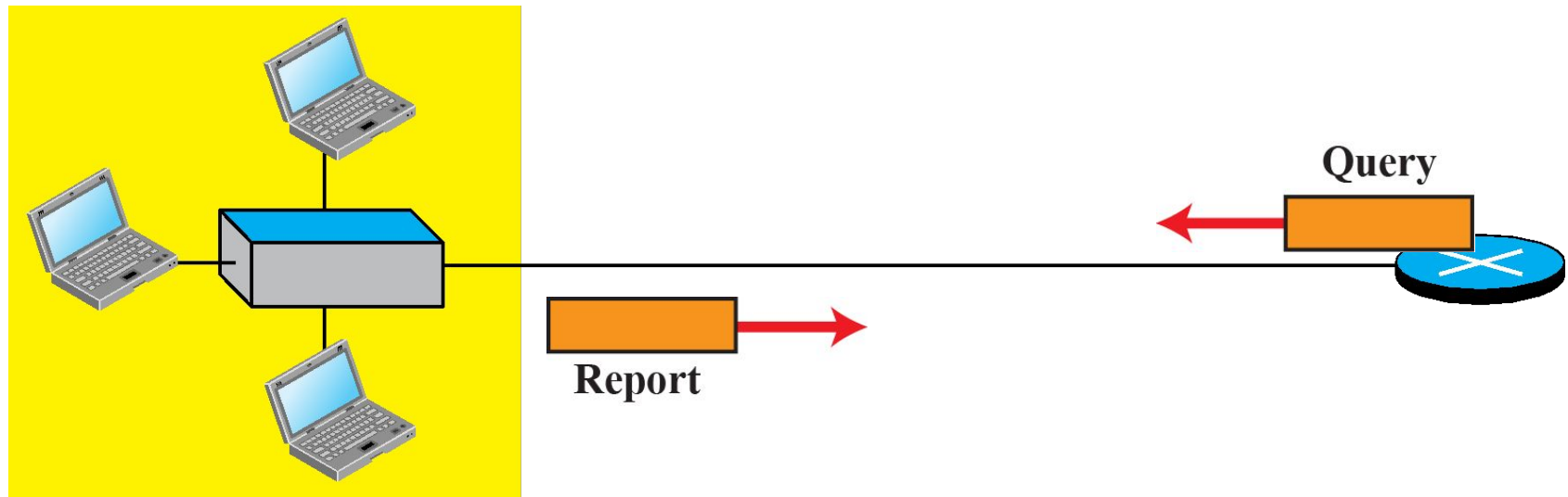
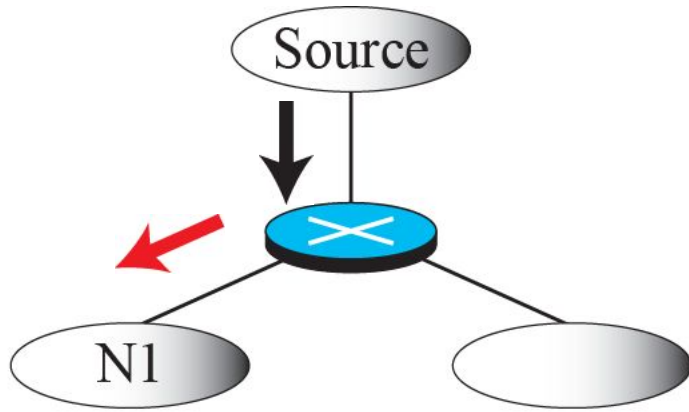
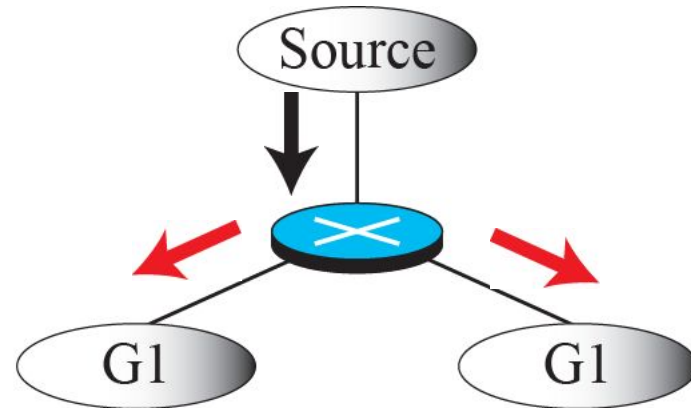


Figure 4.94: *Destination in unicasting and multicasting*

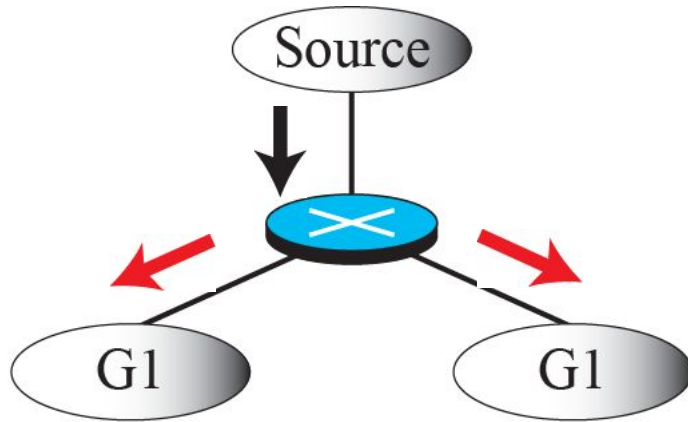


a. Destination in unicasting is one

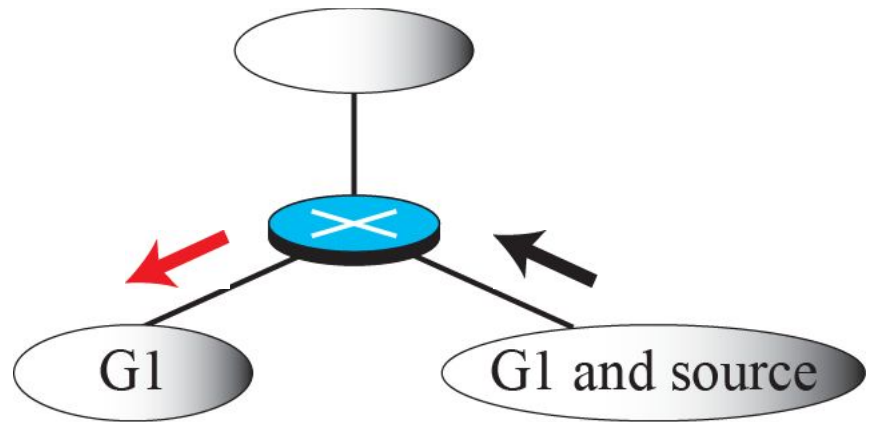


b. Destination in multicasting is more than one

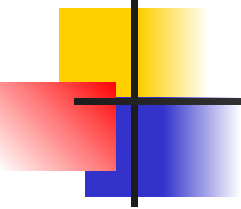
Figure 4.95: *Forwarding depends on the destination and the source*



a. Packet sent out of two interfaces



b. Packet sent out of one interface



4.4.3 Intradomain Routing Protocols

During the last few decades, several intradomain multicast routing protocols have emerged. In this section, we discuss three of these protocols. Two are extensions of unicast routing protocols (RIP and OSPF), using the source-based tree approach; the third is an independent protocol which is becoming more and more popular. It can be used in two modes, employing either the source-based tree approach or the shared-group tree approach.



4.4.3 (continued)

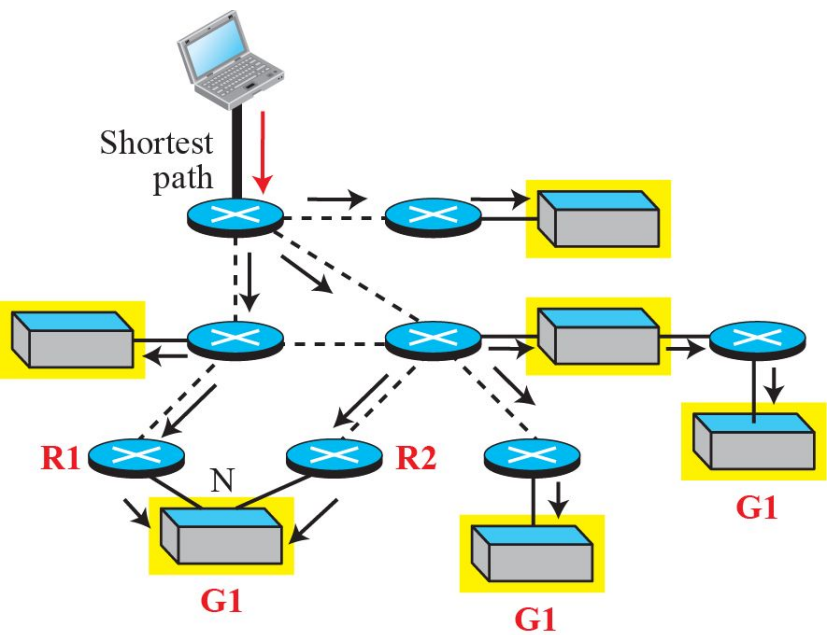
Multicast Distance Vector (DVMRP)

- ◆ *Reverse Path Forwarding (RPF)*
- ◆ *Reverse Path Broadcasting (RPB)*
- ◆ *Reverse Path Multicasting (RPM)*

Multicast Link State (MOSPF)

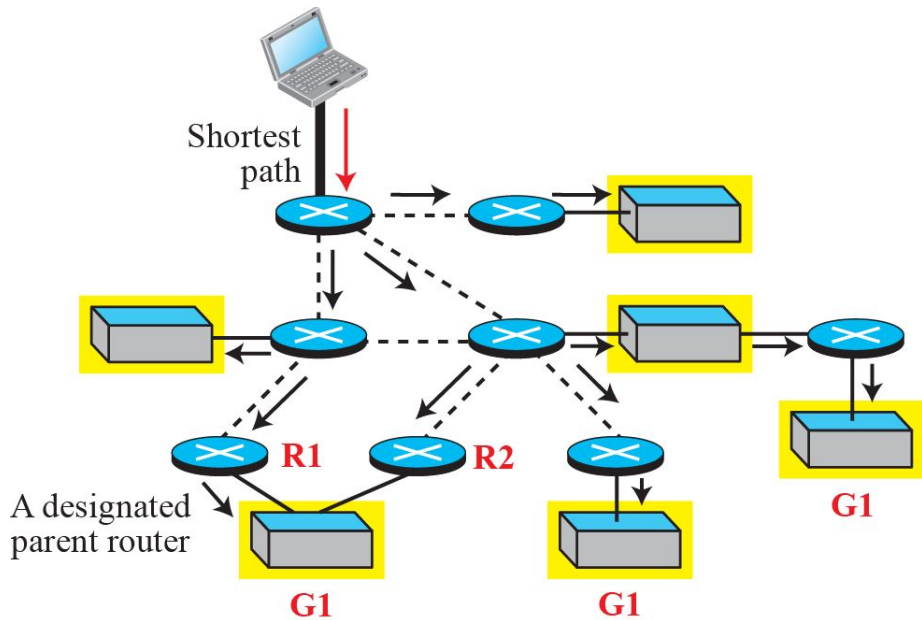
Protocol Independent Multicast (PIM)

Figure 4.96: RPF versus RPB



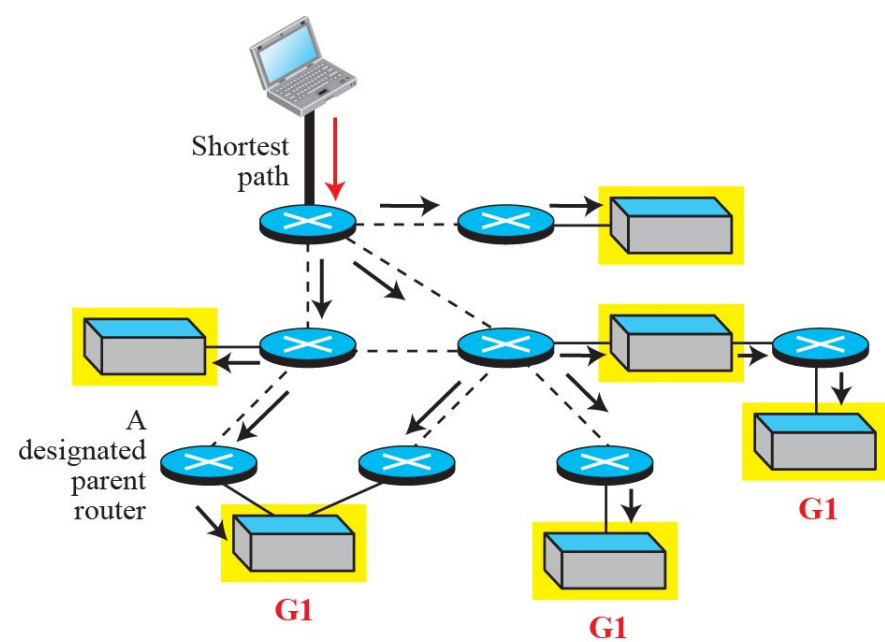
a. Using RPF, N receives two copies.

→ Packet received from the source
→ Copy of packet propagated



b. Using RPB, N receives only one copy.

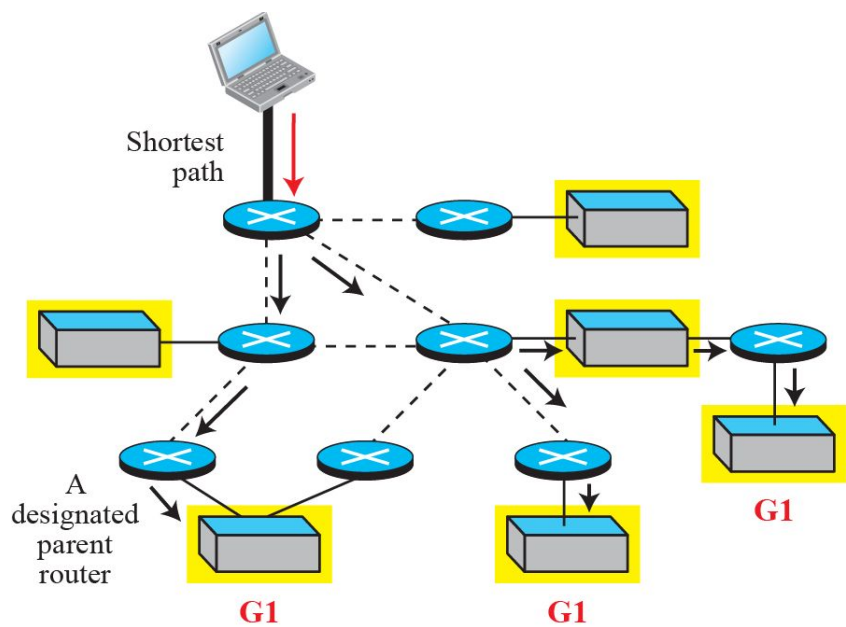
Figure 4.97: RPB versus RPM



a. Using RPB, all networks receive a copy.

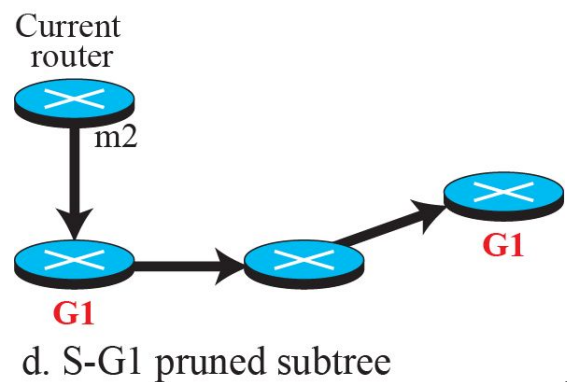
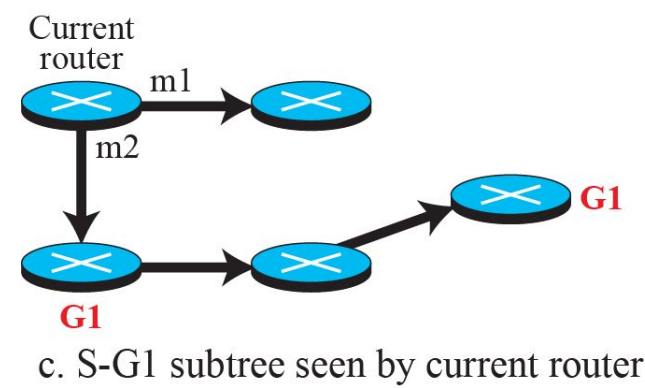
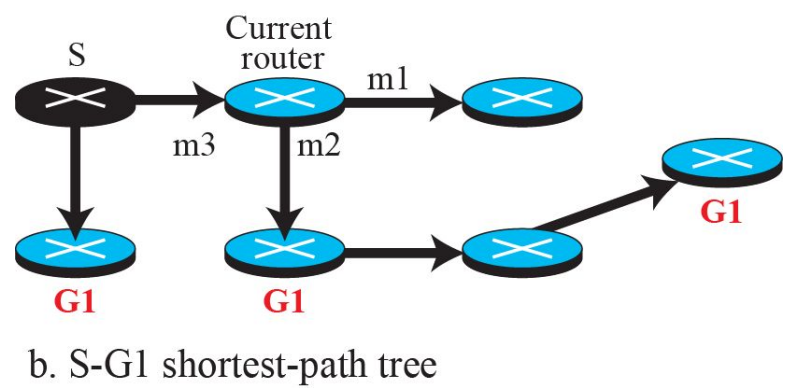
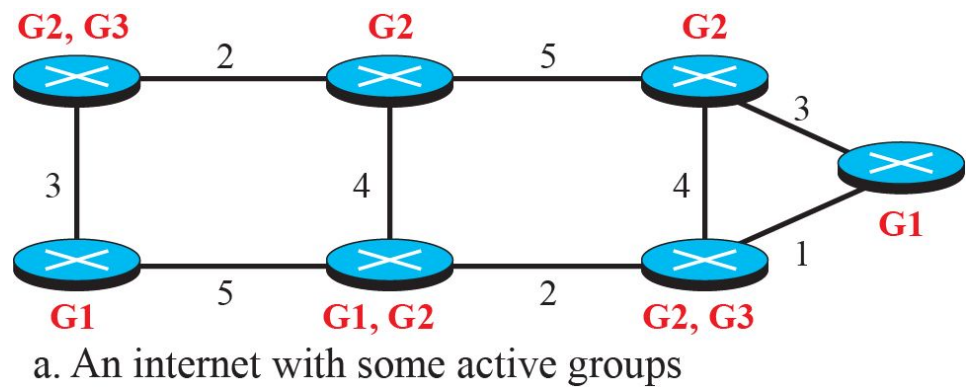
Packet received from the source

Copy of packet propagated



b. Using RPM, only members receive a copy.

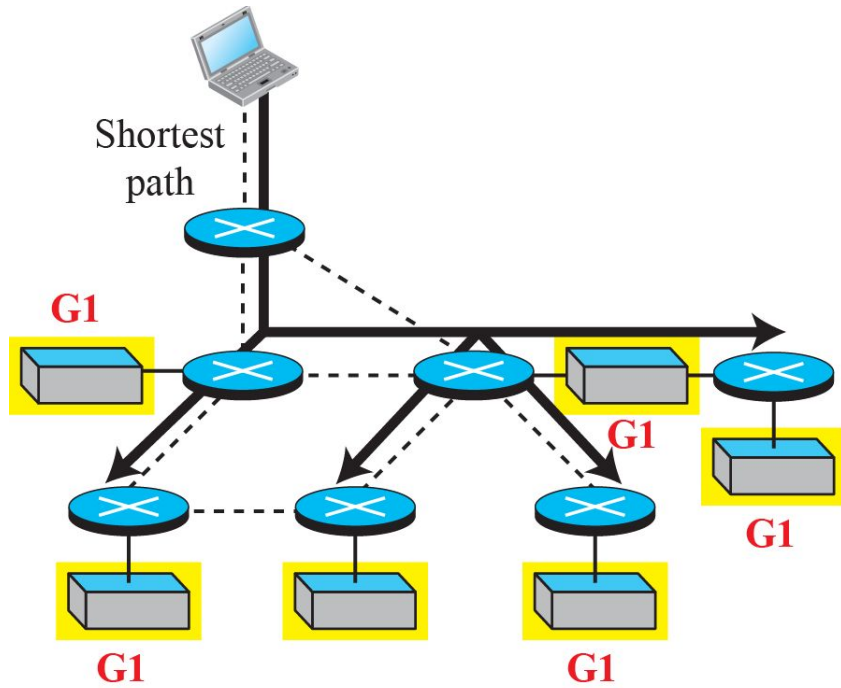
Figure 4.98: Example of tree formation in MOSPF



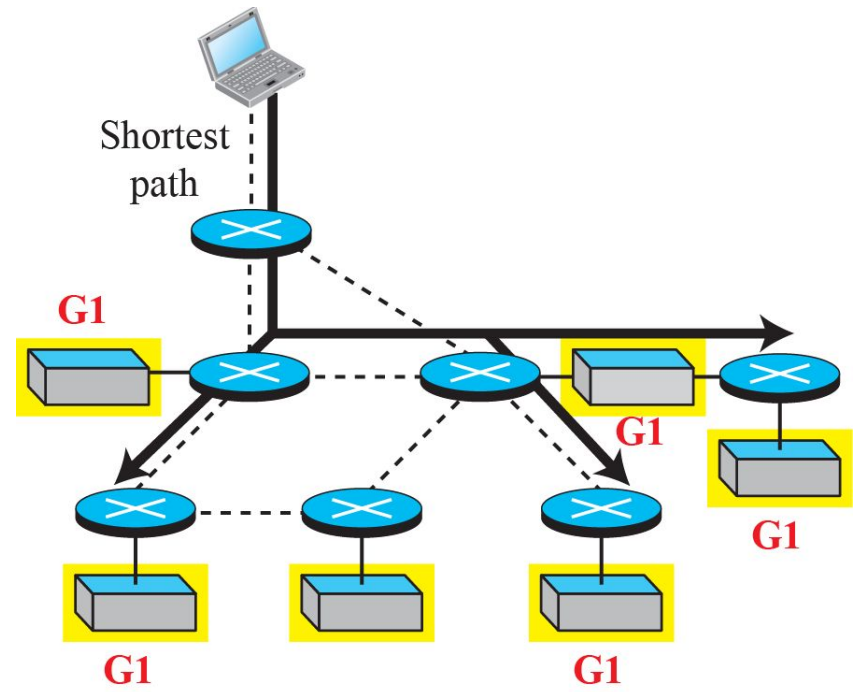
Forwarding table
for current router

Group-Source	Interface
S, G1	m2
...	...

Figure 4.99: Idea behind PIM-DM

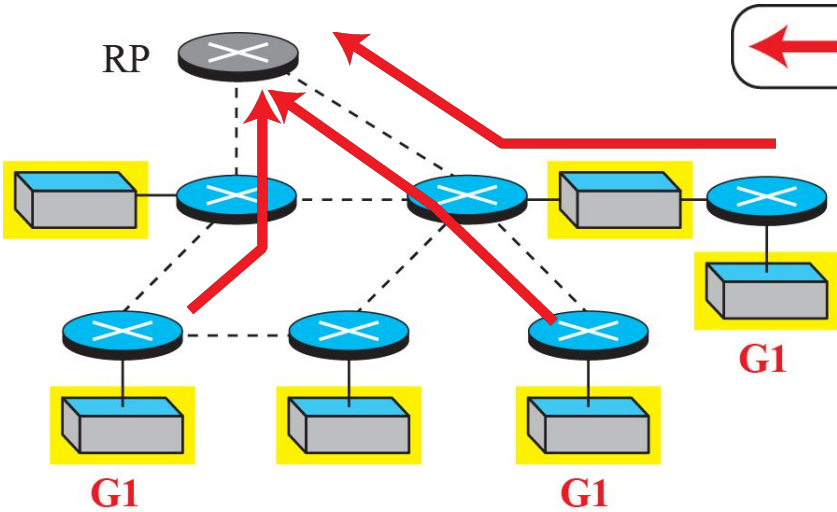


a. First packet is broadcast.

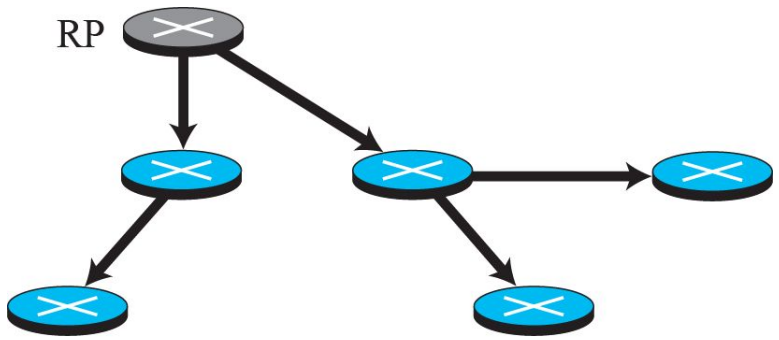


b. Second packet is multicast.

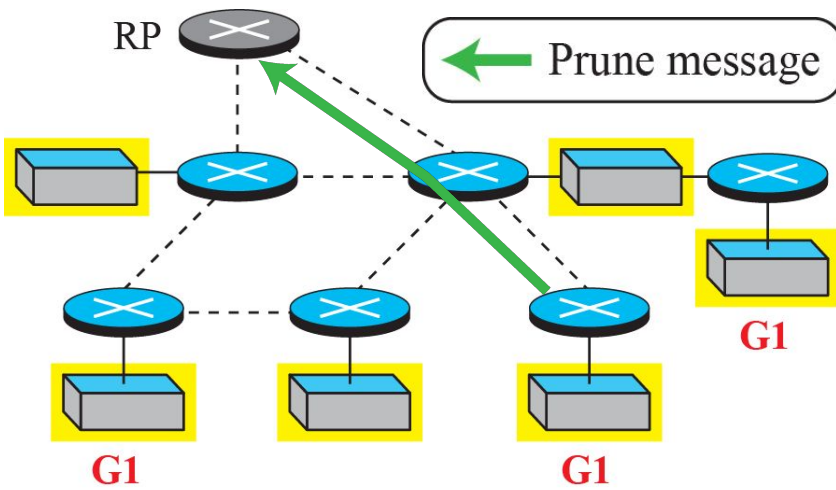
Figure 4.100: Idea behind PIM-SM



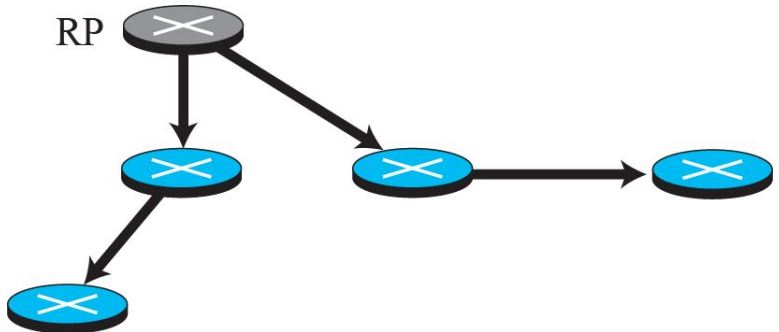
a. Three networks join group G1



b. Multicast tree after joins



c. One network leaves group G1



d. Multicast tree after pruning



4.4.4 Interdomain Routing Protocols

The three protocols we discussed for multicast routing, DVMRP, MOSPF, and PIM, are designed to provide multicast communication inside an autonomous system. When the members of the groups are spread among different domains (ASs), we need an interdomain multicast routing protocol. One common protocol for interdomain multicast routing is called Multicast Border Gateway Protocol (MBGP), which is the extension of BGP protocol we discussed for interdomain unicast routing. MBGP provides two paths between ASs: one for unicasting and one for multicasting.

4-5 NEXT GENERATION IP

The address depletion of IPv4 and other shortcomings of this protocol prompted a new version of IP protocol in the early 1990s. The new version, which is called Internet Protocol version 6 (IPv6) or IP new generation (IPng) was a proposal to augment the address space of IPv4 and at the same time redesign the format of the IP packet and revise some auxiliary protocols such as ICMP.



4.5.1 Packet Format

The IPv6 packet is shown in Figure 4.101. Each packet is composed of a base header followed by the payload. The base header occupies 40 bytes, whereas payload can be up to 65,535 bytes of information.

- ☐ *Concept of Flow and Priority in IPv6*
- ☐ *Fragmentation and Reassembly*
- ☐ *Extension Headers*

Figure 4.101: IPv6 datagram

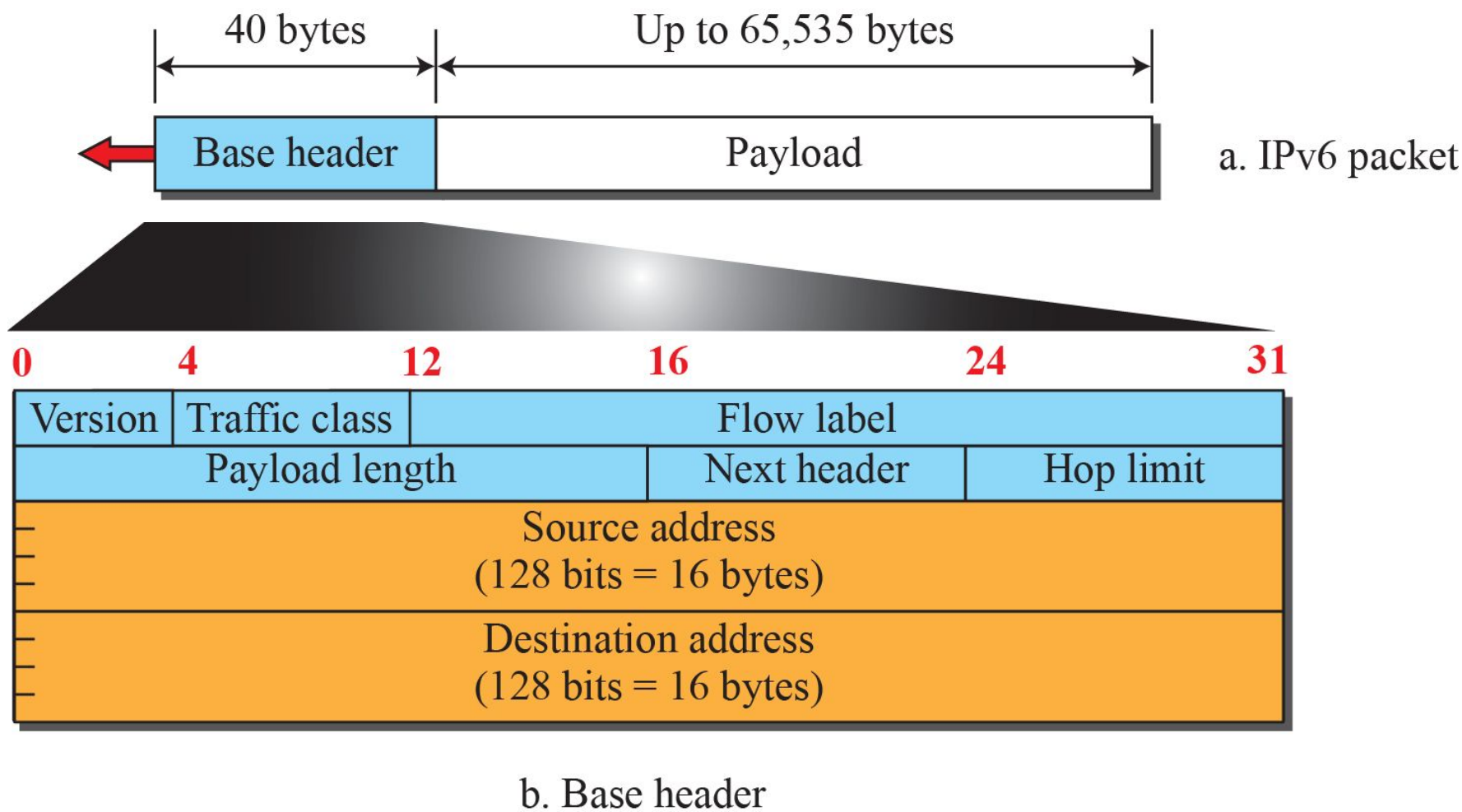
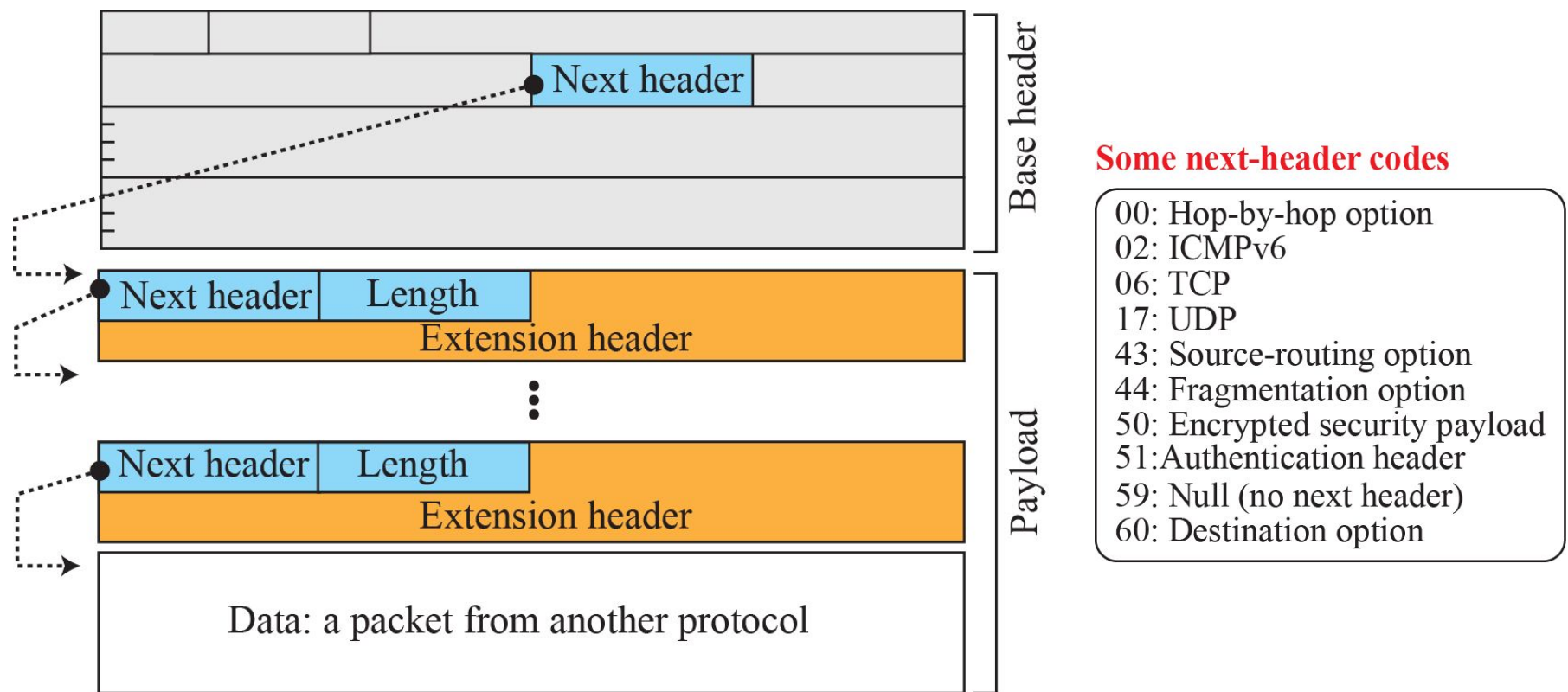
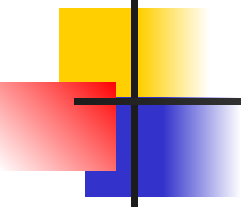


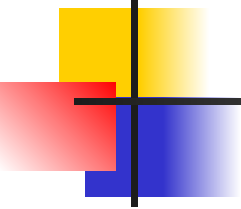
Figure 4.102: Payload in an IPv6 datagram





4.5.2 IPv6 Addressing

The main reason for migration from IPv4 to IPv6 is the small size of the address space in IPv4. In this section, we show how the huge address space of IPv6 prevents address depletion in the future. We also discuss how the new addressing responds to some problems in the IPv4 addressing mechanism. An IPv6 address is 128 bits or 16 bytes (octets) long, four times the address length in IPv4.



4.5.2 (continued)

- *Address Space*

- *Three Address Types*

- *Address Space Allocation*

- ◆ *Global Unicast Addresses*
- ◆ *Special Addresses*
- ◆ *Other Assigned Blocks*



Table 4.7: Prefixes for assigned IPv6 addresses

<i>Block prefix</i>	<i>CIDR</i>	<i>Block assignment</i>	<i>Fraction</i>
0000 0000	0000::/8	Special addresses	1/256
001	2000::/3	Global unicast	1/8
1111 110	FC00::/7	Unique local unicast	1/128
1111 1110 10	FE80::/10	Link local addresses	1/1024
1111 1111	FF00::/8	Multicast addresses	1/256

Figure 4.103: *Global unicast address*

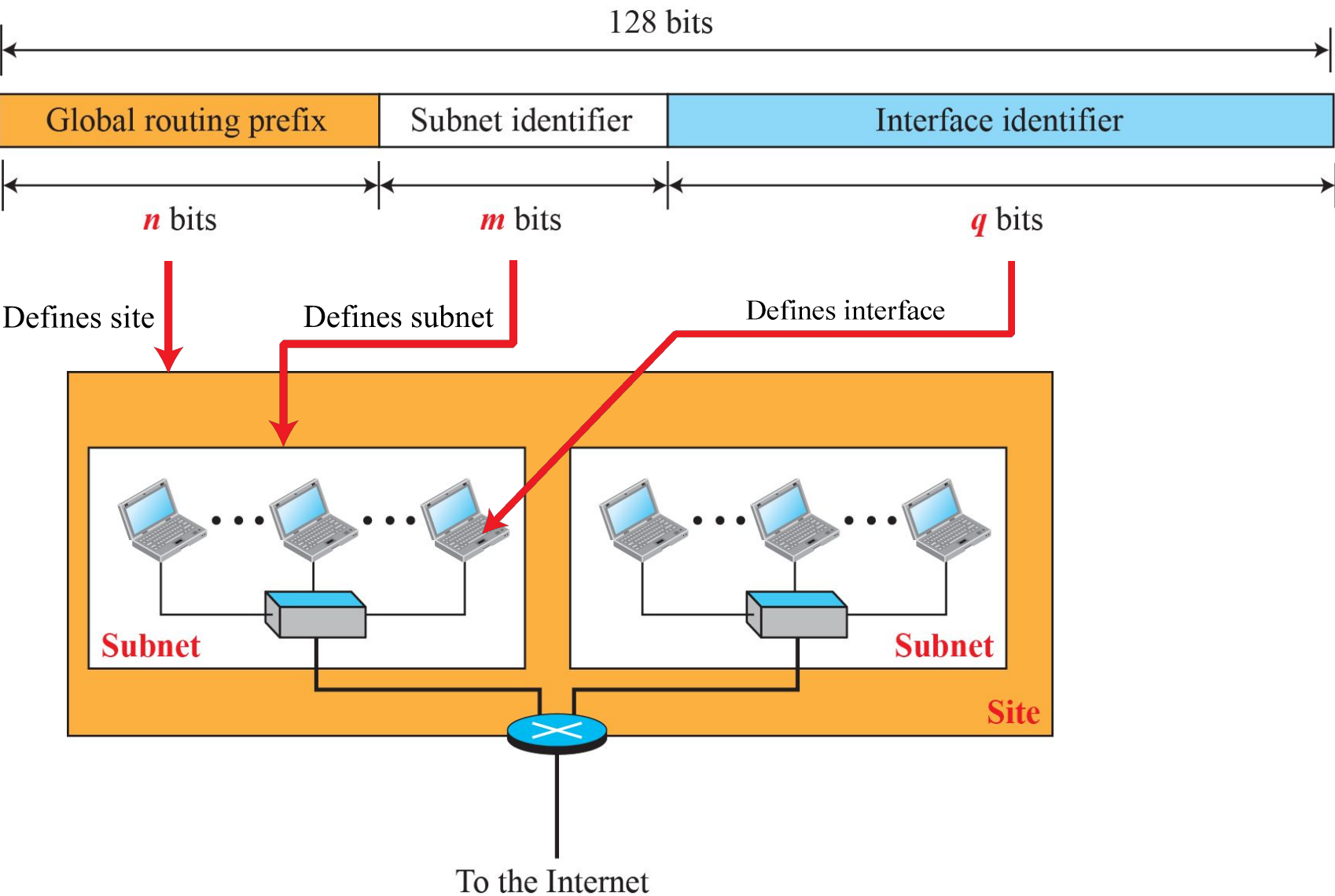


Figure 4.104: Special addresses

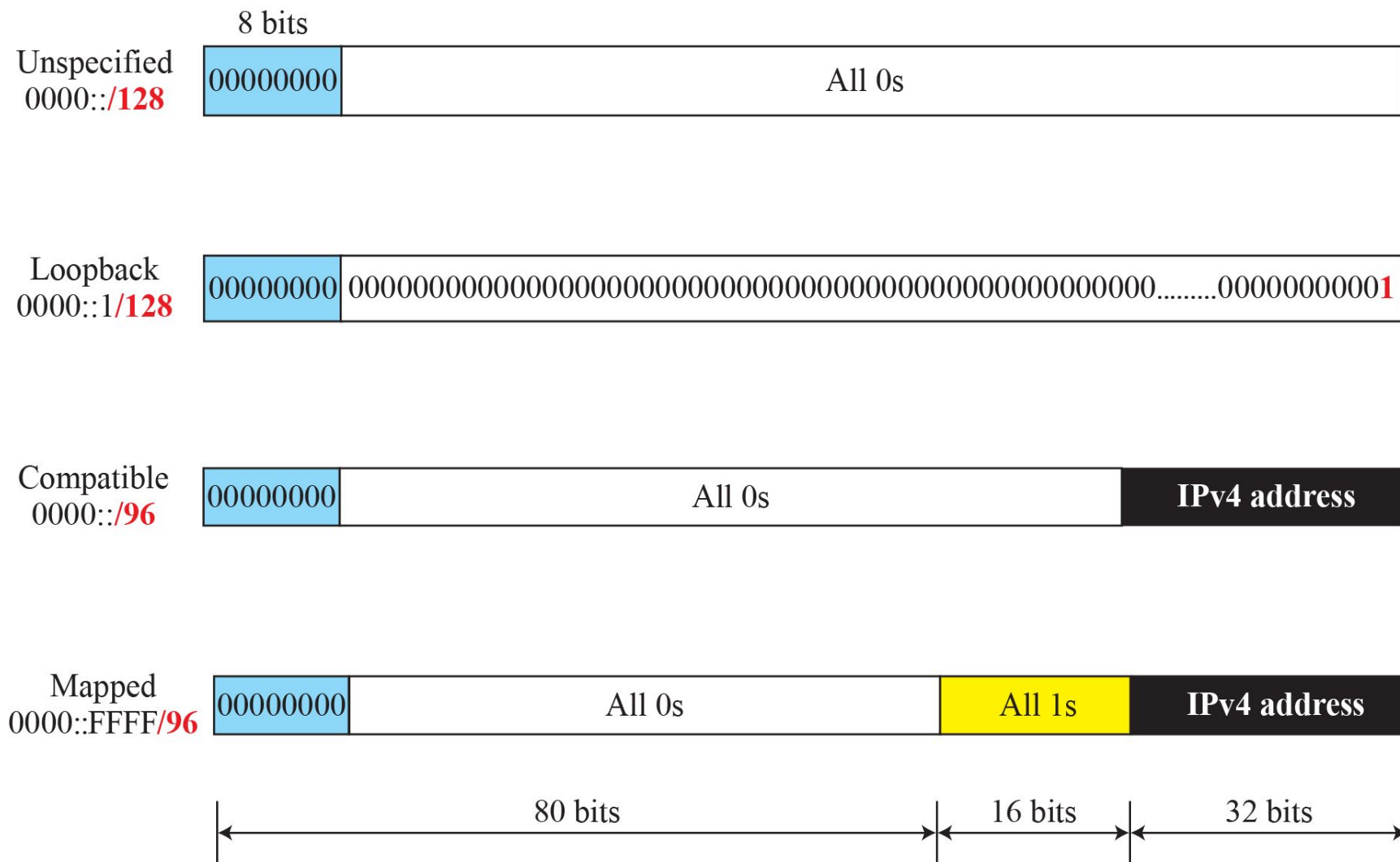
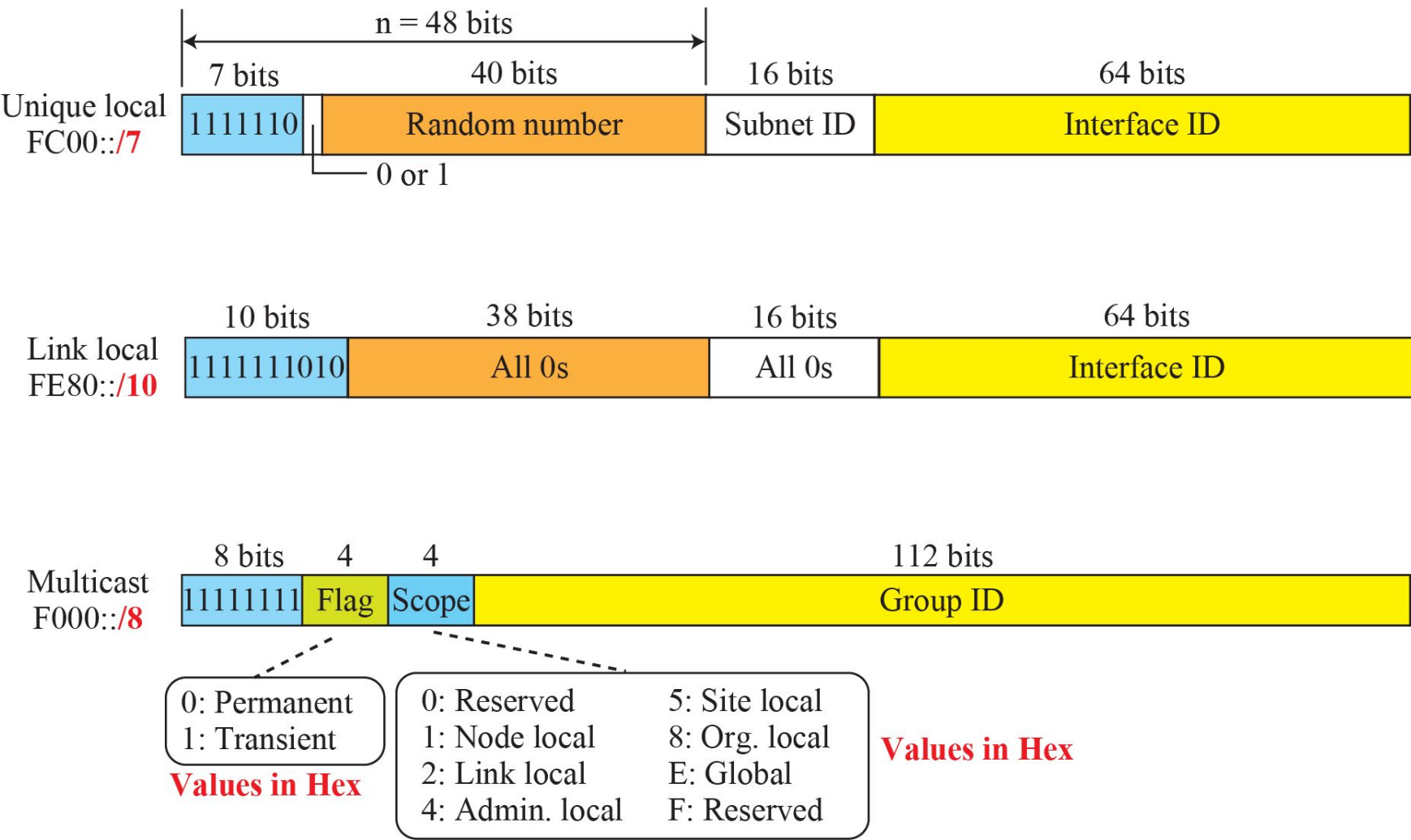
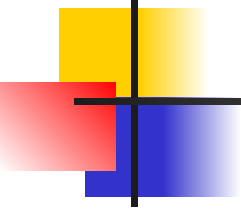


Figure 4.105: Unique local unicast block





4.5.3 *Transition from IPv4 to IPv6*

Although we have a new version of the IP protocol, how can we make the transition to stop using IPv4 and start using IPv6? The first solution that comes to mind is to define a transition day on which every host or router should stop using the old version and start using the new version.

Dual Stack

-  *Tunneling*
-  *Header Translation*

Figure 4.106: Dual stack

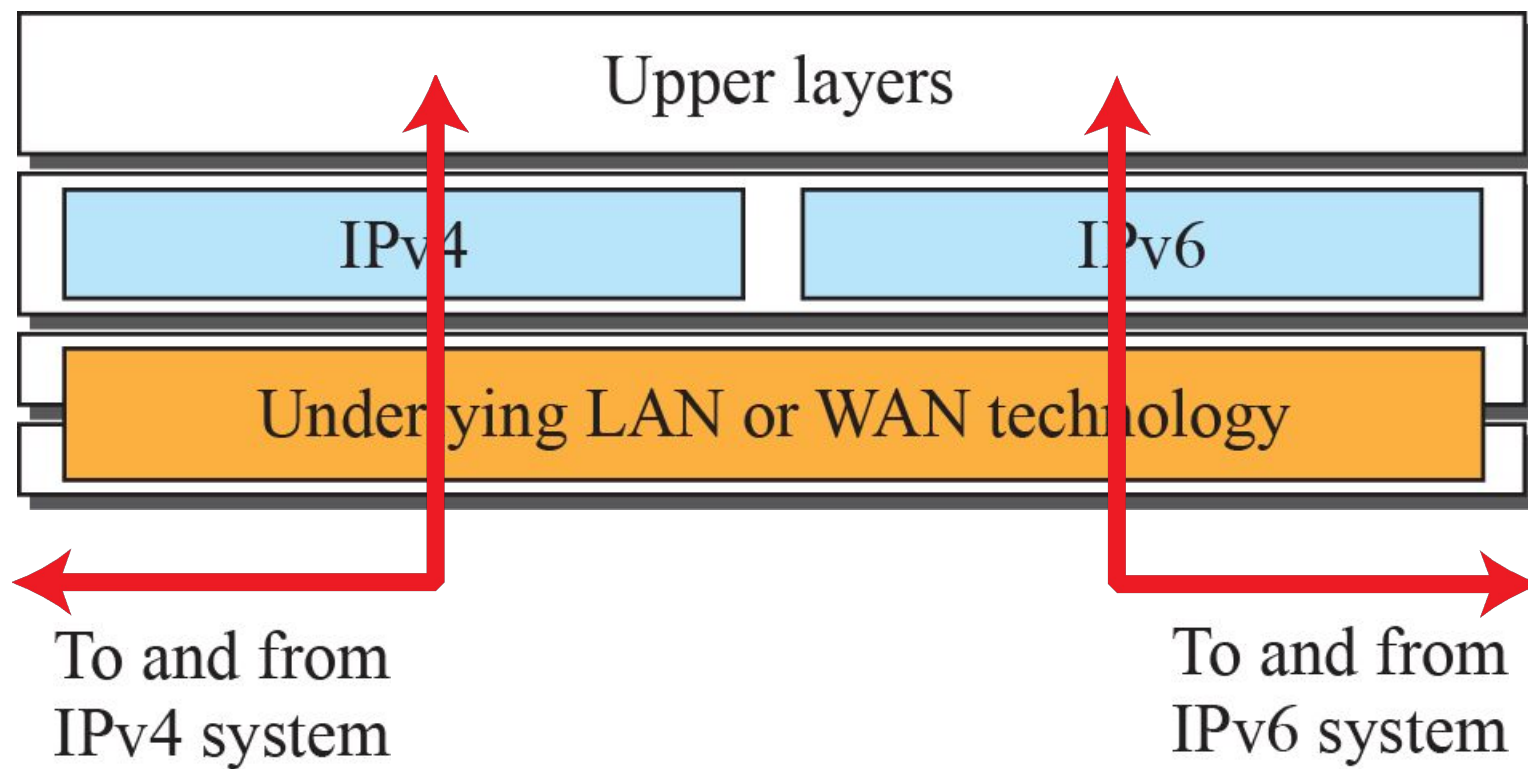


Figure 4.107: Tunneling strategy

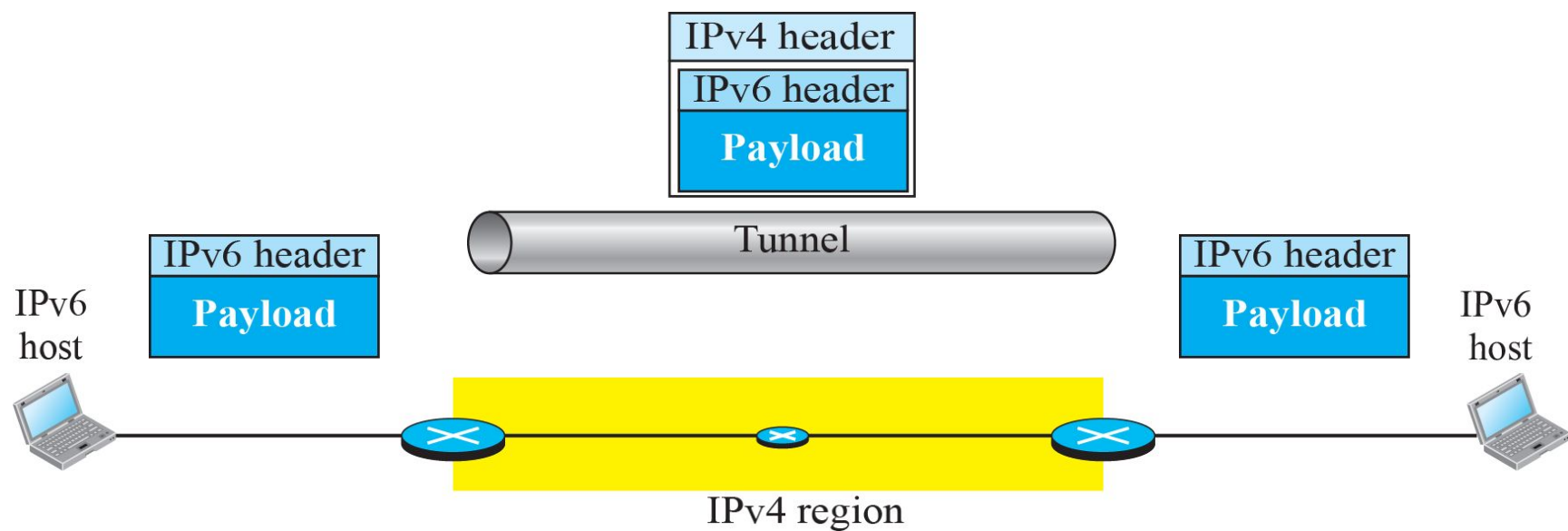


Figure 4.108: *Header translation strategy*

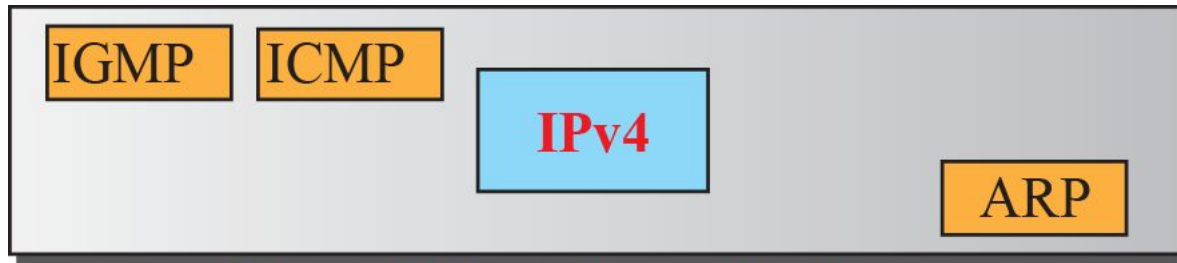




4.5.4 ICMPv6

Another protocol that has been modified in version 6 of the TCP/IP protocol suite is ICMP. ICMPv6 is more complicated than ICMPv4: some protocols that were independent in version 4 are now part of ICMPv6 and some new messages have been added to make it more useful. Figure 4.109 compares the network layer of version 4 to that of version 6. The ICMP, ARP (discussed in Chapter 5), and IGMP protocols in version 4 are combined into one single protocol, ICMPv6.

Figure 4.109: *Comparison of network layer in version 4 and version 6*

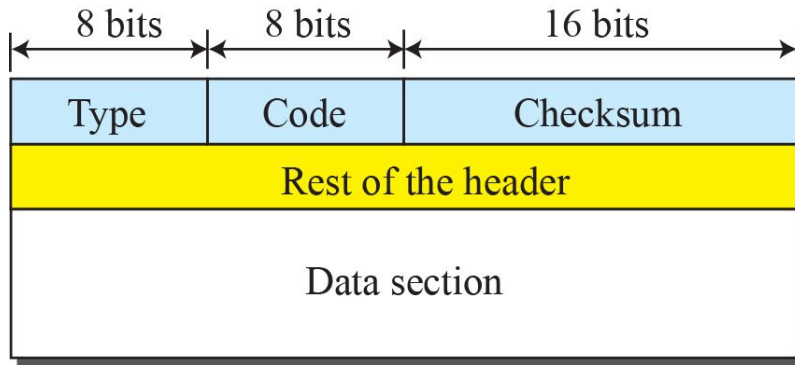


Network layer in version 4

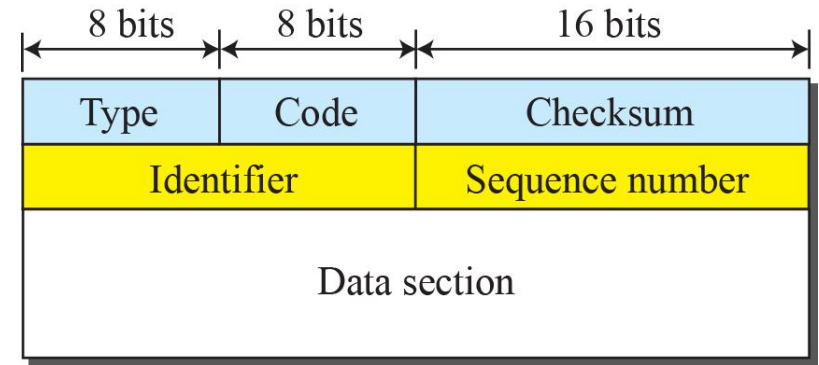


Network layer in version 6

Figure 4.110: ICMPv6 messages (Part I)



Error-reporting messages



Informational messages

Type and code values

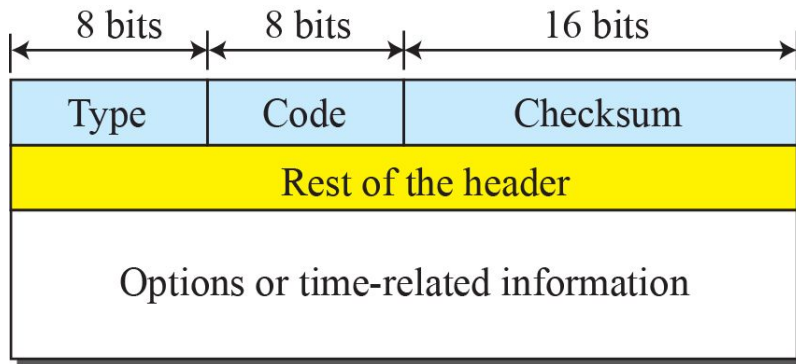
Error-reporting messages

- 01: Destination unreachable (codes 0 to 6)
- 02: Packet too big (code 0)
- 03: Time exceeded (codes 0 and 1)
- 04: Parameter problem (codes 0 to 2)

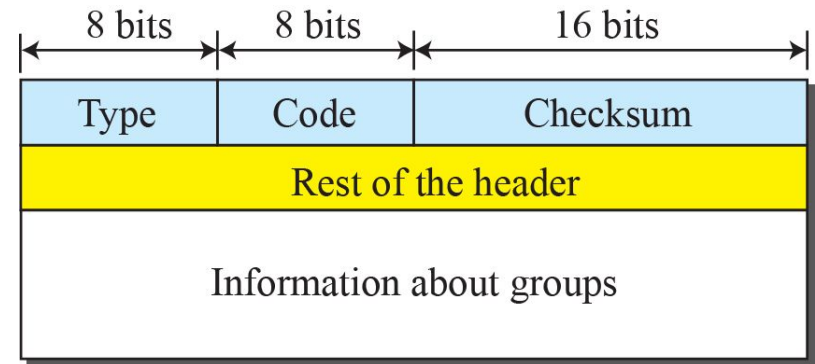
Informational messages

- 128 and 129: Echo request and reply (code 0)

Figure 4.110: ICMPv6 messages (Part II)



Neighbor-discovery messages



Group-membership messages

Type and code values

Neighbor-discovery messages

- 133 and 134: Router solicitation and advertisement (code 0)
- 135 and 136: Neighbor solicitation and advertisement (code 0)
- 137: Redirection (codes 0 to 3)
- 141 and 142: Inverse neighbor solicitation and advertisement (code 0)

Group-membership messages

- 130: Membership query (code 0)
- 131: Membership report

Chapter 4: Summary

- ❑ *The Internet is made of many networks (or links) connected through connecting devices, each of which acts as a router or as a switch. Two types of switching are traditionally used in networking: circuit switching and packet switching. The network layer is designed as a packet-switched network.*
- ❑ *The network layer supervises the handling of packets by the underlying physical networks. The delivery of a packet can be direct or indirect. Two categories of forwarding are defined: forwarding based on the destination address of the IP datagram and forwarding based on the label attached to an IP datagram.*

Chapter 4: Summary (continued)

- ❑ *IPv4 is an unreliable connectionless protocol responsible for source-to-destination delivery. Packets in the IP layer are called datagrams. The identifiers used in the IP layer of the TCP/IP protocol suite are called the IP addresses. An IPv4 address is 32 bits long, divided into two parts: the prefix and the suffix. All addresses in the block have the same prefix; each address has a different suffix.*
- ❑ *The Internet Control Message Protocol (ICMP) supports the unreliable and connectionless Internet Protocol (IP).*
- ❑ *To be able to route a packet, a router needs a forwarding table. Routing protocols are specific application programs that have the duty of updating forwarding tables.*

Chapter 4: Summary (continued)

- ❑ *Multicasting is the sending of the same message to more than one receiver simultaneously. The Internet Group Management Protocol (IGMP) is involved in collecting local membership group information.*
- ❑ *IPv6, the latest version of the Internet Protocol, has a 128-bit address space. IPv6 uses hexadecimal colon notation with abbreviation methods available.*